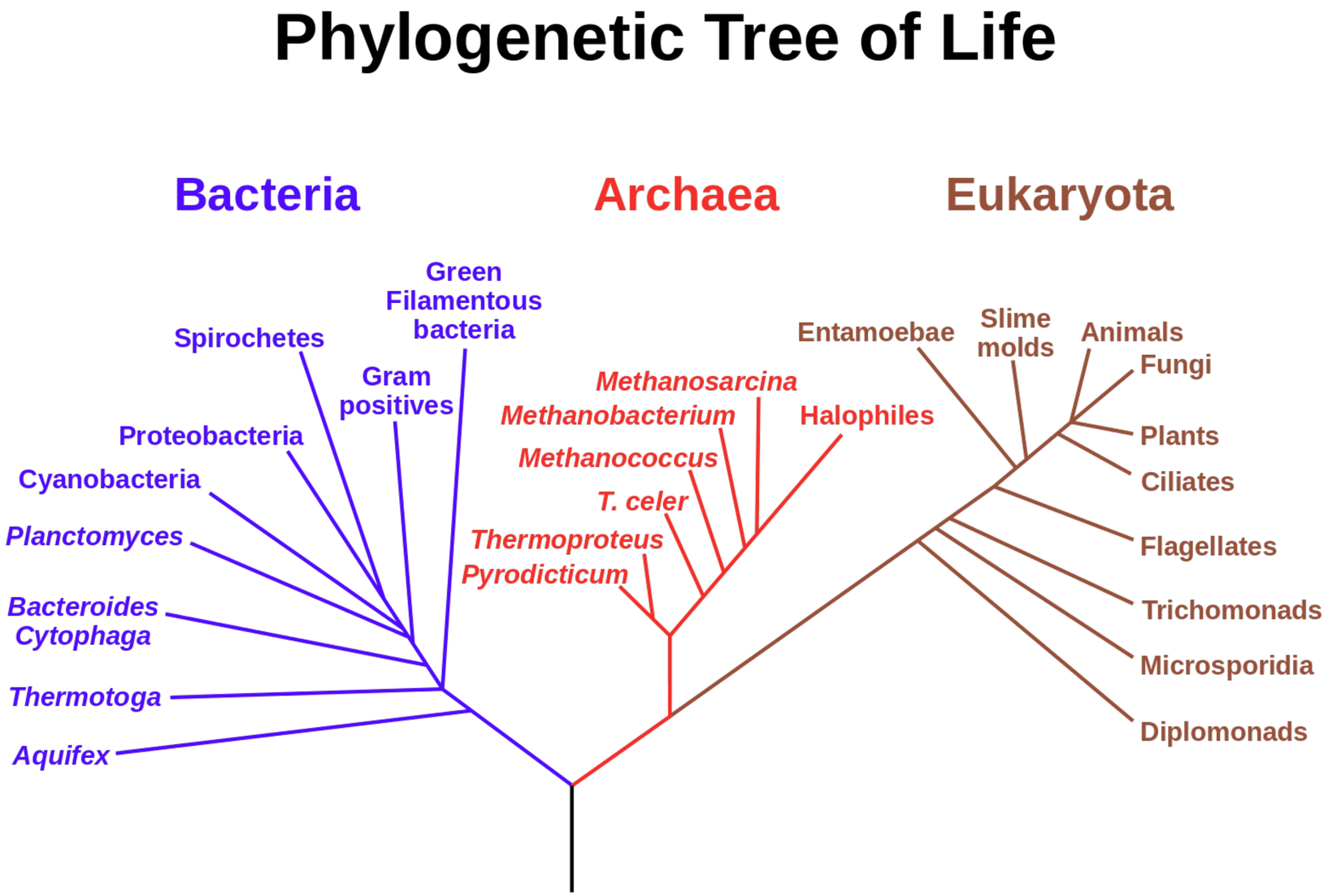
Sequence similarity and alignment

UNIVERSITY OF MARYLAND

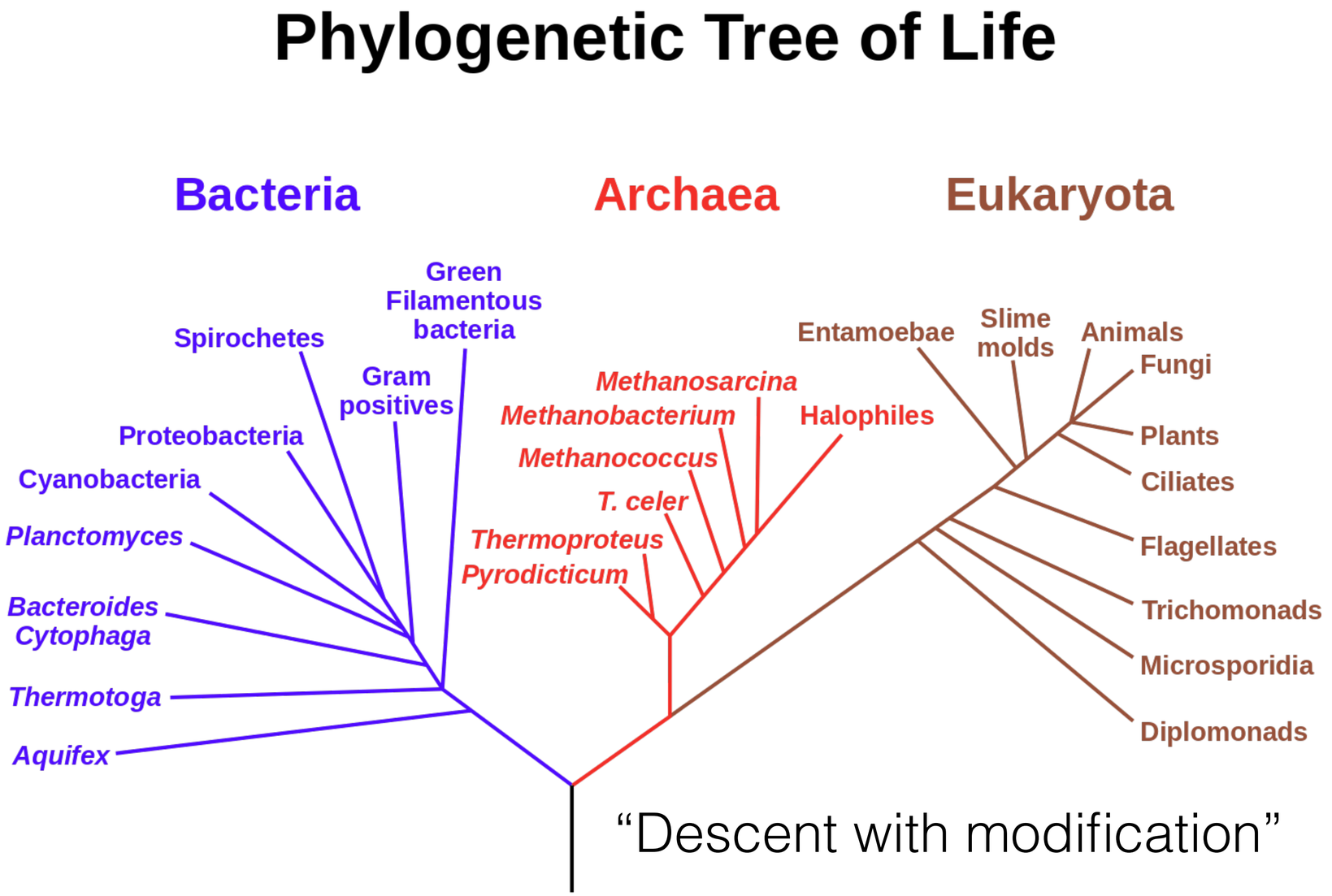
slides (w/*) courtesy of Carl Kingsford

Relatedness of Biological Sequence



https://en.wikipedia.org/wiki/Phylogenetic_tree

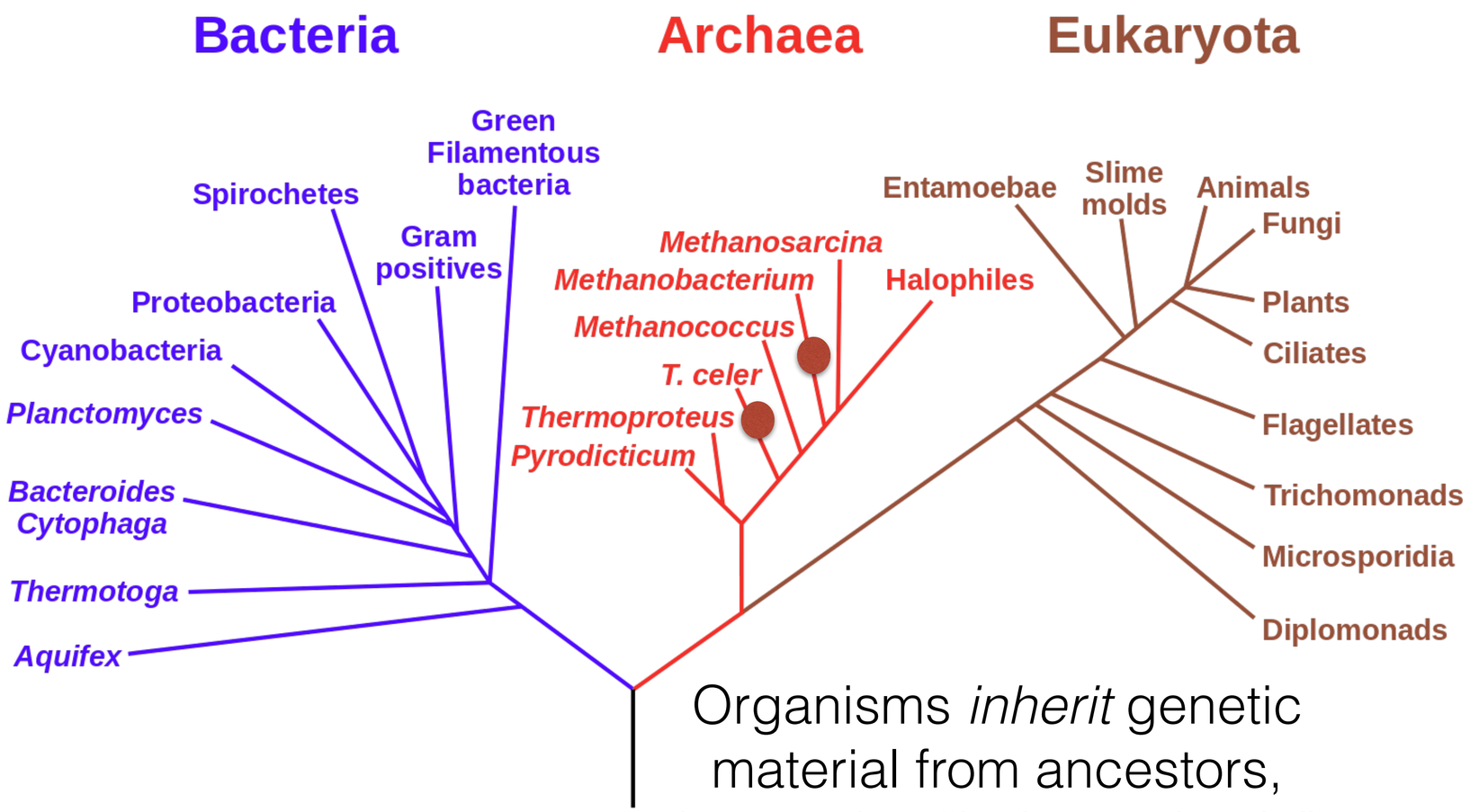
Relatedness of Biological Sequence



"Descent with modification"

https://en.wikipedia.org/wiki/Phylogenetic_tree

Relatedness of Biological Sequence

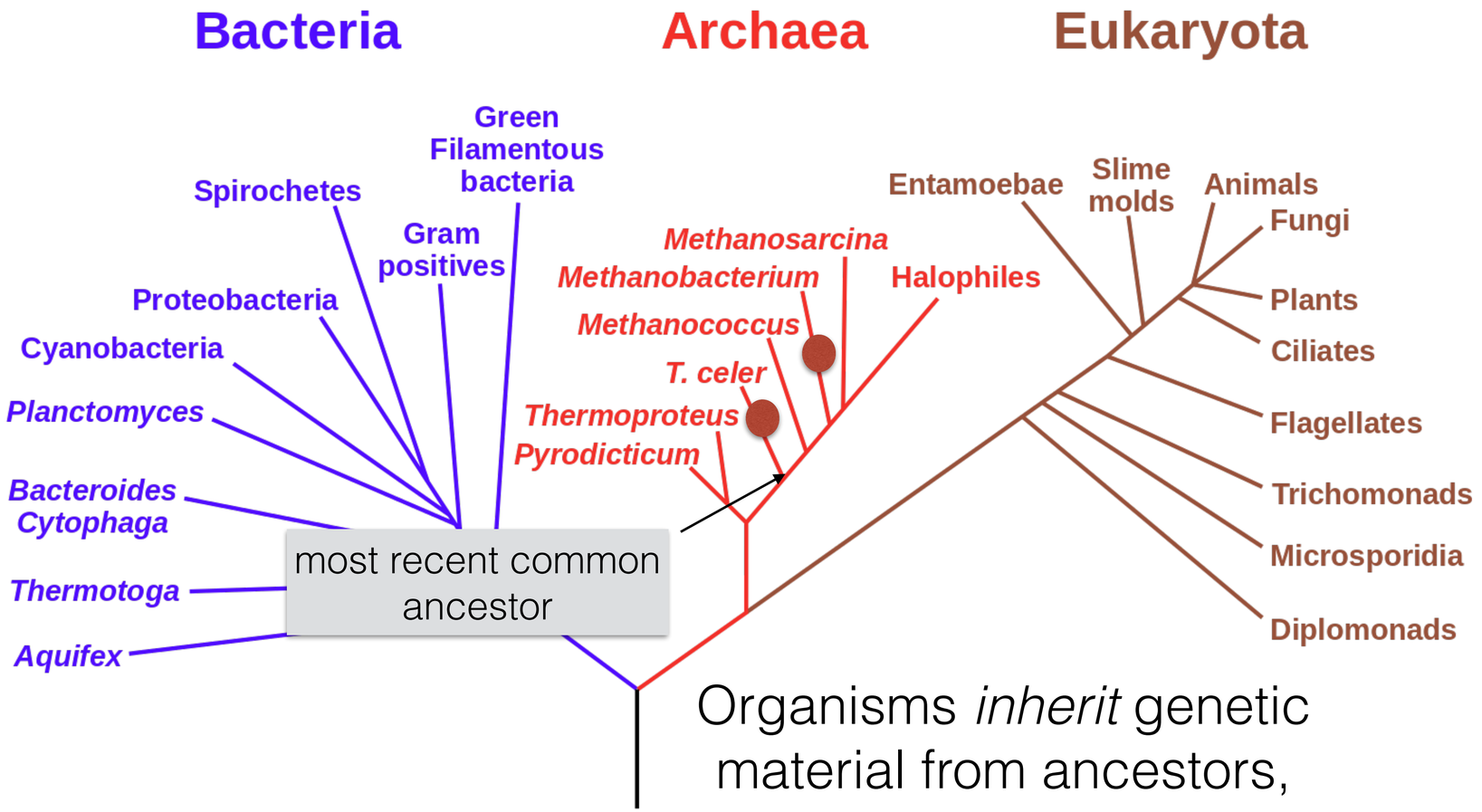


Organisms *inherit* genetic material from ancestors, but evolve "independently"

https://en.wikipedia.org/wiki/Phylogenetic_tree

Relatedness of Biological Sequence

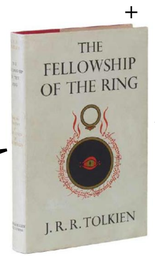
Phylogenetic Tree of Life



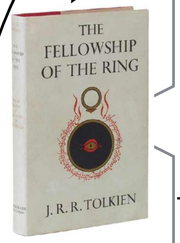
Organisms *inherit* genetic material from ancestors, but evolve "independently"

https://en.wikipedia.org/wiki/Phylogenetic_tree

Consider an analogy

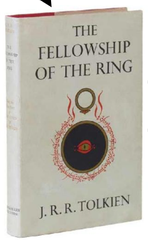


"When Mr. Bilbo Baggins of Bag End announced that he would shortly be celebrating his eleventy-first birthday with a party of special magnificence, there was much talk and excitement in Hobbiton"



"When Mr. Bilbo Baggins of Bag End announced that he would shortly be celebrating his **eleventh**-first birthday with a party of special magnificence, there was much talk and excitement in Hobbiton"

"When Mr. Bilbo **Baggens** of Bag End announced that he would shortly be celebrating his **eleventh**-first birthday with a party of special magnificence, there was much talk and excitement in Hobbiton"



"When Mr. Bilbo Baggins of Bag End announced that he would shortly be celebrating his eleventh-first birthday with a party of special magnificence, there was much talk and excitement in **Hobbit-town**"

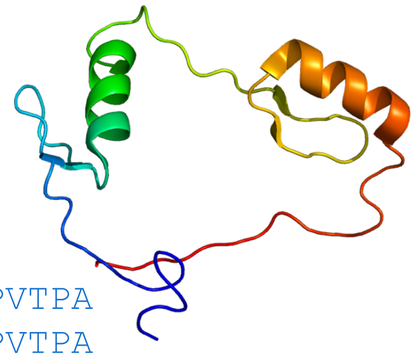
"When **Mrs**. Bilbo Baggins of Bag End announced that **she** would shortly be celebrating his eleventh-first birthday with a party of special magnificence, there was much talk and excitement in **Hobbit-town**"

+:https://en.wikipedia.org/wiki/The_Fellowship_of_the_Ring

Sequence tells a story

- If two sequences are *similar*, this provides evidence of descent from a common ancestor
- Sequences are *conserved* at different rates
- Very similar sequence can indicate a very *recent common ancestor*, or a *highly conserved function*

Why compare DNA or protein sequences?



Partial CTCF protein sequence in 8 organisms:

```
H. sapiens        -EDSSDS-ENAEPDLDDNEDEEEPAVEIEPEPE----------PQPVTPA
P. troglodytes    -EDSSDS-ENAEPDLDDNEDEEEPAVEIEPEPE----------PQPVTPA
C. lupus          -EDSSDS-ENAEPDLDDNEDEEEPAVEIEPEPE----------PQPVTPA
B. taurus         -EDSSDS-ENAEPDLDDNEDEEEPAVEIEPEPE----------PQPVTPA
M. musculus       -EDSSDSEENAEPDLDDNEEEEPAVEIEPEPE--PQPQPPPPPQPVAPA
R. norvegicus     -EDSSDS-ENAEPDLDDNEEEEPAVEIEPEPEPQPQPQPQPQPQPVAPA
G. gallus         -EDSSDSEENAEPDLDDNEDEEETAVEIEAEPE----------VSAEAPA
D. rerio          DDDDDDSDEHGEPDLDDIDEEDEDDL-LDEDQMGLLDQAPPSVPIP-APA
```

- Identify important sequences by finding conserved regions.
- Find genes similar to known genes.
- Understand evolutionary relationships and distances (D. rerio aka zebrafish is farther from humans than G. gallus aka chicken).
- Interface to databases of genetic sequences.
- As a step in genome assembly, and other sequence analysis tasks.
- Provide hints about protein structure and function (next slides).

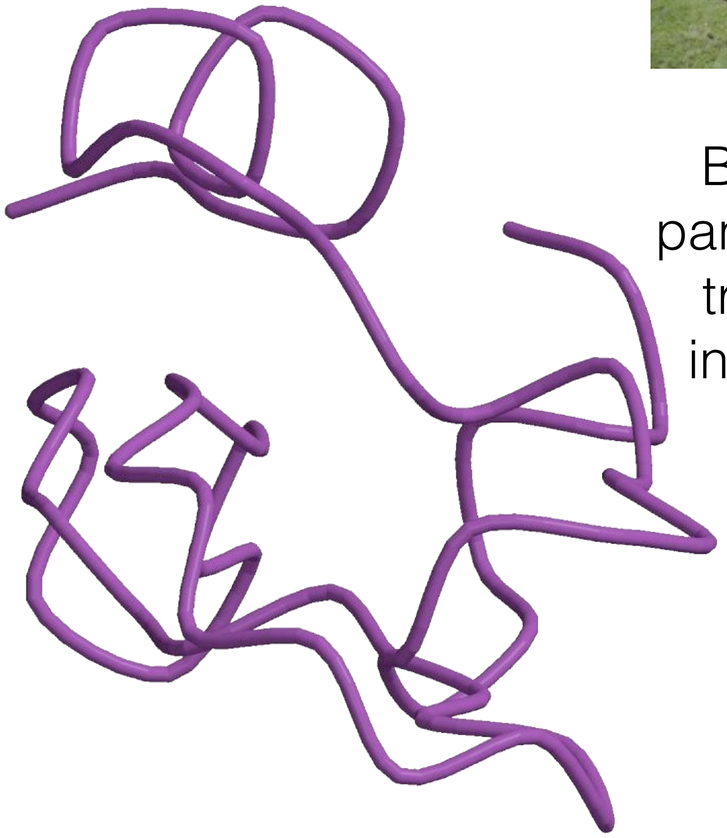
*

Sequence can reveal structure

dendrotoxin K

(a) 1dtk

Bovine pancreatic trypsin inhibitor



(b) 5pti

```
1dtk   XAKYCKLPLRIGPCKRKIPSFYYKWKAKQCLPFDYSGCGGNANRFKTIEECRRTCVG-
5pti   RPDFCLEPPYTGPCKARIIRYFYNAKAGLCQTFVYGGCRAKRNNFKSAEDCMRTCGGA
```

*

Why Not Exact Matching?

Suffix tree / array and BWT / FM-index are powerful tools for finding exact patterns in a large text, but exact matching is insufficient. Reads have **errors** and there is **true genomic variation** between a reference and a sample.

Typical strategy (many variants):

- Find all places where a substring of the query matches the reference exactly (seeds)
- Filter out regions with insufficient exact matches to warrant further investigation
- Perform a “constrained” alignment that includes these exact matching “seeds”

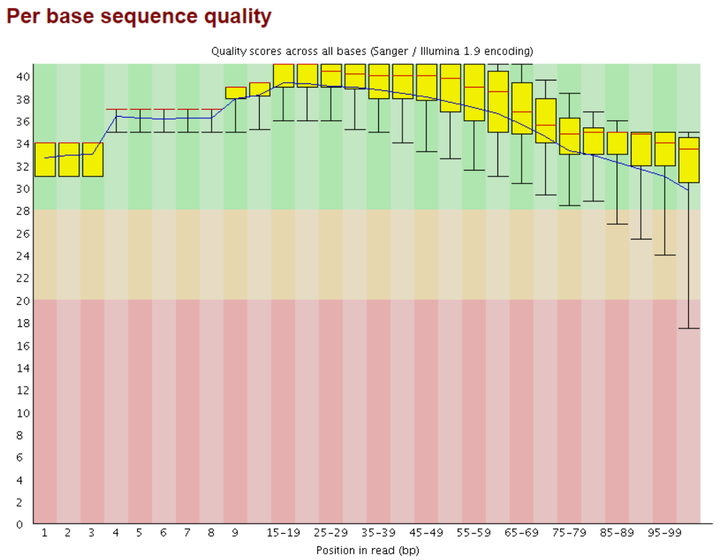
Requires efficient exact search

Here is where we use our alignment DPs

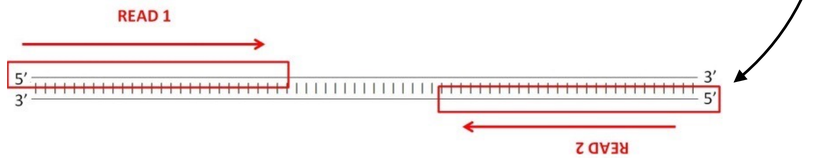
Why Is This Possible?

This is (*usually*) a **heuristic** (doesn't guarantee you find all alignment locations within the budget for a read).

But, due to the error profiles of reads, this often works well.



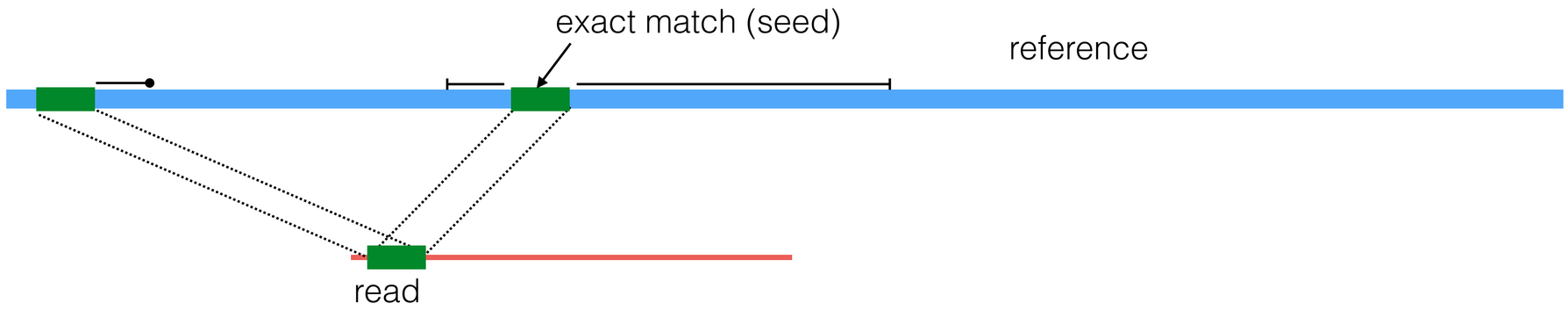
	error type	error rate	read length
Illumina	subst.	~0.1%	50-300
Nanopore	indel	10-30%	5-10kb
Pac Bio	indel	10-15%	10-15kb



2nd generation reads are often "paired-end"

Typical Strategy

Seed & Extend:



The Language of Strings

A string **s** is a finite sequence of characters

|**s**| denotes the length of the string — the number of characters in the sequence.

A string is defined *over* an alphabet, Σ

Σ_{DNA} = {A,T,C,G}
Σ_{RNA} = {A,U,C,G}
$\Sigma_{AminoAcid}$ = {A, R, N, D, C, E, Q, G, H, I, L, K, M, F, P, S, T, W, Y, V}

The empty string is denoted ϵ — |ϵ| = 0

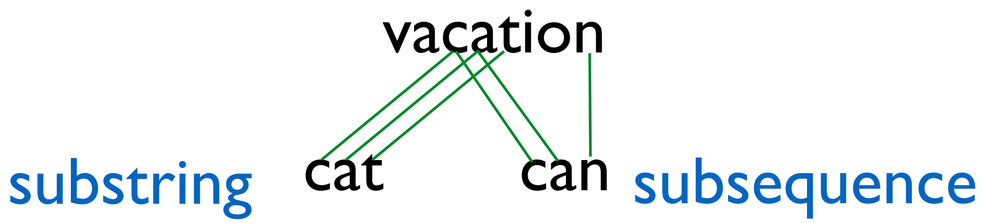
adapted from a slide by Ben Langmead

The Language of Strings

Given two strings **s,t** over the same alphabet Σ, we denote the concatenation as **st** — this is the sequence of **s** followed by the sequence of **t**

String **s** is a substring of **t** if there exist two (potentially empty) strings **u** and **v** such that **t = usv**

String **s** is a subsequence of **t** if the characters of **s** appear in order (but not nece consecutively) in **t**



String **s** is a prefix/suffix of **t** if **t** = **su/us** — if neither **s** nor **u** are ϵ, then **s** is a prefix/suffix of **t**

adapted from a slide by Ben Langmead

The Simplest String Comparison Problem

Given: Two strings

$$a = a_1 a_2 a_3 a_4 \ldots a_m$$
$$b = b_1 b_2 b_3 b_4 \ldots b_n$$

where a_i, b_i are letters from some alphabet, Σ, like {A,C,G,T}.

Compute how similar the two strings are.

What do we mean by "similar"?

Edit distance between strings a and b = the smallest number of the following operations that are needed to transform a into b:

- mutate (replace) a character
- delete a character
- insert a character

riddle $\xrightarrow{\text{delete}}$ ridle $\xrightarrow{\text{mutate}}$ riple $\xrightarrow{\text{insert}}$ triple

*

The String Alignment Problem

Parameters:

- "*gap*" is the cost of inserting a "-" character, representing an insertion or deletion (insertion/deletion are dual operations depending on the string)

- *cost*(*x*,*y*) is the cost of aligning character *x* with character *y*.
  In the simplest case, *cost*(*x*,*x*) = 0 and *cost*(*x*,*y*) = mismatch penalty.

Goal:

- Can compute the edit distance by finding the **lowest cost alignment**. (often phrased as finding **highest scoring alignment**.)

- Cost of an alignment is: sum of the *cost*(*x*,*y*) for the pairs of characters that are aligned + *gap* × number of - characters inserted.

e.g. *gap* = 3, *cost('D', 'P')* = 1

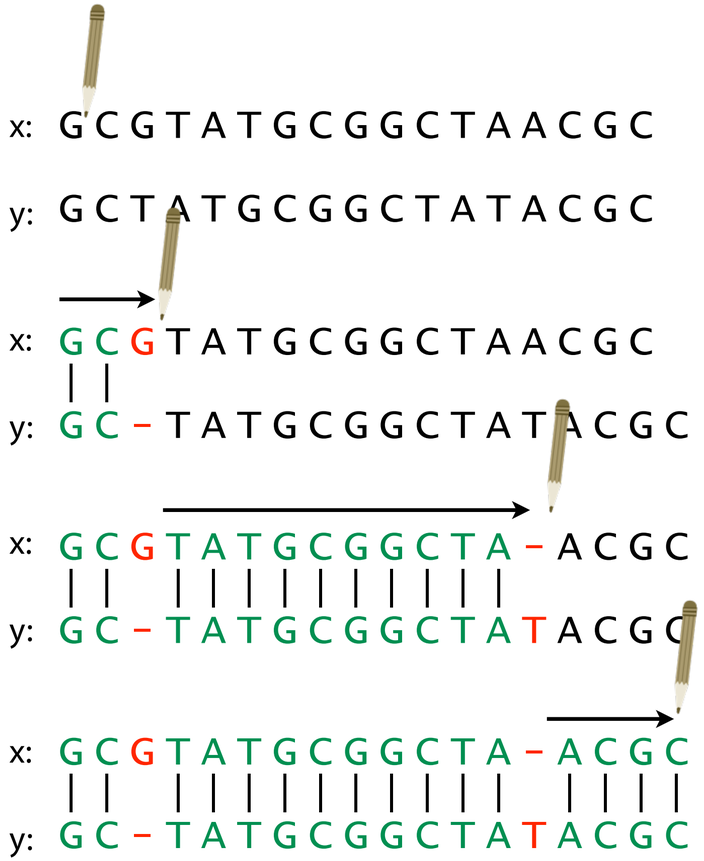
- RIDDLE
TRIP - LE

Total cost = 3+0+0+1+3+0+0 = 7

*

Representing alignments as edit transcripts

Can think of edits as being introduced by an *optimal editor* working left-to-right.
Edit transcript describes how editor turns *x* into *y*.



Operations:
M = match, **R** = replace,
I = insert into *x*, **D** = delete from *x*

MMD

MMDMMMMMMMMMMI

MMDMMMMMMMMMMIMMMM

Slide courtesy of Ben Langmead

Representing edits as alignments

```
prin-ciple
|||| |||XX
prinncipal
(1 gap, 2 mm)
MMMMIMMMRR
```

```
misspell
||| ||||
mis-pell
(1 gap)
MMMIMMMM
```

```
aa-bb-ccaabb
|X || | | |
ababbbc-a-b-
(5 gaps, 1 mm)
MRIMMIMDMDMD
```

```
prin-cip-le
|||| ||| |
prinncipal-
(3 gaps, 0 mm)
MMMMIMMMIMD
```

```
 prehistoric
    ||||||||
 ---historic
 (3 gaps)
 DDDMMMMMMMM
```

```
al-go-rithm-
|| XX ||X |
alKhwariz-mi
(4 gaps, 3 mm)
MMIRRIMMRDMI
```

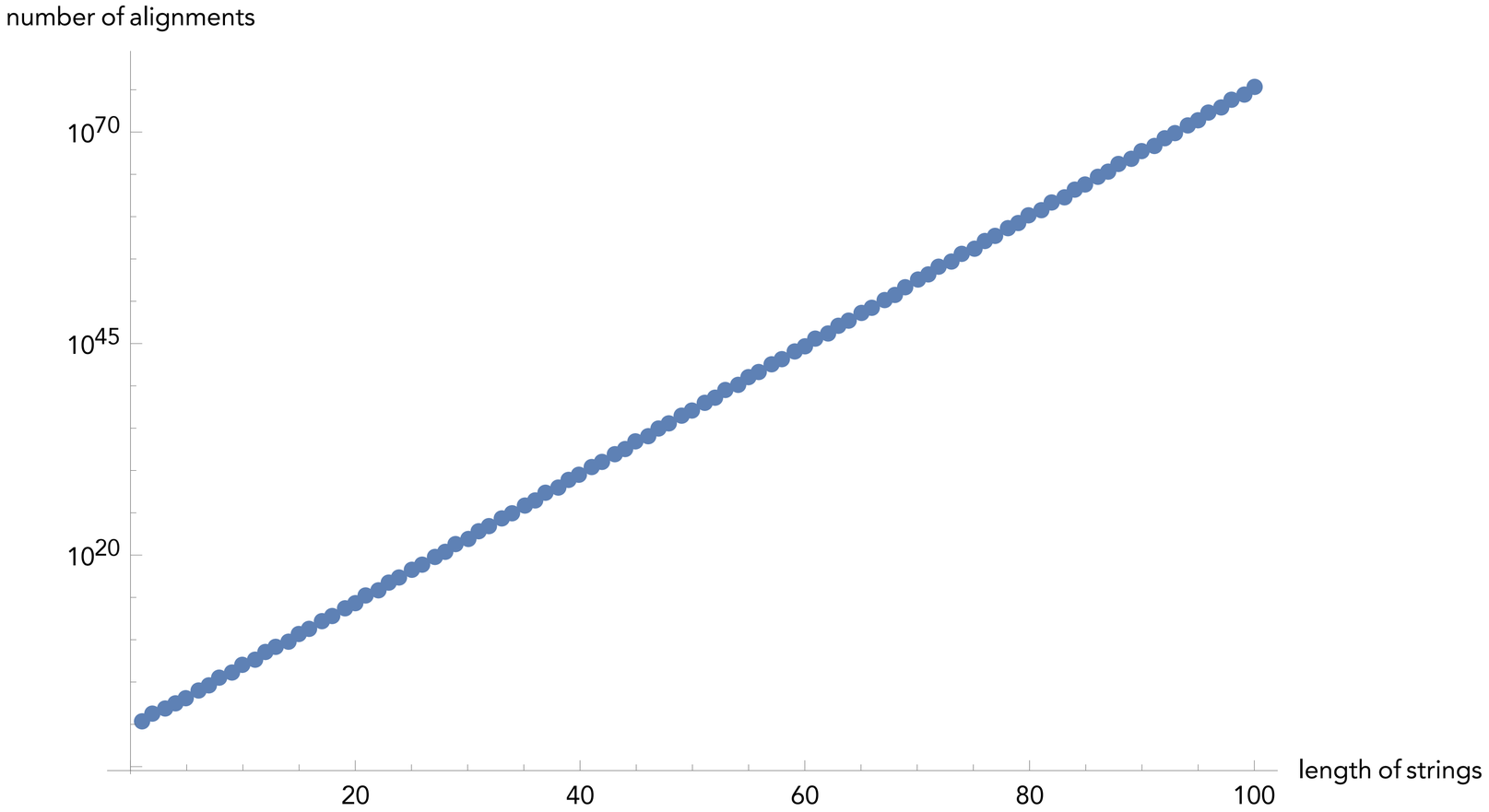
*

NCBI BLAST DNA Alignment

```
>gb|AC115706.7| Mus musculus chromosome 8, clone RP23-382B3, complete sequence

Query  1650   gtgtgtgtgggtgcacatttgtgtgtgtgtgcgcctgtgtgtgtgggtgcctgtgtgtgt  1709
              ||||||||||  |    ||  | |||||||| | ||||||||    ||| ||  |||||
Sbjct  56838  GTGTGTGTGGAAGTGAGTTCATCTGTGTGTGCACATGTGTGTGCA--TGCATGCATGTGT  56895

Query  1710   gtg-gggcacatttgtgtgtgtgtgtgcctgtgtgtgggtgcacatttgtgtgtgtgc  1768
              ||  |||||      ||  ||| ||||||| |||||||| |||  |||  ||||| || |
Sbjct  56896  GTCCGGGCA------TGCATGTCTGTGTGCATGTGTGTGTGTGTGCAT--GTGTGAGTAC  56947

Query  1769   ctgtgtgtgtgtgcctgtgtgtgggggtgcacatttgtgtgtgtgtgtgcctgtgtgtgg  1828
              |||||||||| ||| ||| |||| | |||  ||| |||||  ||||||   |||||  |
Sbjct  56948  CTGTGTGTGTATGCTTGTATGTGTGTGTGTGCATGTGTGTAGGTGTGTATATGTGTAAGT  57007

Query  1829   gggtgcacatttgtgtgtgtgtgtgcctgtgtgtgtgggtgcacatttgtgtgtgtgtgt  1888
                     ||| ||||||| ||||||  |||| | |||  ||||    |||||||||| ||
Sbjct  57008  T------CATCTGTGTGTATGTGTG--TGTGAGAGTGCATGCA----TGTGTGTGTGAGT  57055

Query  1889   gcctgtgtgt--gtgggtgcacatttgtgtgtgtgtgcctgtg--tgtgt--gggtgcac  1942
               | | |||||  |||  |||  ||  || |   | |  |||||  |||||  | |||  |
Sbjct  57056  TCATCTGTGTCAGTGTATGCTTATGGGTATAACT-TAACTGTGCATGTGTAAGTGTGTTC  57114

Query  1943   atttgtgtgtgtgtgtgcctgtgtgtgtgggtgcacatttgtgtgtgtgcctgtgtgtgg  2002
              || ||||| ||||||||  ||||||  ||  ||  |  | ||||||||   |||||||
Sbjct  57115  ATCTGTGTATGTGTGTG--TGTGTGAGTTAGTTCA----TCTGTGTGTGAGAGTGTGTGA  57168

Query  2003   gtgcacatttgtgtgtgtgtgcctgtgtgtgtgtgcctgtgtgtgtgggtgcacatttgt  2062
              |  | ||| |||||||| ||  | | ||| ||  |||  |||||  |||| ||| |||  ||
Sbjct  57169  G--CTCATCTGTGTGTGAGTTCATCTGTATGAGTG--TGTGTATGTGTGTGTACAAATGA  57224

Query  2063   gtgtgtgtgtgcctgtgtgtgtgggtgcacatttgtgtgtgtgtgtgtgcctgtgtgtgt  2122
              ||   | ||||| ||||||||||        |||  ||||||   | ||  |||| ||||
Sbjct  57225  GTTCATCTGTGCATGTGTGTGTG--------TTTAAGTGTGTTCATCTG--TGTGCGTGT  57274
```

*

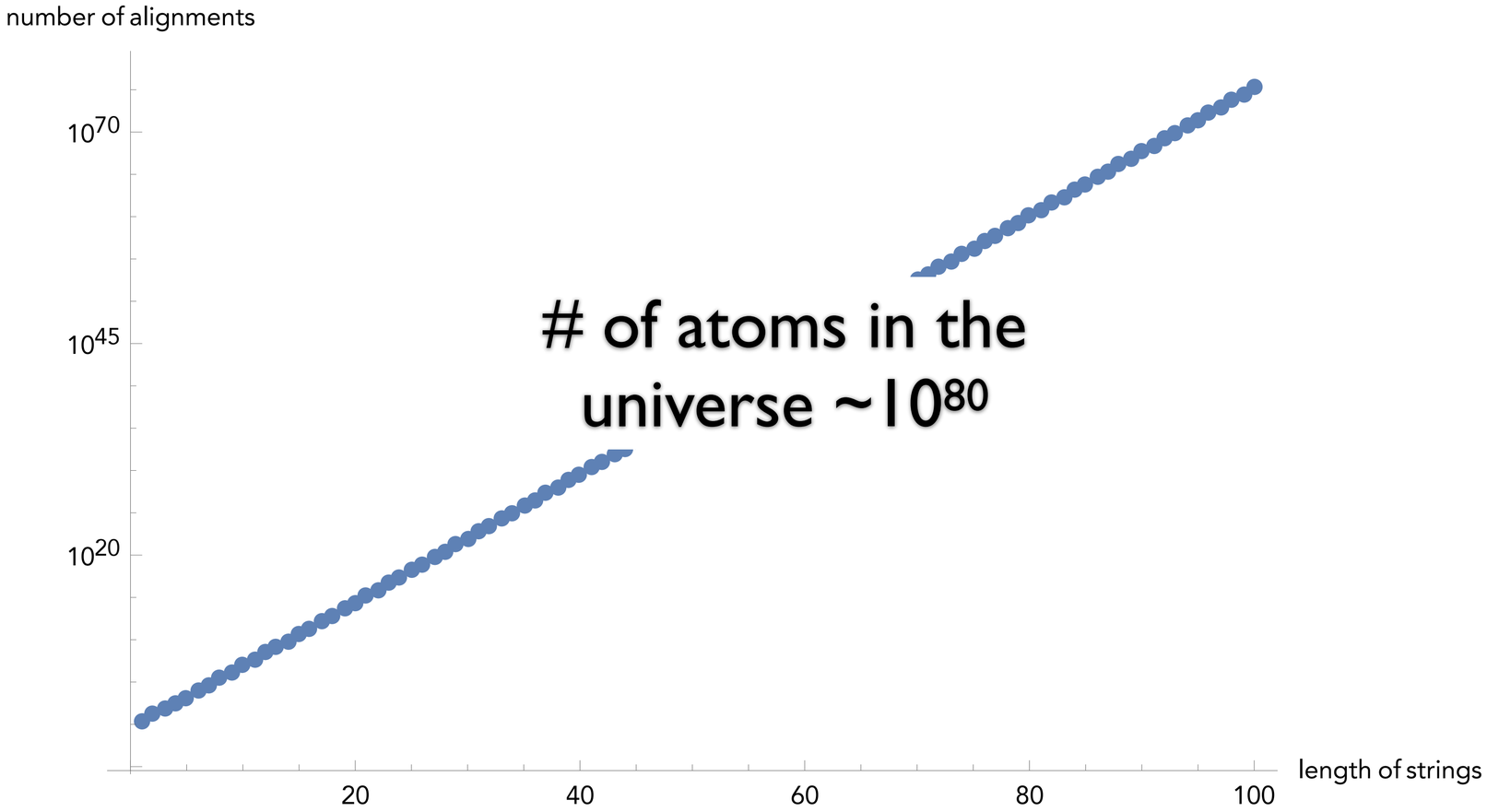
How many alignments are there?



$$f(n,m) = \sum_{k=0}^{\min(m,n)} 2^k \binom{m}{k} \binom{n}{k}$$

Andrade, Helena, et al. "The number of reduced alignments between two DNA sequences." BMC bioinformatics 15.1 (2014): 94.

How many alignments are there?



$$f(n,m) = \sum_{k=0}^{\min(m,n)} 2^k \binom{m}{k} \binom{n}{k}$$

Andrade, Helena, et al. "The number of reduced alignments between two DNA sequences." BMC bioinformatics 15.1 (2014): 94.

Interlude: Dynamic Programming

General and powerful *algorithm design* technique

“Programming” in the mathematical sense — nothing to do with e.g. code

To apply DP, we need optimal substructure and overlapping subproblems

optimal substructure — can combine solutions to “smaller” problems to generate solutions to “larger” problems.

overlapping subproblems — solutions to subproblems can be “re-used” in multiple contexts (to solve multiple) larger problems

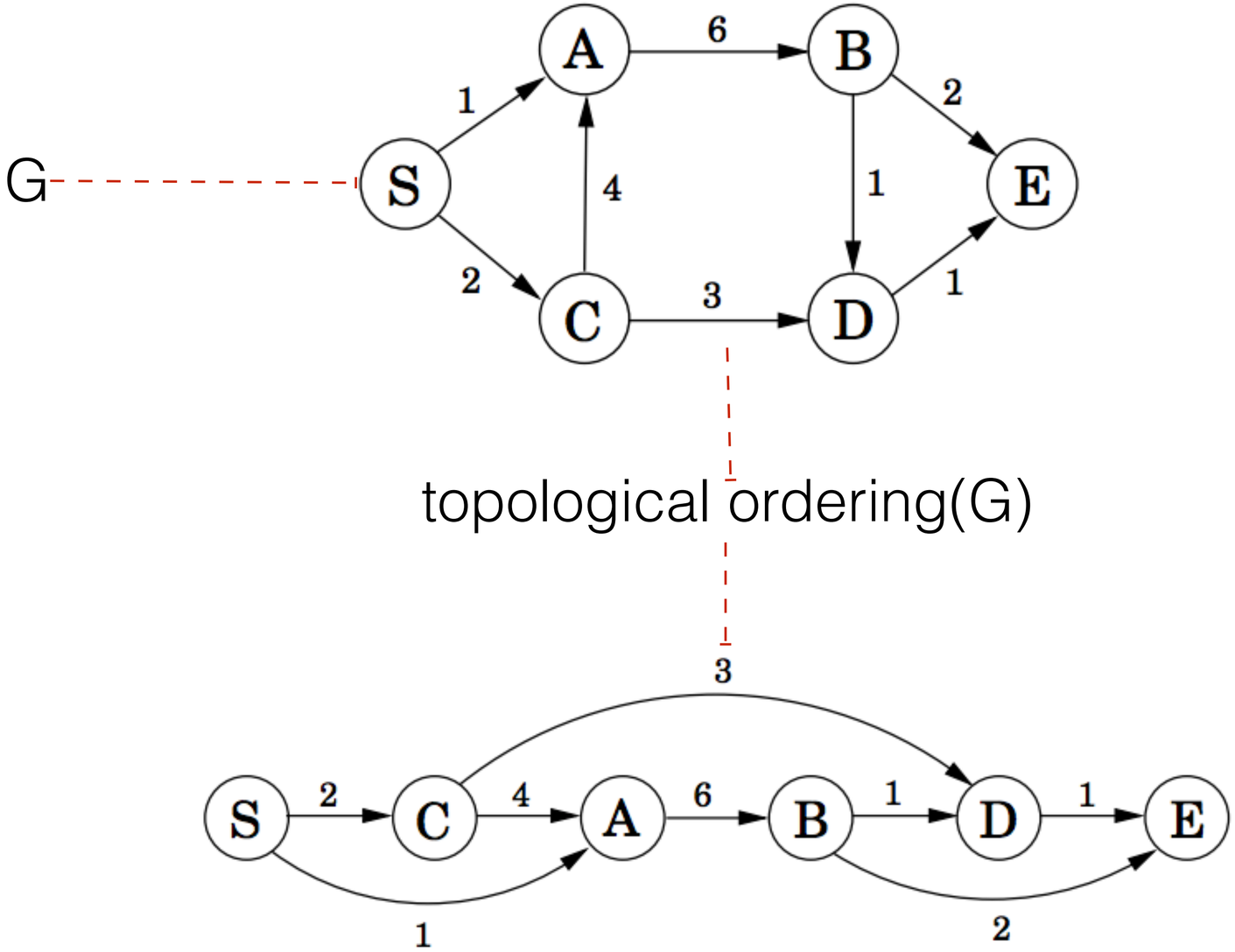
Example: Shortest Path in a DAG

Let G = (V,E) be a **d**irected **a**cyclic **g**raph (DAG) with vertex set V and edge set E.

Since G directed and free of cycles, there exists a (at least one) **topological order** of G — an ordering $p(v_1)$, $p(v_2)$, …, $p(v_n)$ such that for all $e = (v_i, v_j)$ in E, $p(v_i) < p(v_j)$

In other words, we can label the nodes of G such that all edges point from a vertex with a smaller label to a vertex with a larger label.

Example: Shortest Path in a DAG



Dasgupta, Sanjoy, Christos H. Papadimitriou, and Umesh Vazirani. Algorithms. McGraw-Hill, Inc., 2006.

Obtaining a topological ordering

Kahn's algorithm

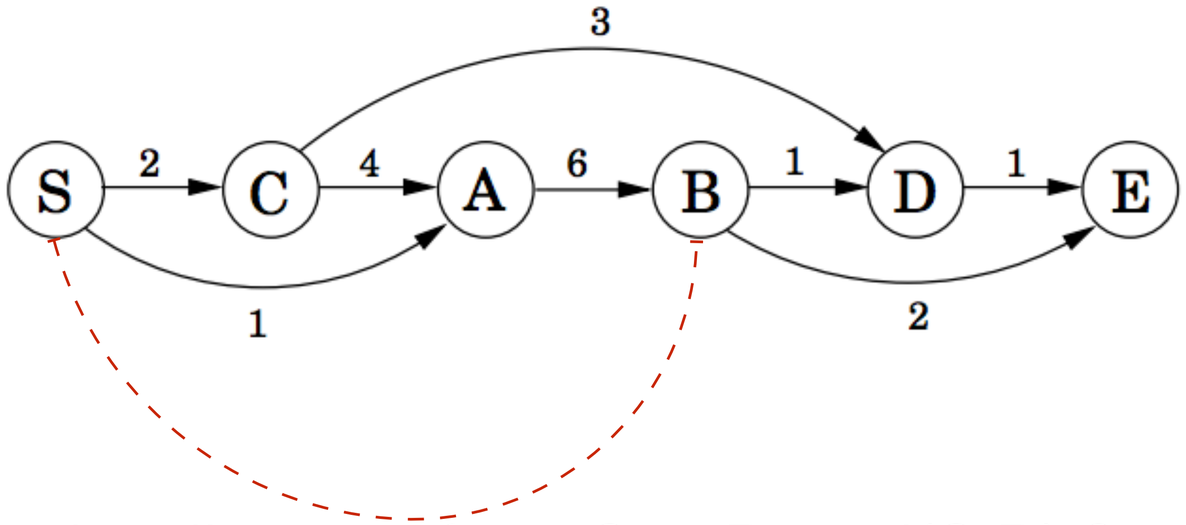
Builds up a valid topo order node-by-node

```
L ← Empty list that will contain the sorted elements
S ← Set of all nodes with no incoming edges
while S is non-empty do
    remove a node n from S
    add n to tail of L
    for each node m with an edge e from n to m do
        remove edge e from the graph
        if m has no other incoming edges then
            insert m into S
if graph has edges then
    return error (graph has at least one cycle)
else
    return L (a topologically sorted order)
```

O(|V| + |E|); why?

https://en.wikipedia.org/wiki/Topological_sorting

Example: Shortest Path in a DAG

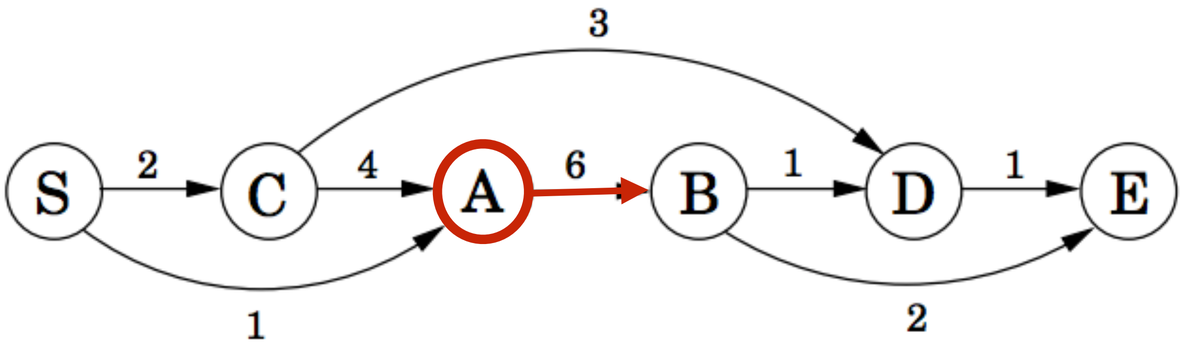


What's the distance from S to B — d(S,B) ?

Dasgupta, Sanjoy, Christos H. Papadimitriou, and Umesh Vazirani. Algorithms. McGraw-Hill, Inc., 2006.

Example: Shortest Path in a DAG

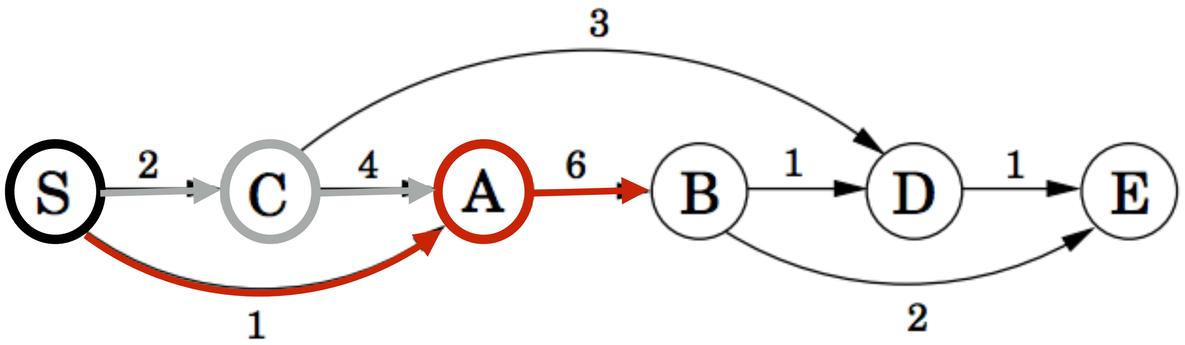
First, I **must** go through A, so it's at least d(S,A) + 6



Dasgupta, Sanjoy, Christos H. Papadimitriou, and Umesh Vazirani. Algorithms. McGraw-Hill, Inc., 2006.

Example: Shortest Path in a DAG

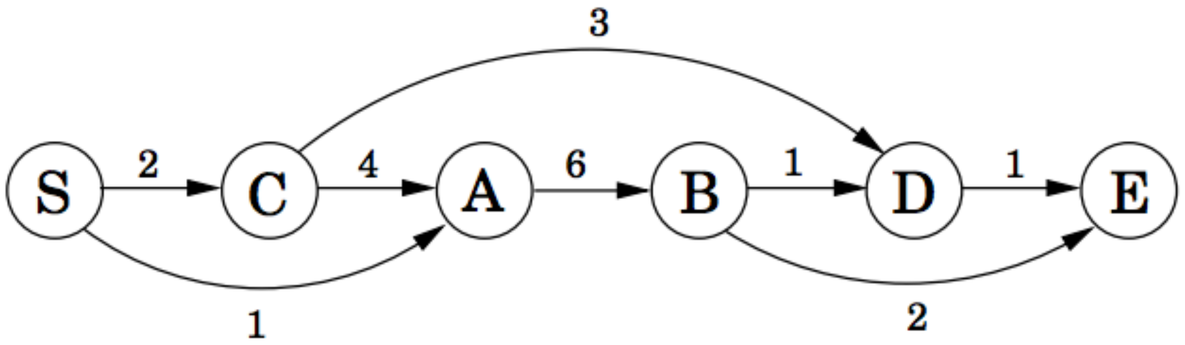
Then, there are 2 ways of getting to A — we choose the shortest.



Dasgupta, Sanjoy, Christos H. Papadimitriou, and Umesh Vazirani. Algorithms. McGraw-Hill, Inc., 2006.

Example: Shortest Path in a DAG

In general, d(S,X) is the
minimum value of d(S,Y) + d(Y,X) for all Y that precede X
and are connected by an edge



$$d(S,X) = \min_{Y \mid (Y,X) \in E} \{d(S,Y) + d(Y,X)\}$$

This becomes the DP recurrence for our problem

Dasgupta, Sanjoy, Christos H. Papadimitriou, and Umesh Vazirani. Algorithms. McGraw-Hill, Inc., 2006.

Example: Shortest Path in a DAG

The problem is solved efficiently by the following algorithm

initialize all $\text{dist}(\cdot)$ values to ∞
$\text{dist}(s) = 0$
for each $v \in V \backslash \{s\}$, in linearized order:
    $\text{dist}(v) = \min_{(u,v) \in E} \{\text{dist}(u) + l(u, v)\}$

Dasgupta, Sanjoy, Christos H. Papadimitriou, and Umesh Vazirani. Algorithms. McGraw-Hill, Inc., 2006.

Algorithm for Computing Edit Distance

Consider the last characters of each string:

$$a = a_1 a_2 a_3 a_4 \ldots a_m$$
$$b = b_1 b_2 b_3 b_4 \ldots b_n$$

One of these possibilities must hold:

1. (a_m,b_n) are matched to each other
2. a_m is not matched at all
3. b_n is not matched at all
4. a_m is matched to some b_j ($j \neq n$) and b_n is matched to some a_k ($k \neq m$).

*

Algorithm for Computing Edit Distance

Consider the last characters of each string:

$$a = a_1 a_2 a_3 a_4 \ldots a_m$$
$$b = b_1 b_2 b_3 b_4 \ldots b_n$$

One of these possibilities must hold:

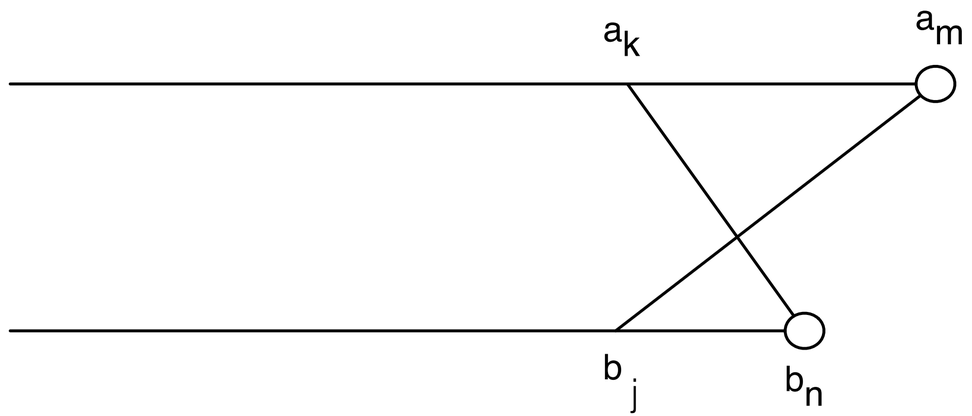
1. (a_m,b_n) are matched to each other
2. a_m is not matched at all
3. b_n is not matched at all
4. a_m is matched to some b_j ($j \neq n$) and b_n is matched to some a_k ($k \neq m$).

#4 can't happen! Why?

*

No Crossing Rule Forbids #4

4. a_m is matched to some b_j ($j \neq n$) and b_n is matched to some a_k ($k \neq$m).



So, the only possibilities for what happens to the last characters are:

1. (a_m, b_n) are matched to each other
2. a_m is not matched at all
3. b_n is not matched at all

*

Recursive Solution

Turn the 3 possibilities into 3 cases of a recurrence:

$$OPT(i,j) = \min \begin{cases} \mathbf{cost}(a_i, b_j) + OPT(i-1, j-1) & \text{match } a_i, b_j \\ \mathbf{gap} + OPT(i-1, j) & a_i \text{ is not matched} \\ \mathbf{gap} + OPT(i, j-1) & b_j \text{ is not matched} \end{cases}$$

Cost of the optimal alignment between $a_1 \ldots a_i$ and $b_1 \ldots b_j$

Written in terms of the costs of smaller problems

Key: we don't know which of the 3 possibilities is the right one, so we try them all.

Base case: $OPT(i,0) = i \times \mathbf{gap}$ and $OPT(0,j) = j \times \mathbf{gap}$

(Aligning i characters to 0 characters must use i gaps.)

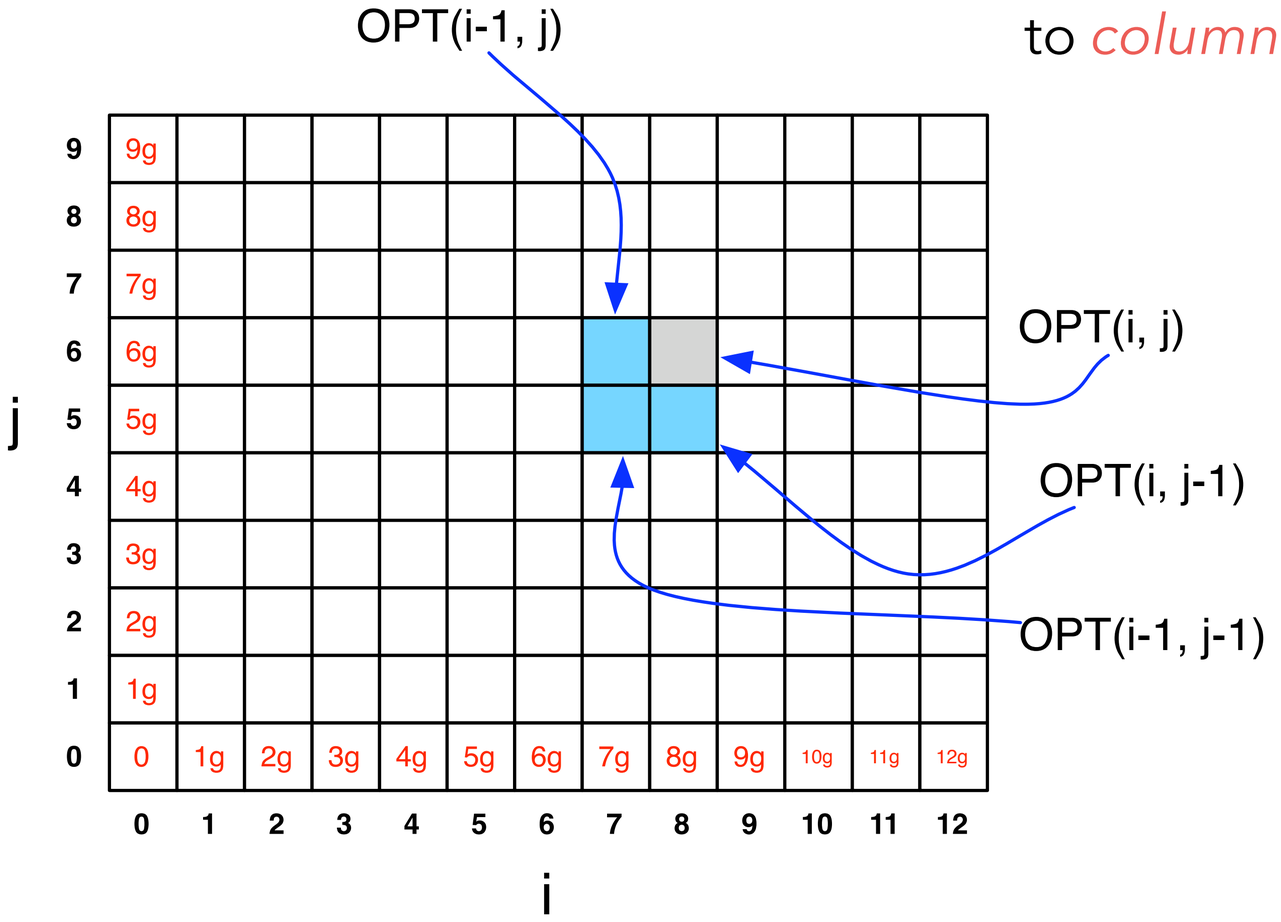
*

Computing OPT(i,j) Efficiently

We're ultimately interested in $OPT(n,m)$, but we will compute all other $OPT(i,j)$ $(i \le n, j \le m)$ on the way to computing $OPT(n,m)$.

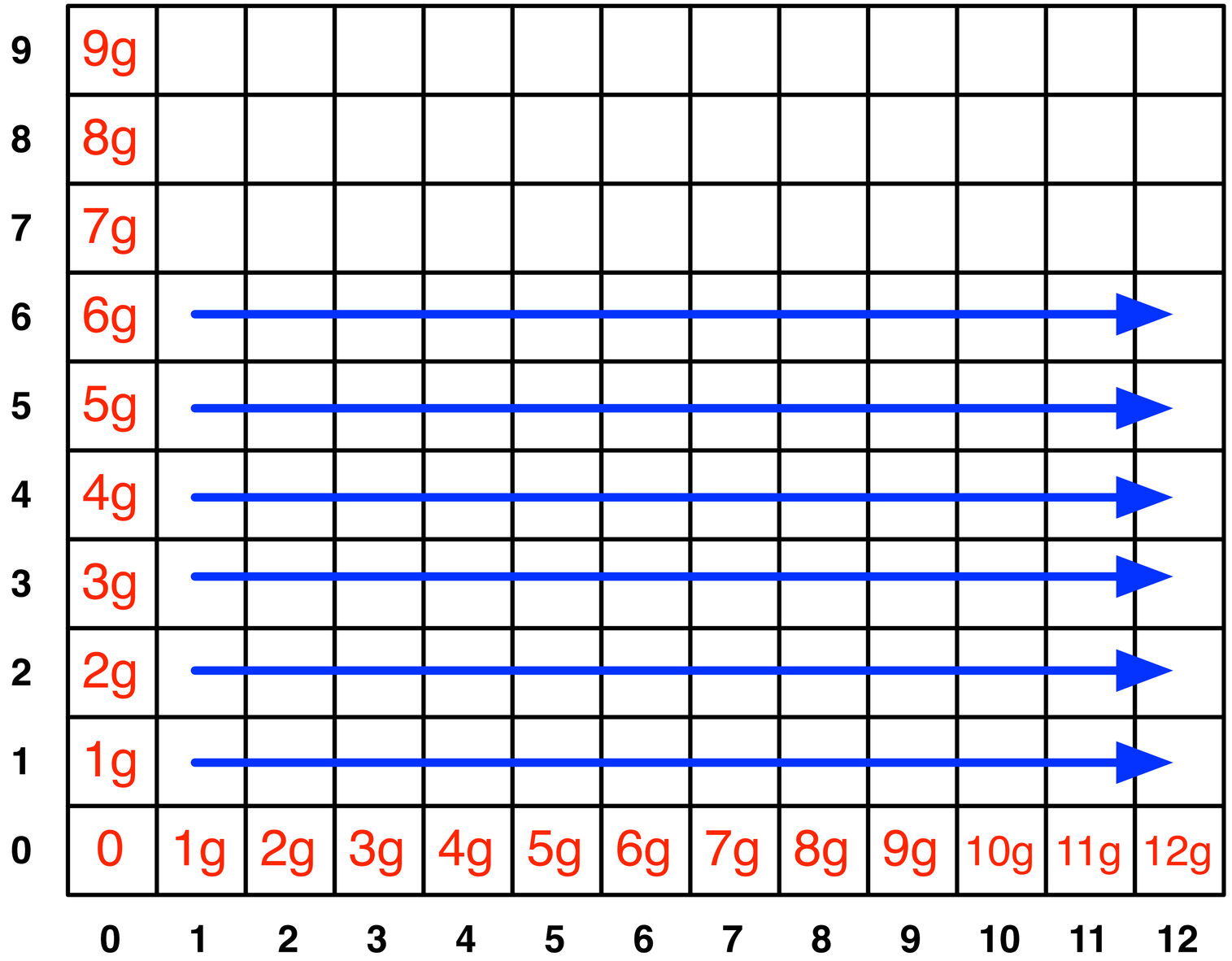
Store those values in a 2D array:

NOTE: observe the non-standard notation here; OPT(**i**,**j**) is referring to *column* i, *row* j of the matrix.



*

Filling in the 2D Array



*

Edit Distance Computation

```
EditDistance(X,Y):
    For i = 1,...,m: A[i,0] = i*gap
    For j = 1,...,n: A[0,j] = j*gap

    For i = 1,...,m:
        For j = 1,...,n:
            A[i,j] = min(
                cost(a[i],b[j]) + A[i-1,j-1],
                gap + A[i-1,j],
                gap + A[i,j-1]
            )
        EndFor
    EndFor
    Return A[m,n]
```

*

Where's the answer?

OPT(n,m) contains the edit distance between the two strings.

Why? By induction: EVERY cell contains the optimal edit distance between some prefix of string 1 with some prefix of string 2.

Running Time

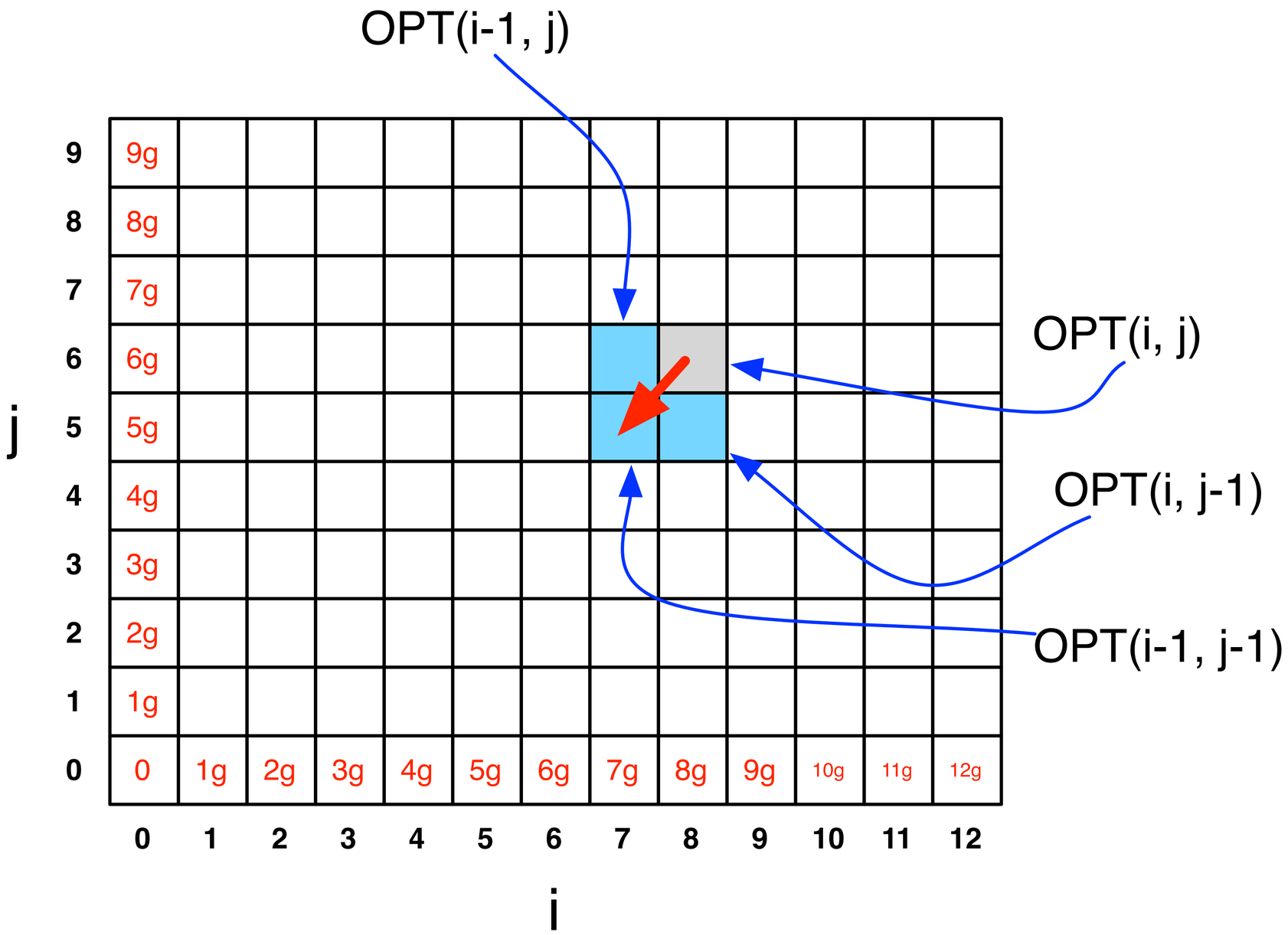
Number of entries in array = O($m \times n$), where m and n are the lengths of the 2 strings.

Filling in each entry takes constant O(1) time.

Total running time is O(mn).

*

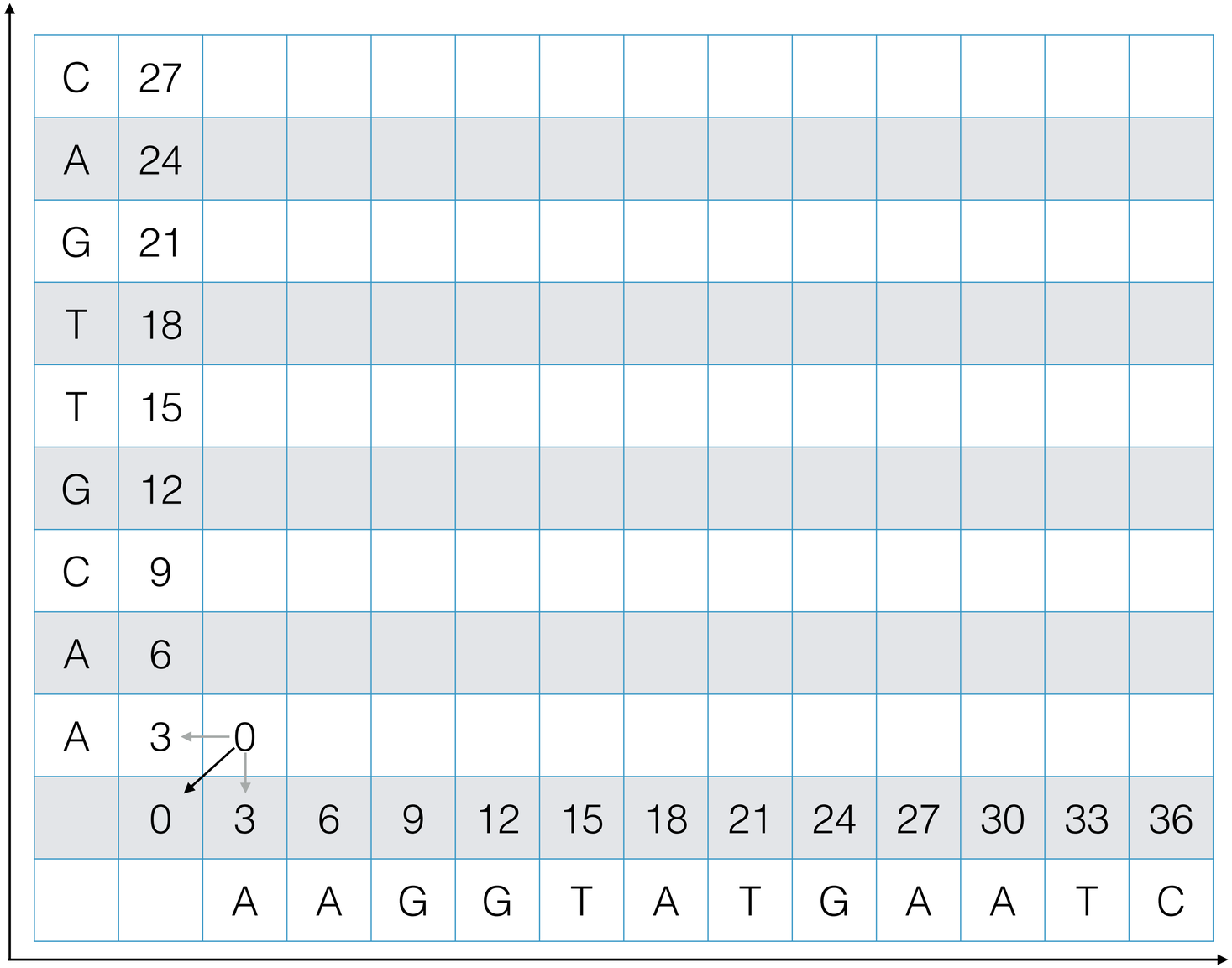
Finding the actual alignment



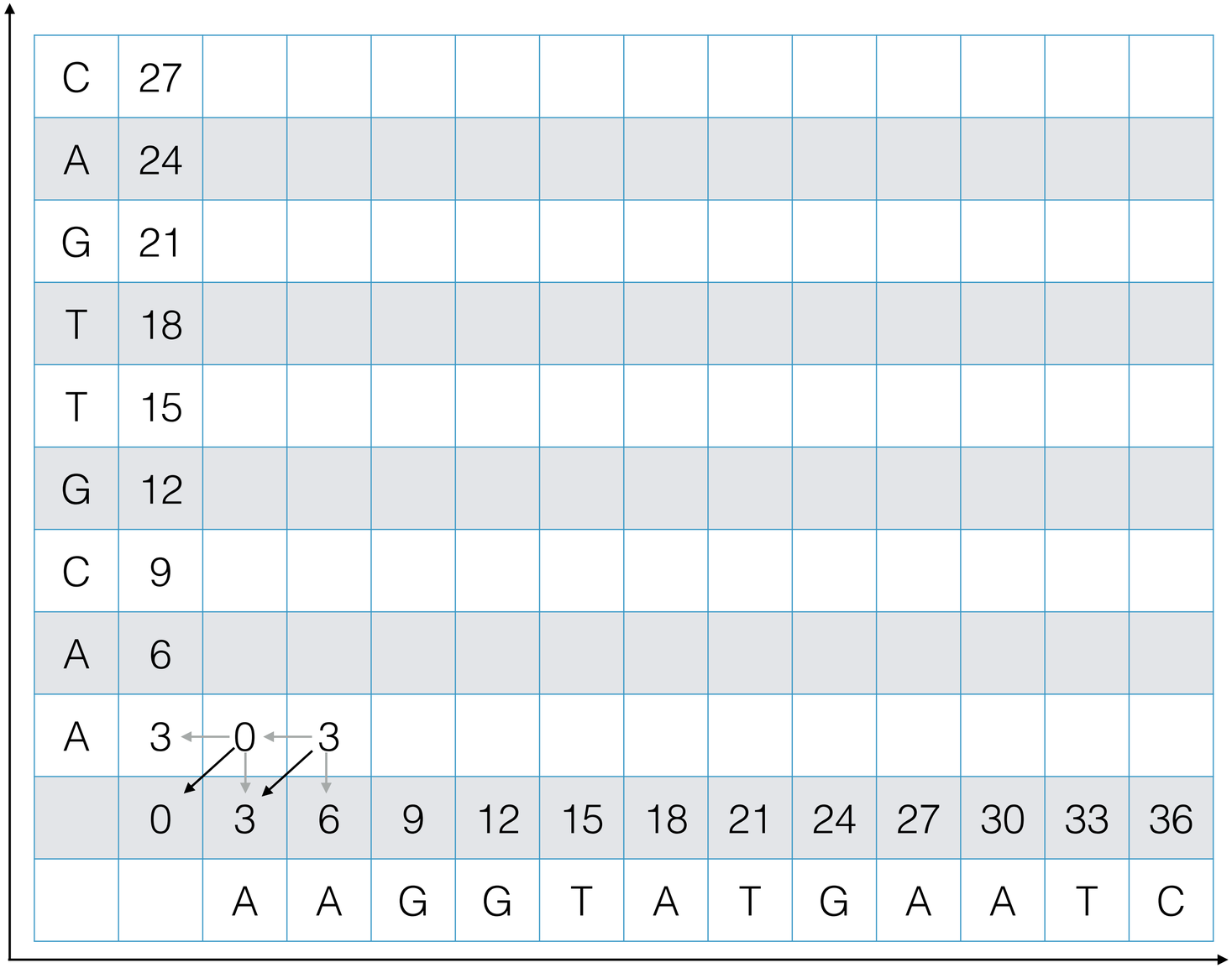
*

gap cost = 3
mismatch cost = 1

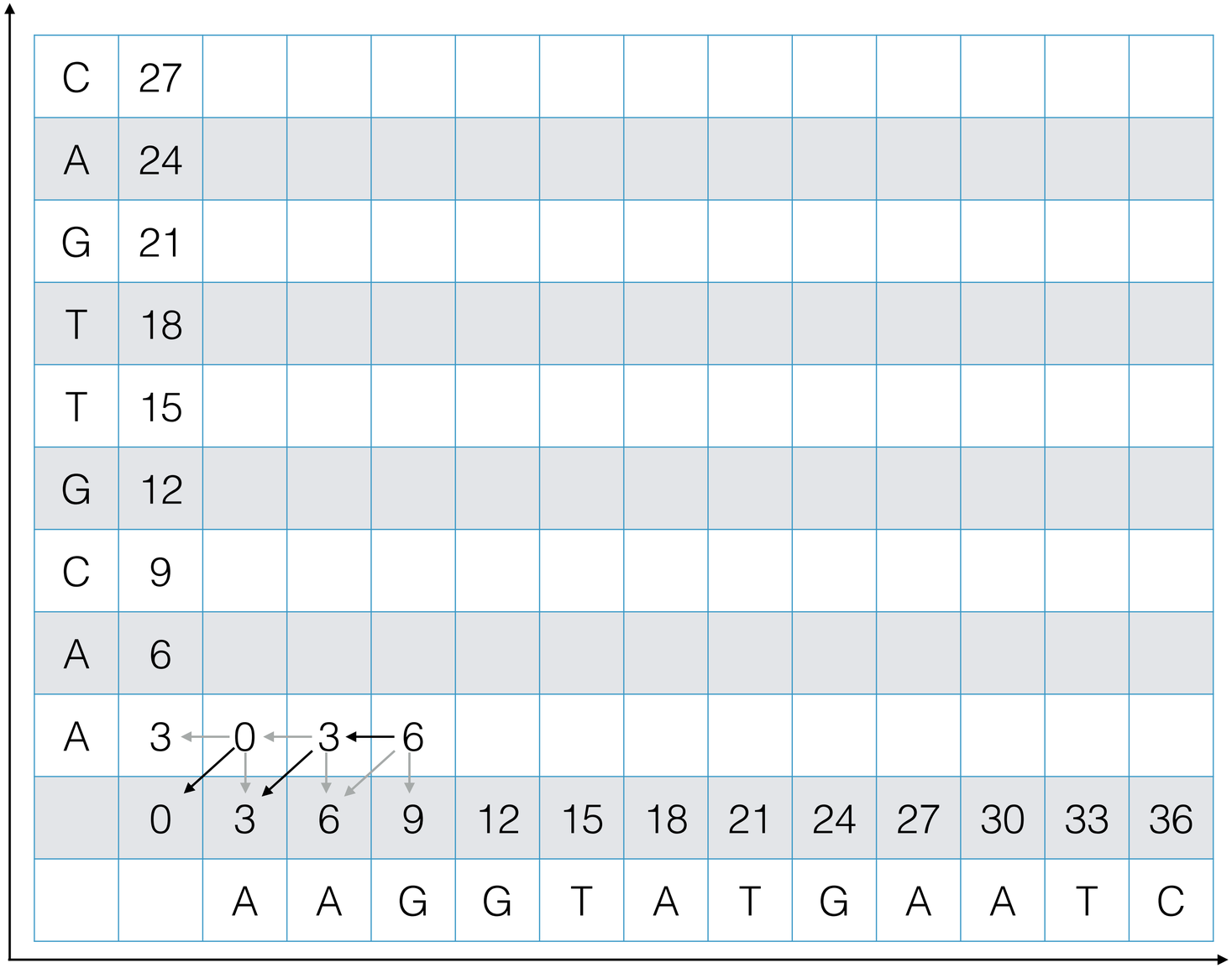
Example



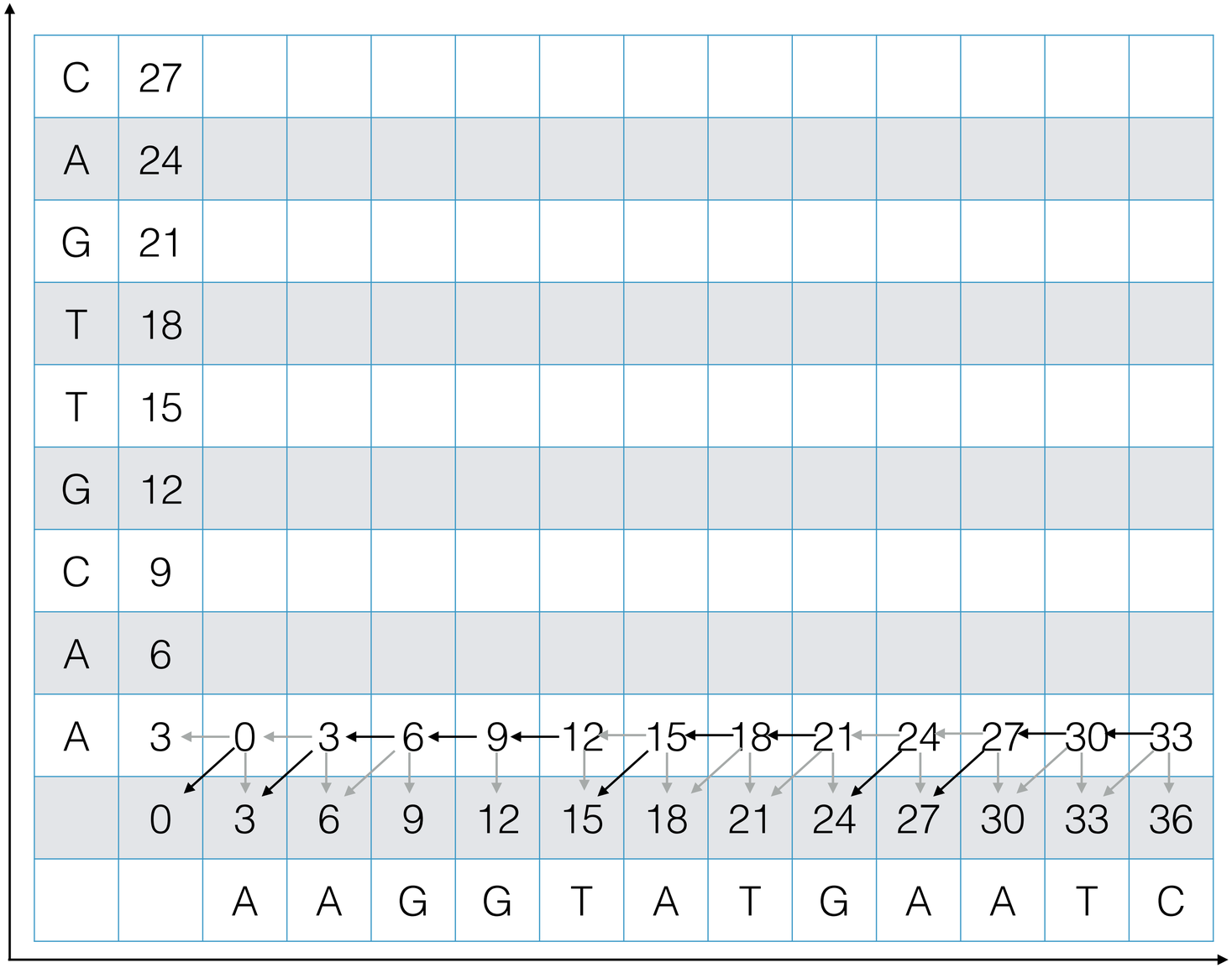
Example


C 27
A 24
G 21
T 18
T 15
G 12
C 9
A 6
A 3 0 3
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

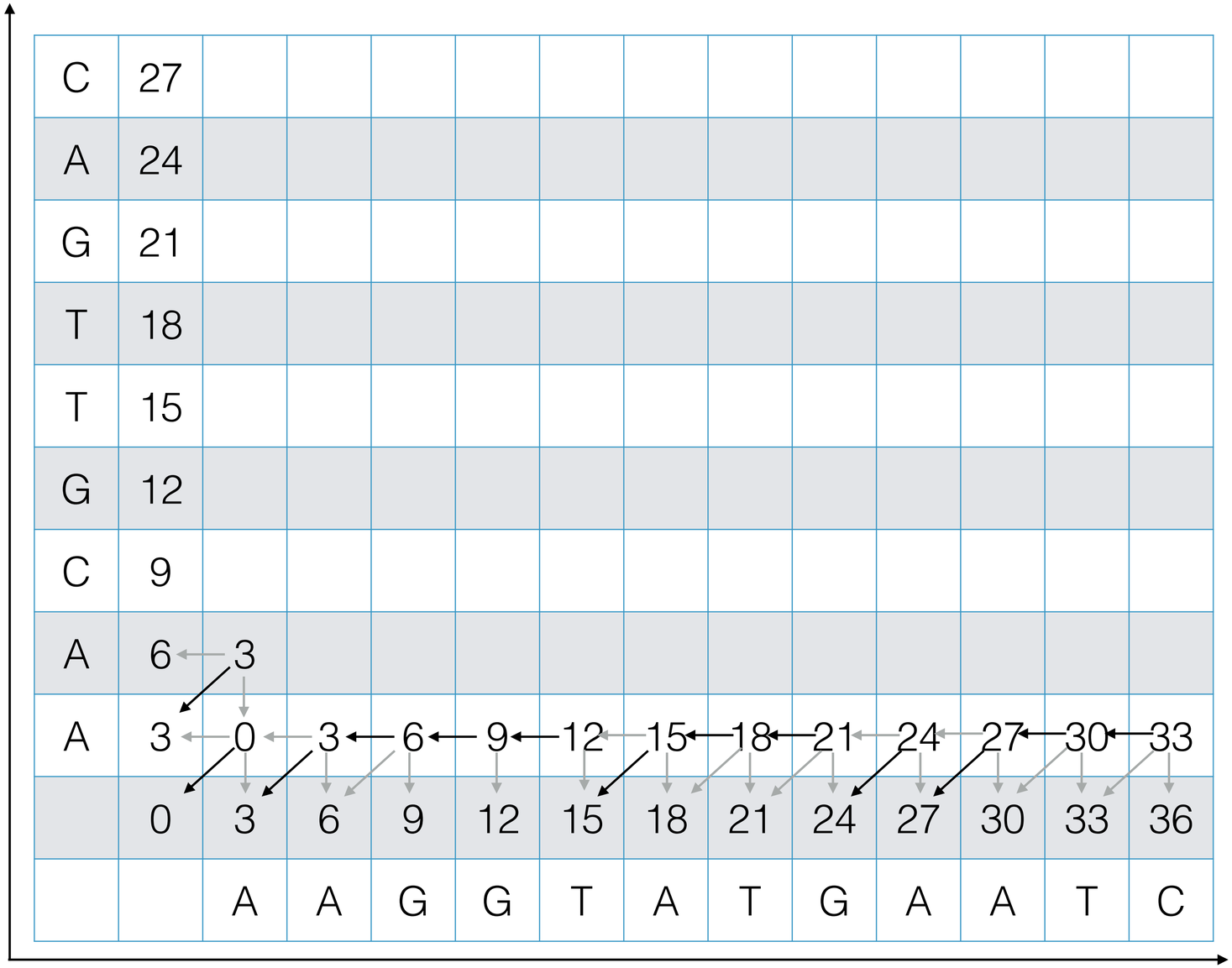
Example


C 27
A 24
G 21
T 18
T 15
G 12
C 9
A 6
A 3 0 3 6
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

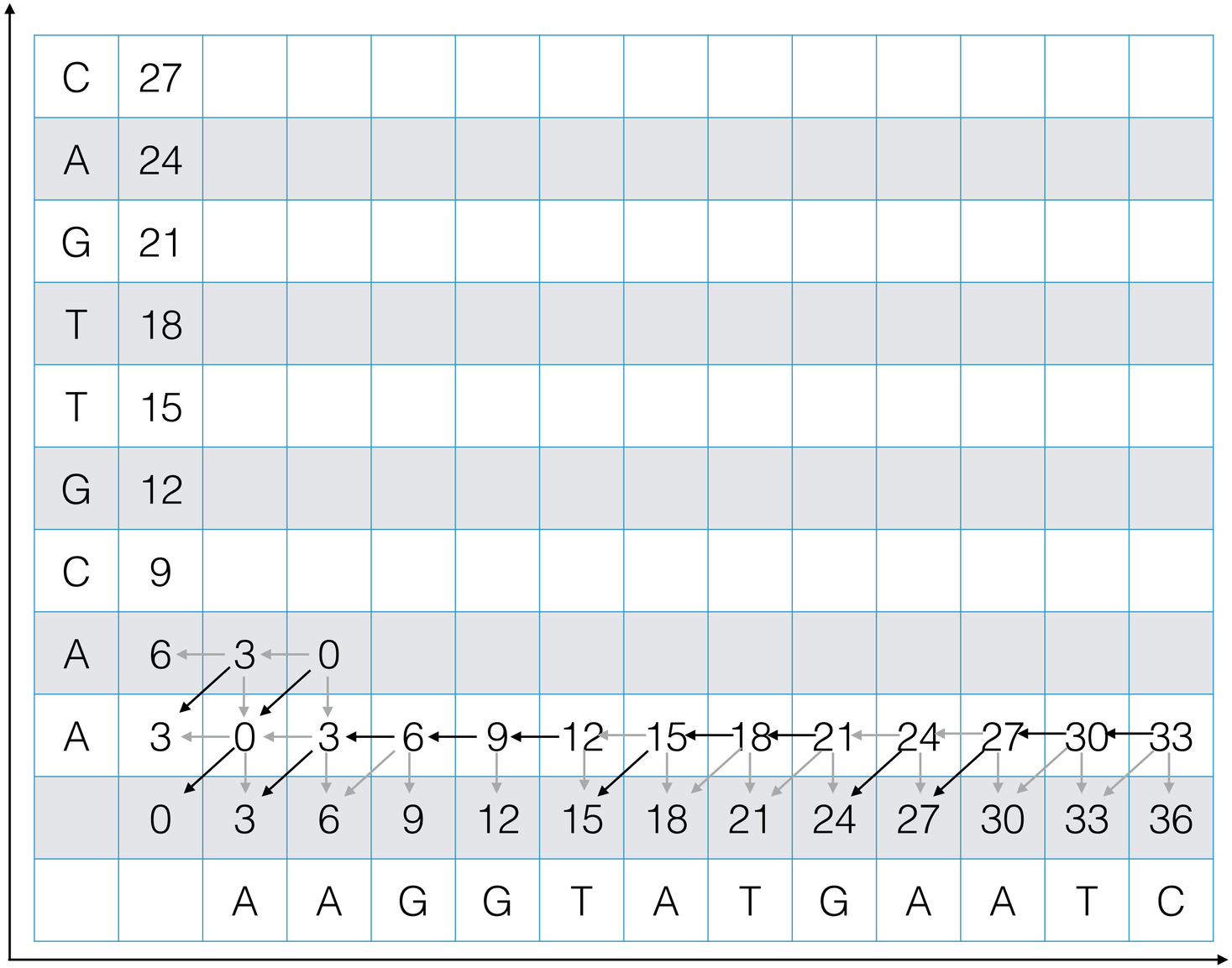
Example


C 27
A 24
G 21
T 18
T 15
G 12
C 9
A 6
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

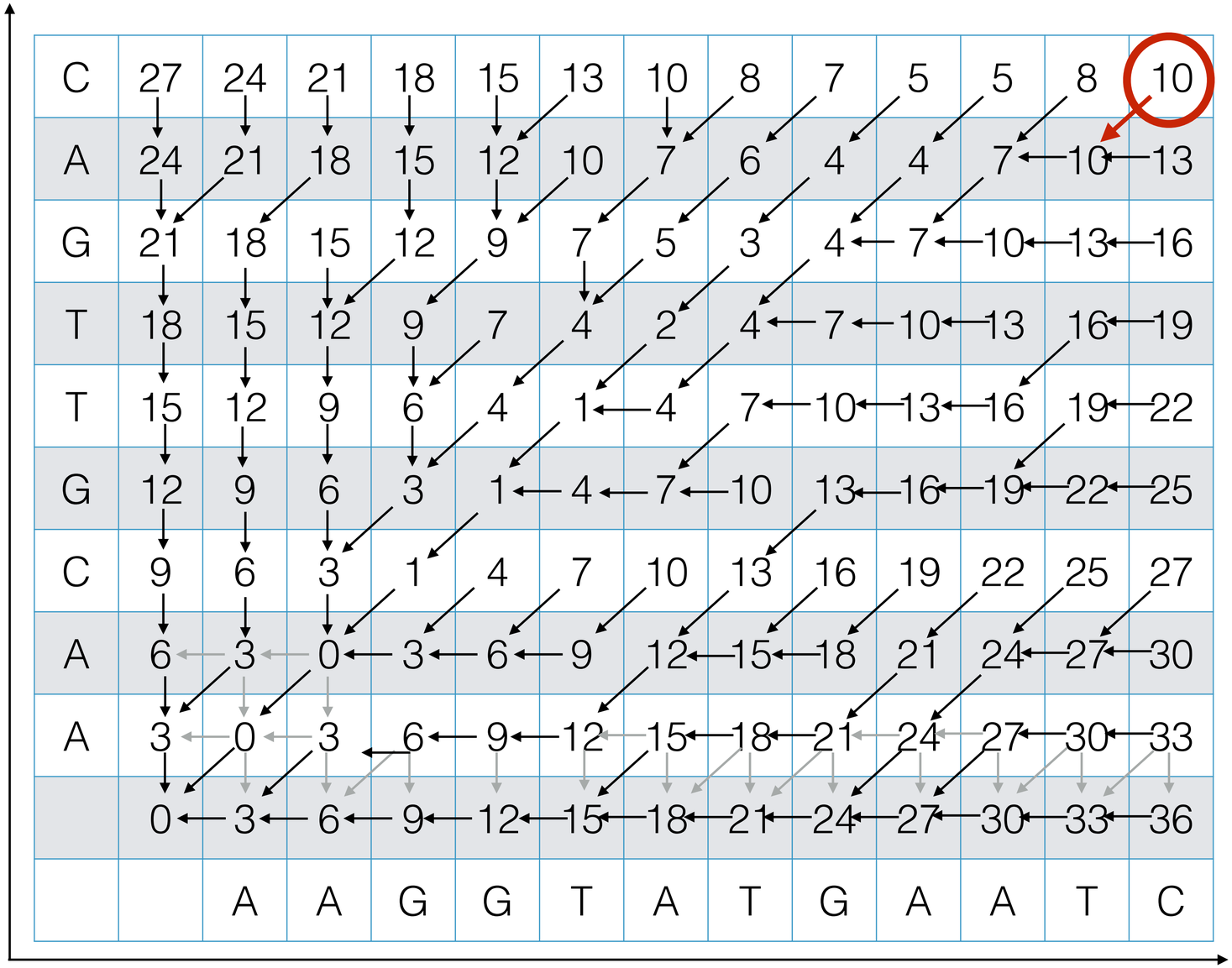
Example


C 27
A 24
G 21
T 18
T 15
G 12
C 9
A 6 3
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

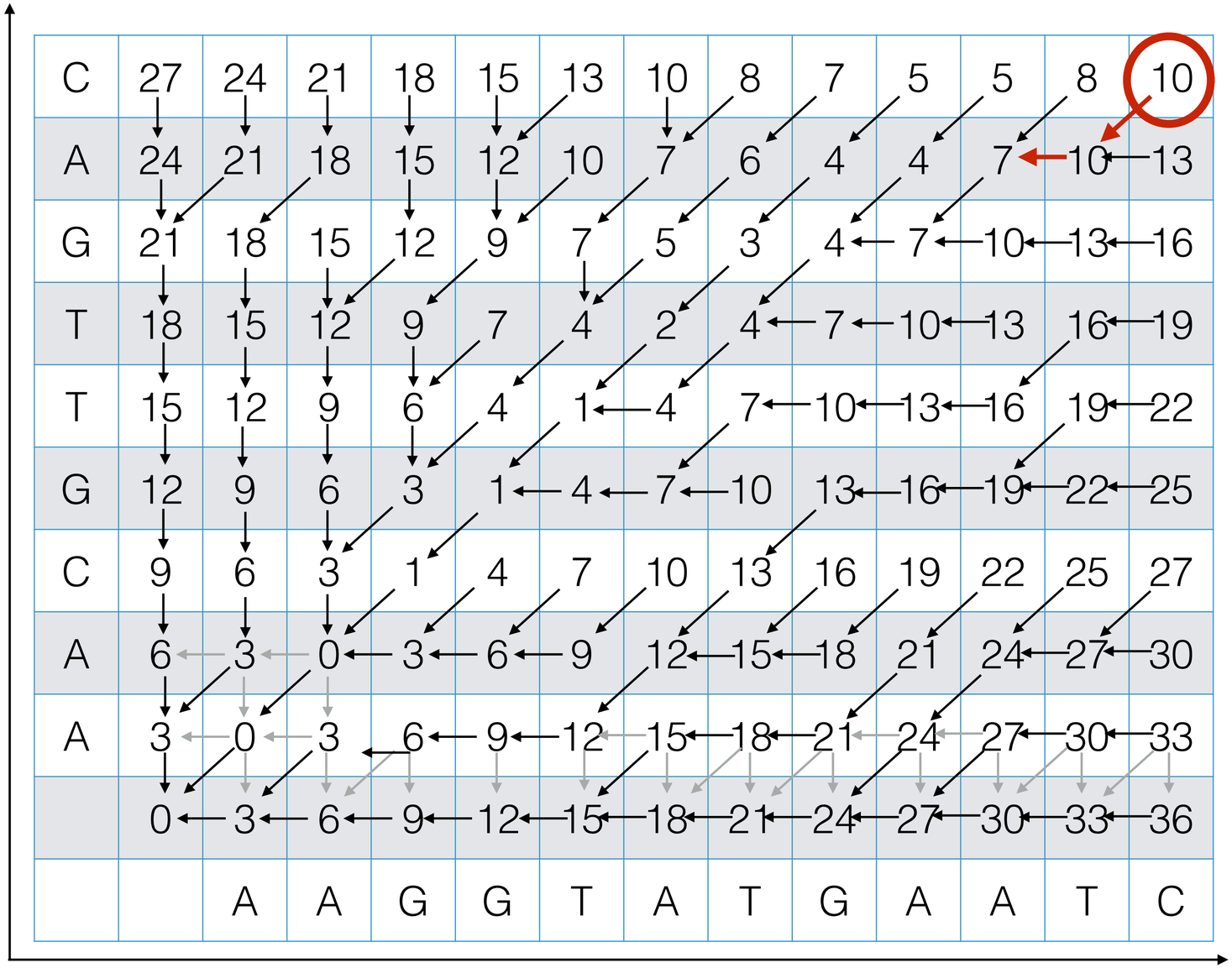
Example


C 27
A 24
G 21
T 18
T 15
G 12
C 9
A 6 3 0
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

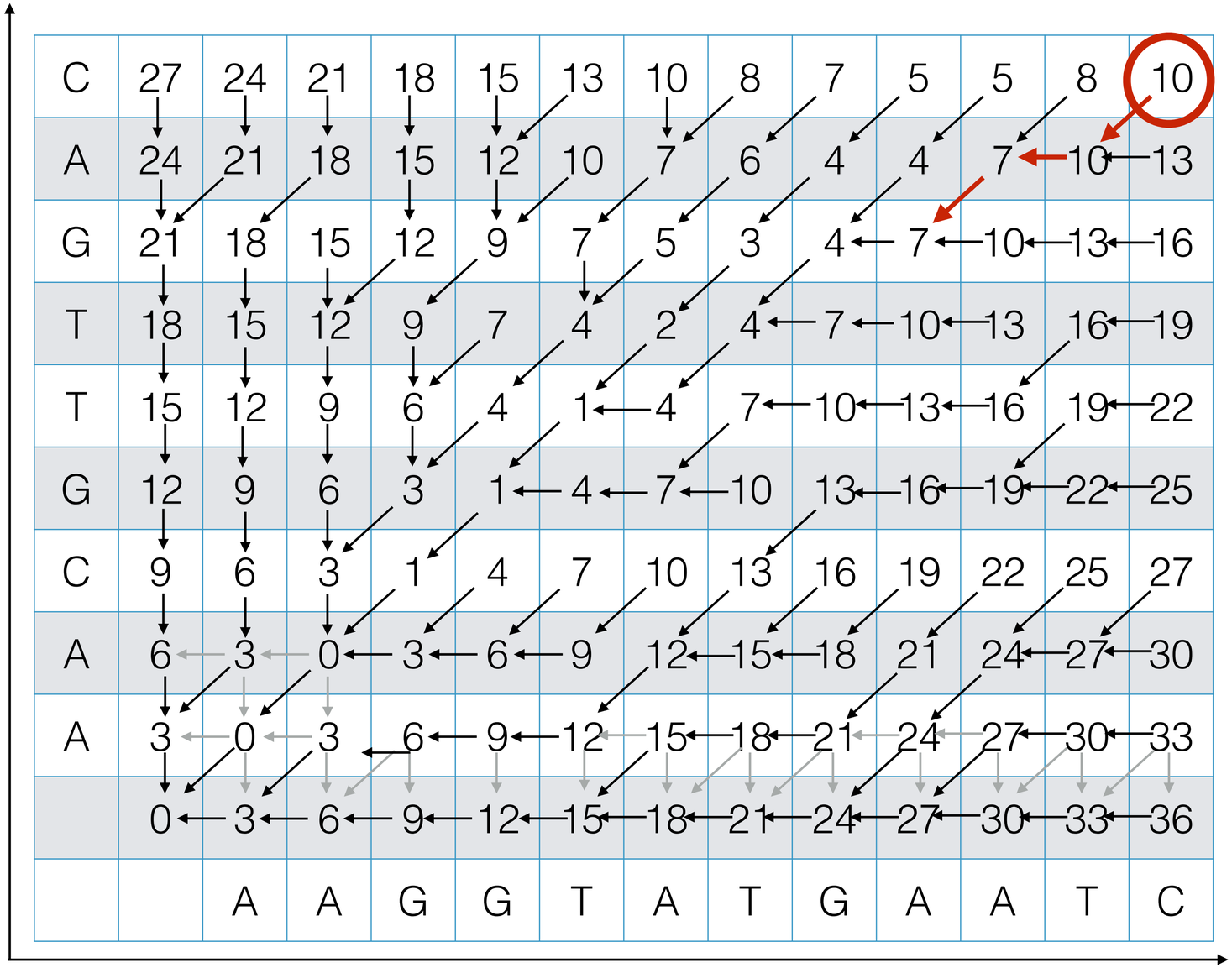
Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

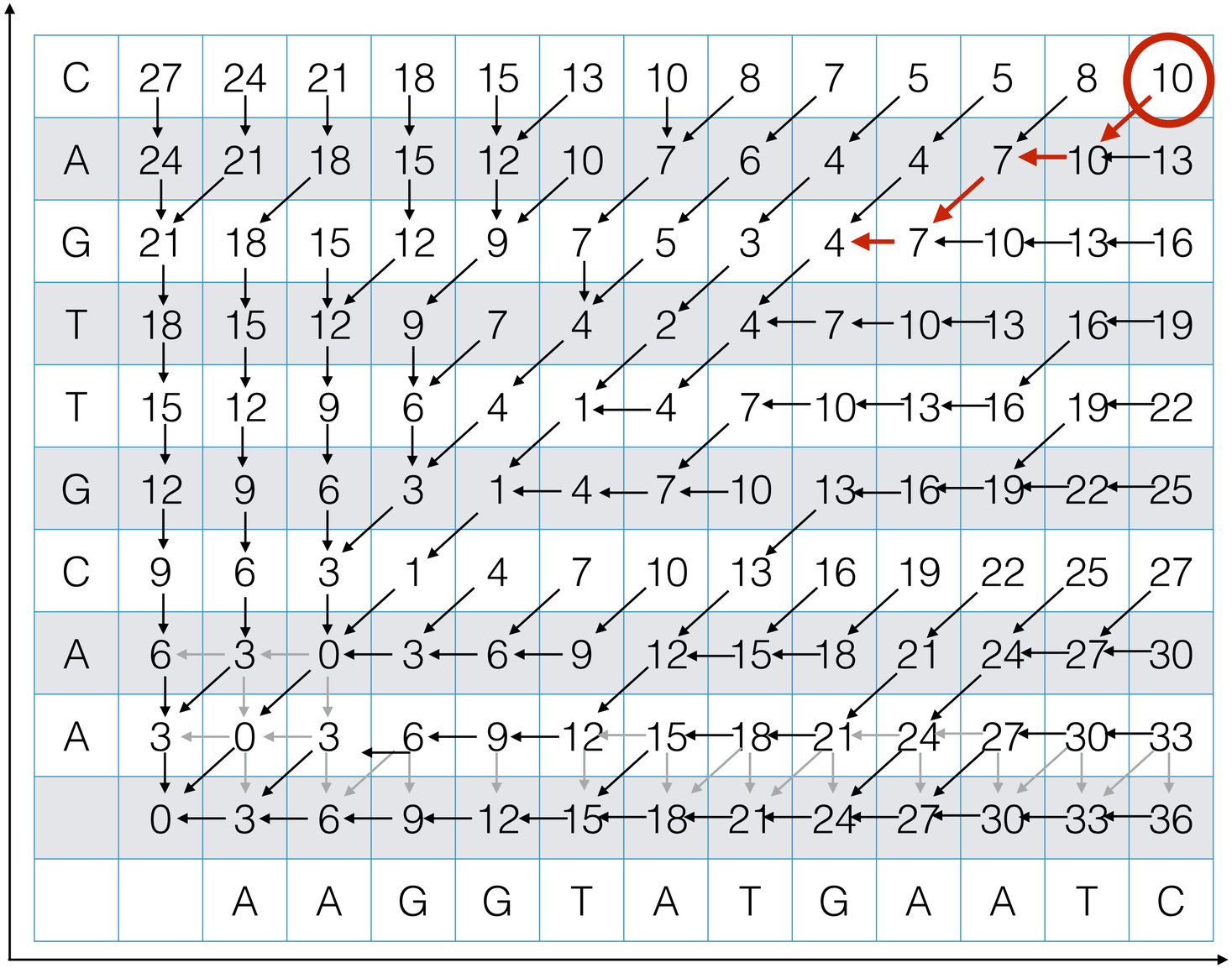
Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

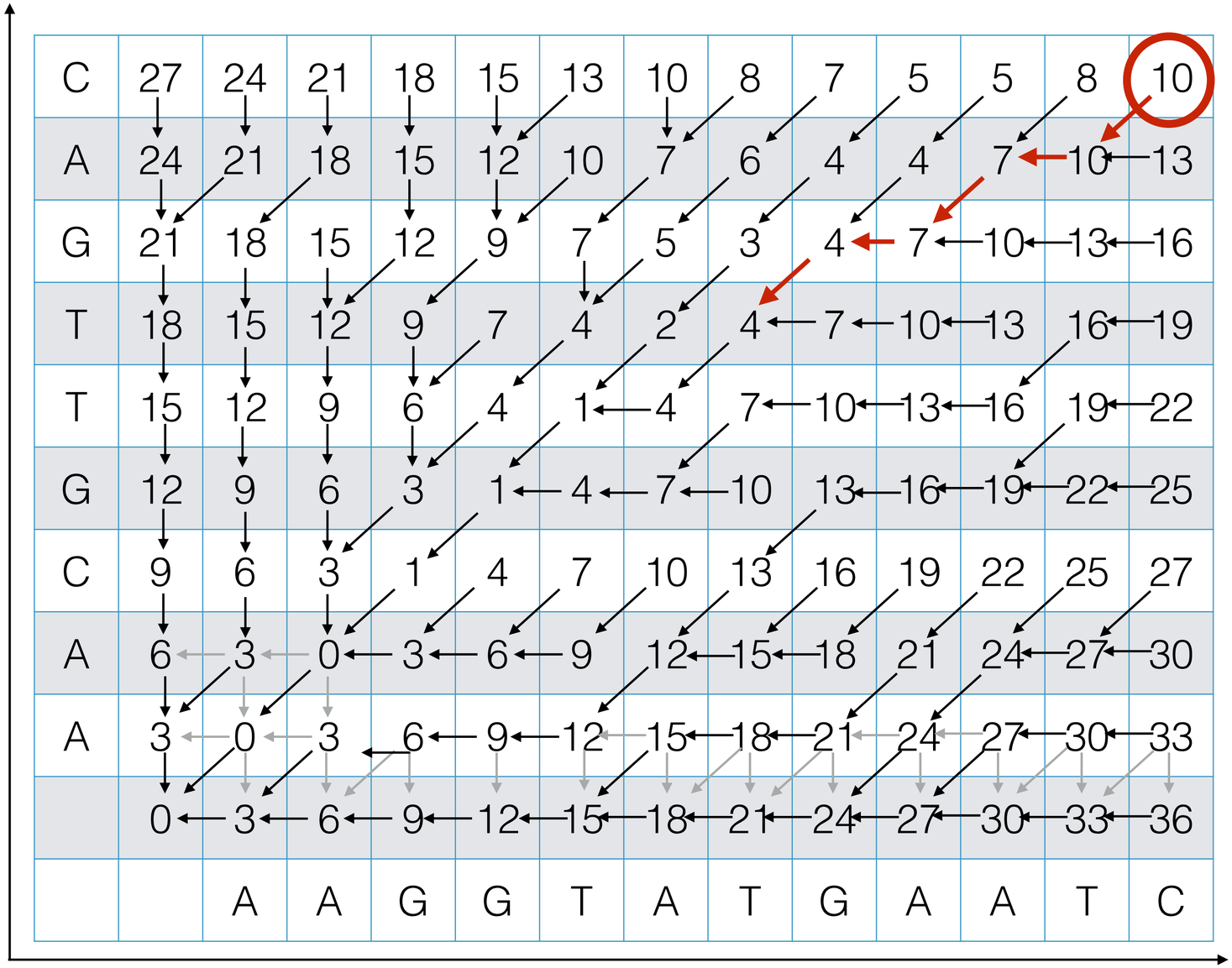
Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

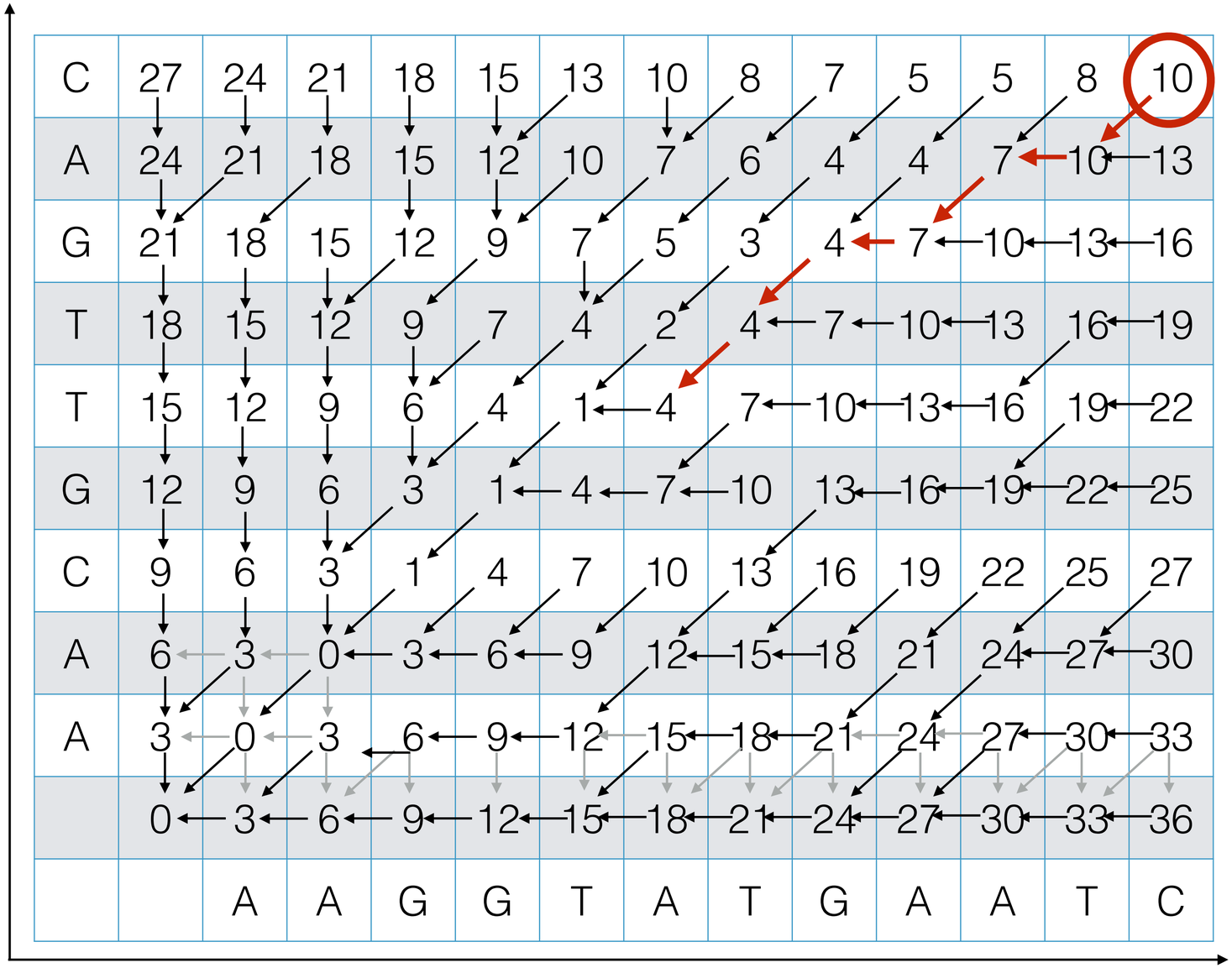
Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

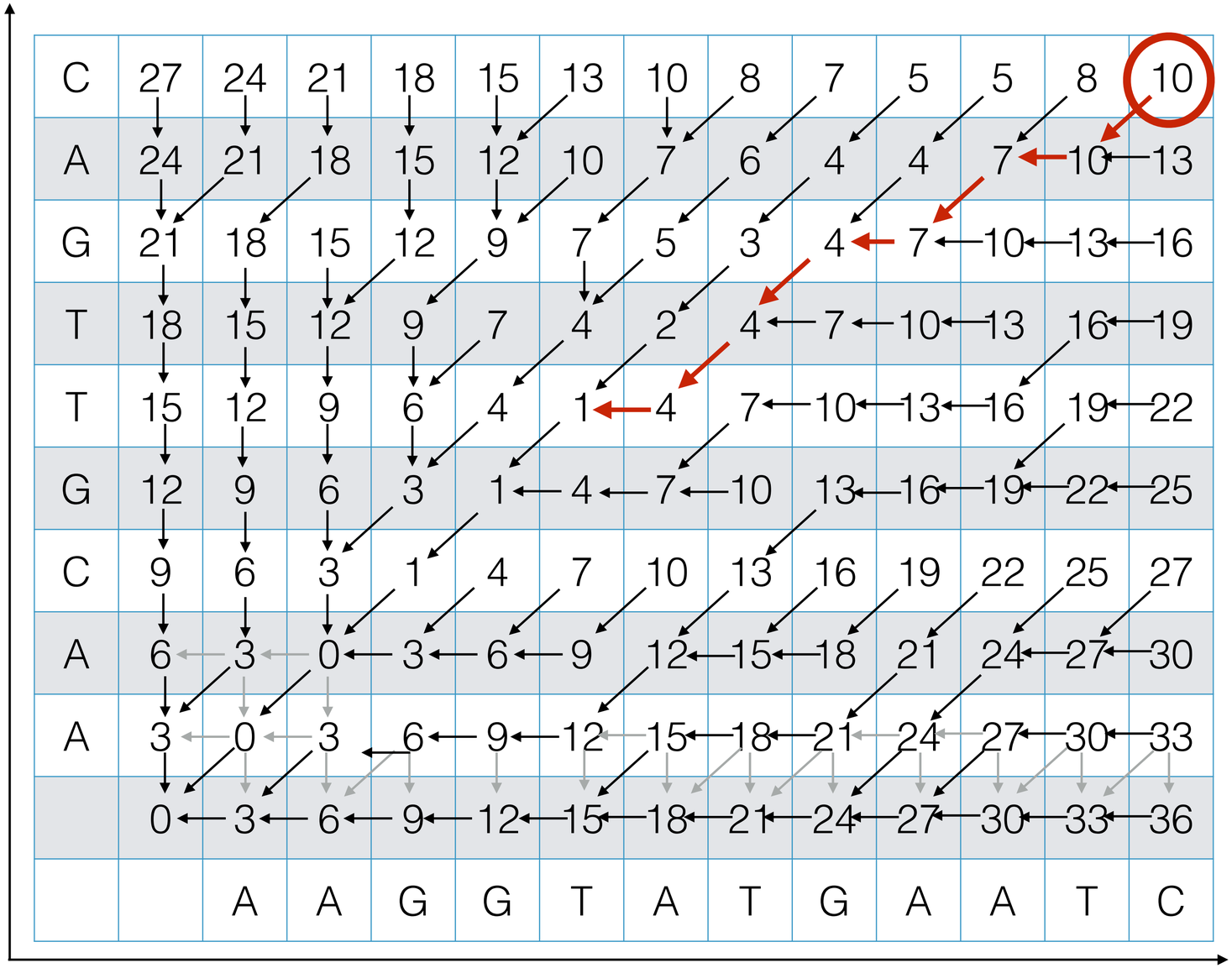
Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

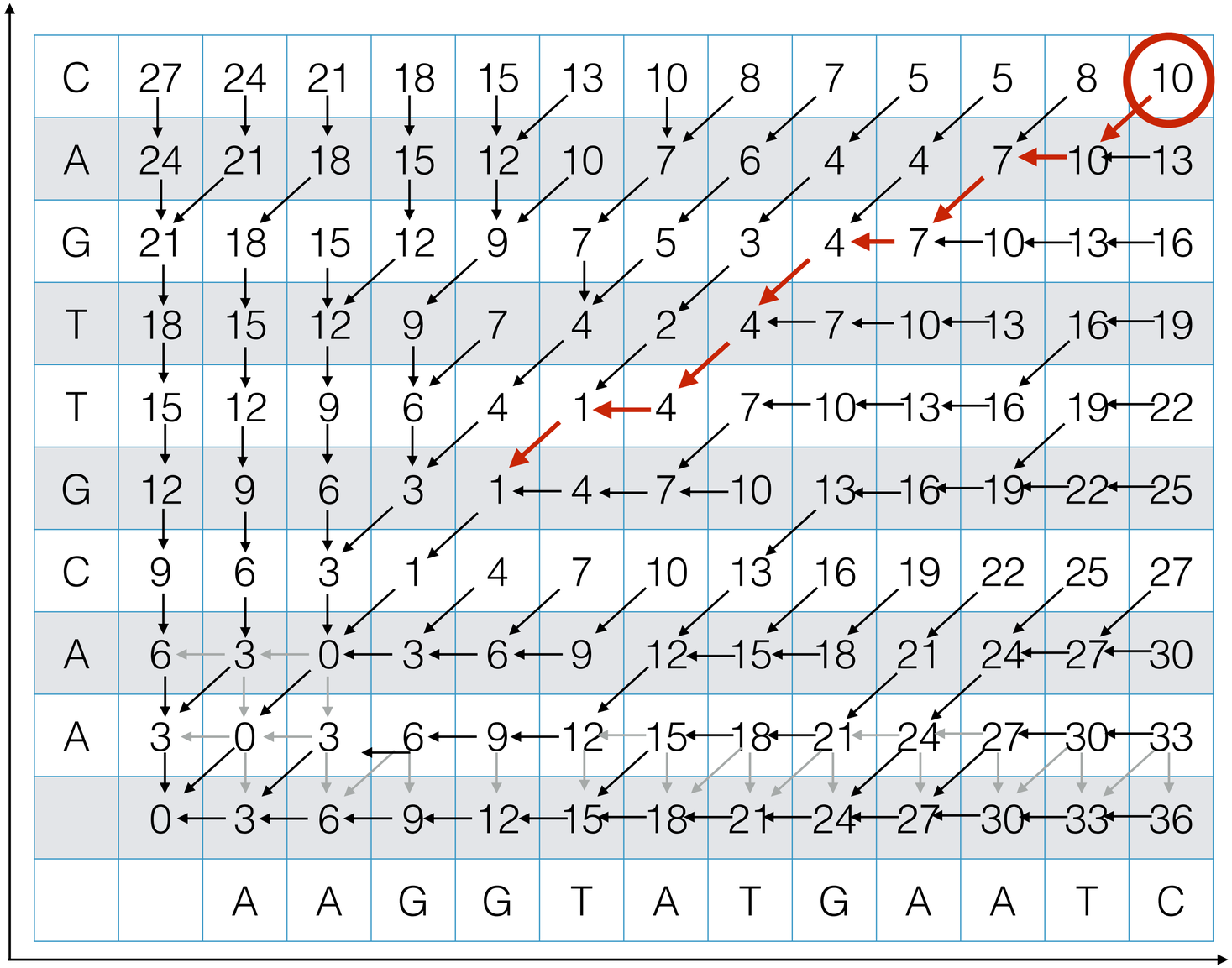
Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

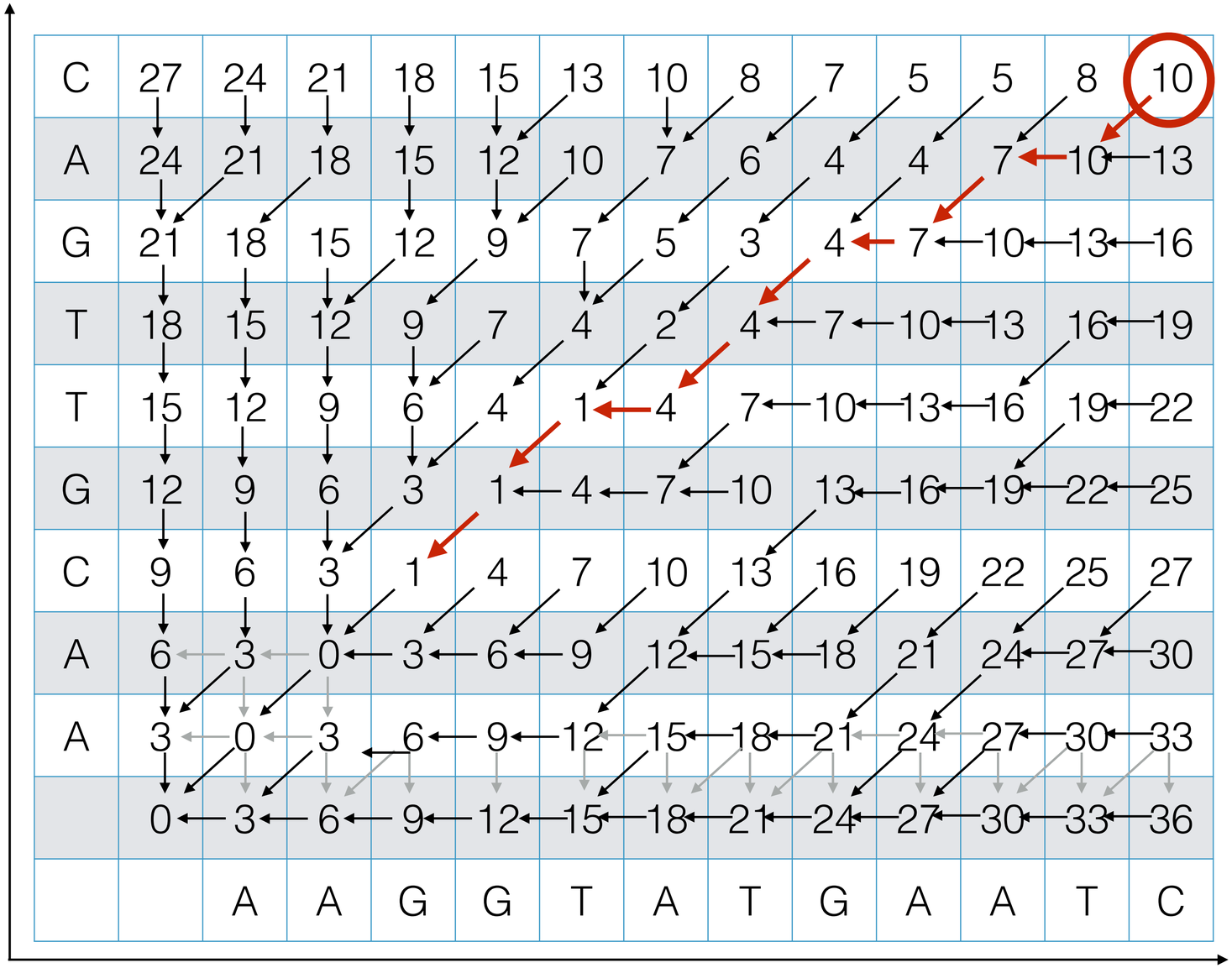
Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

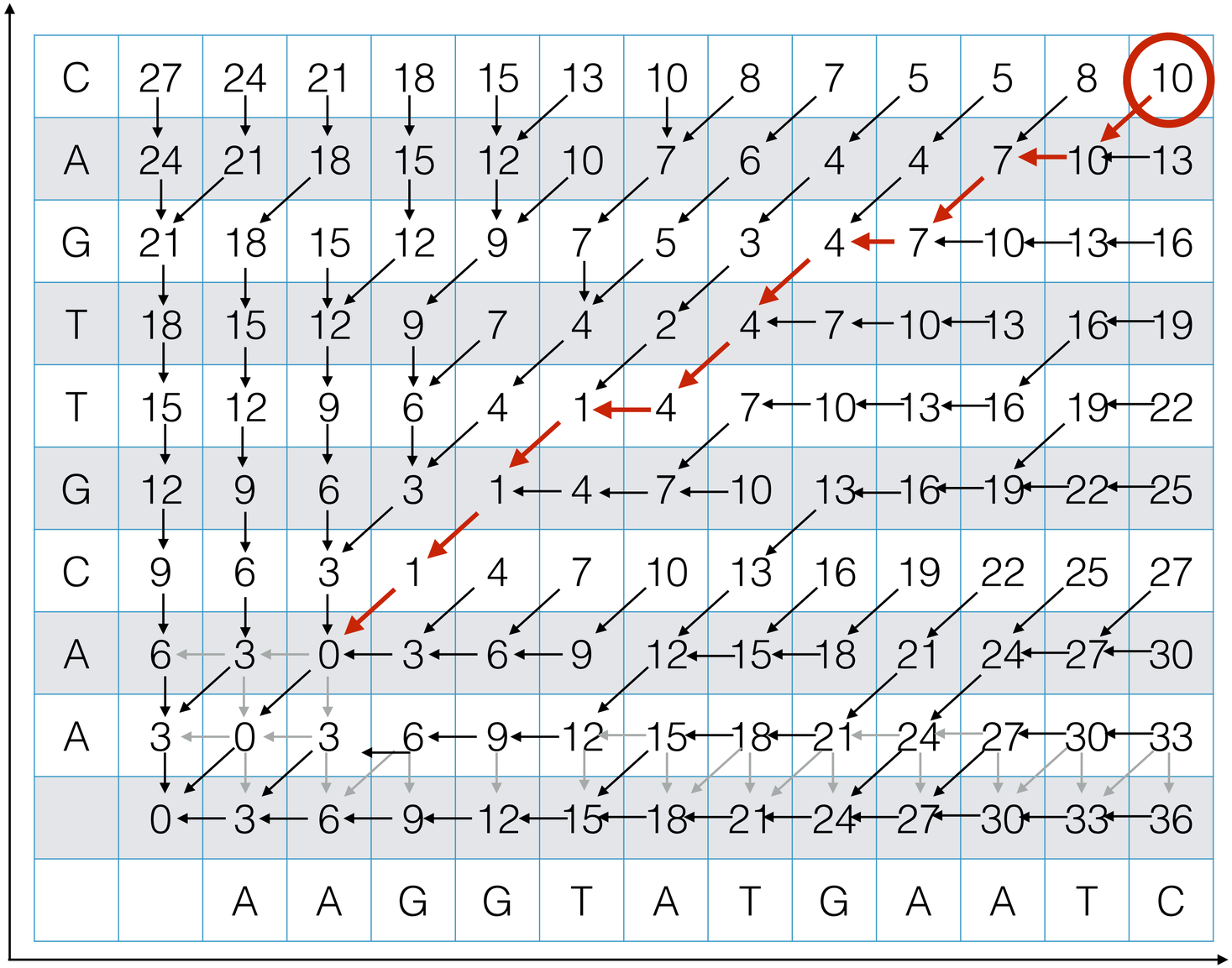
Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

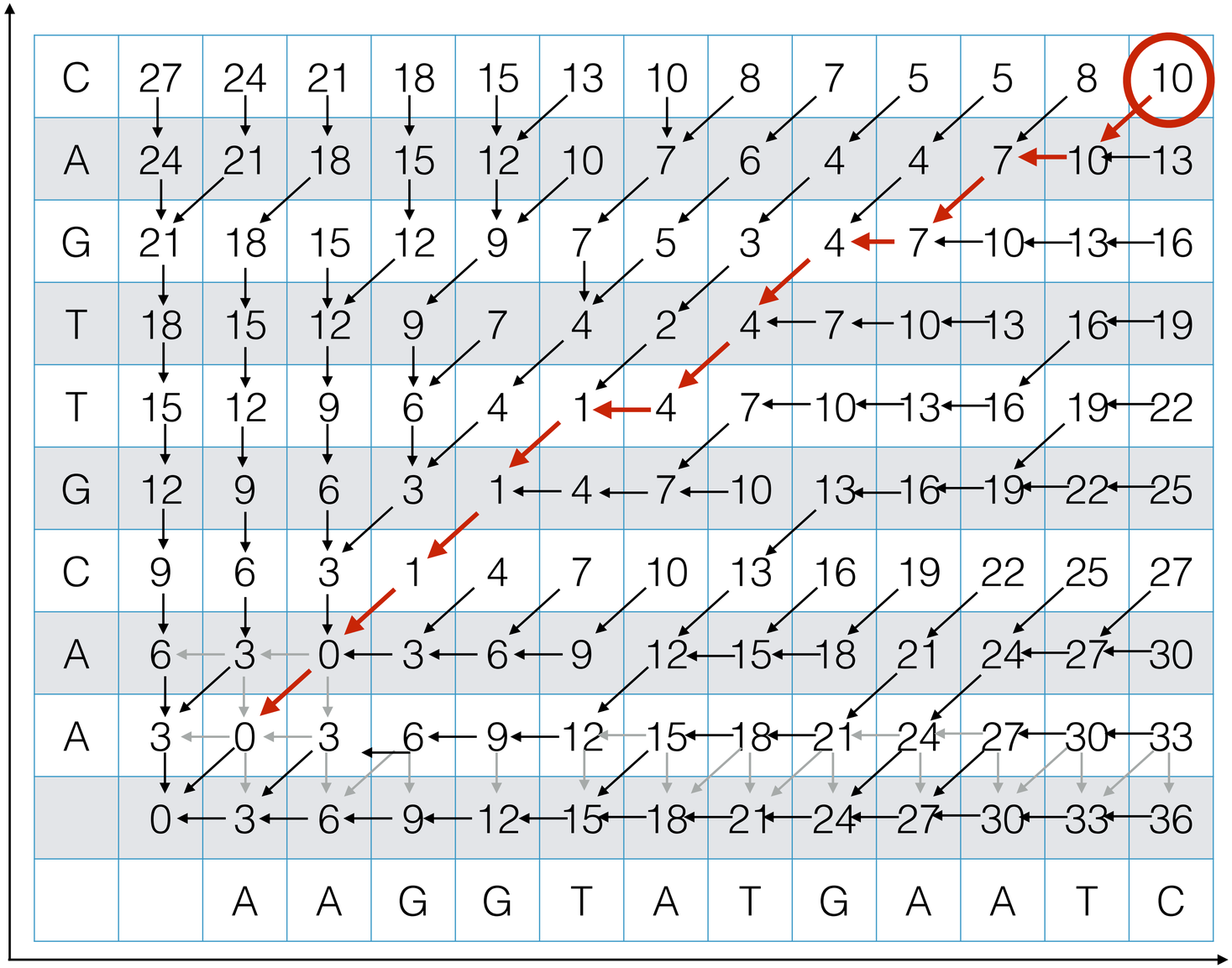
Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

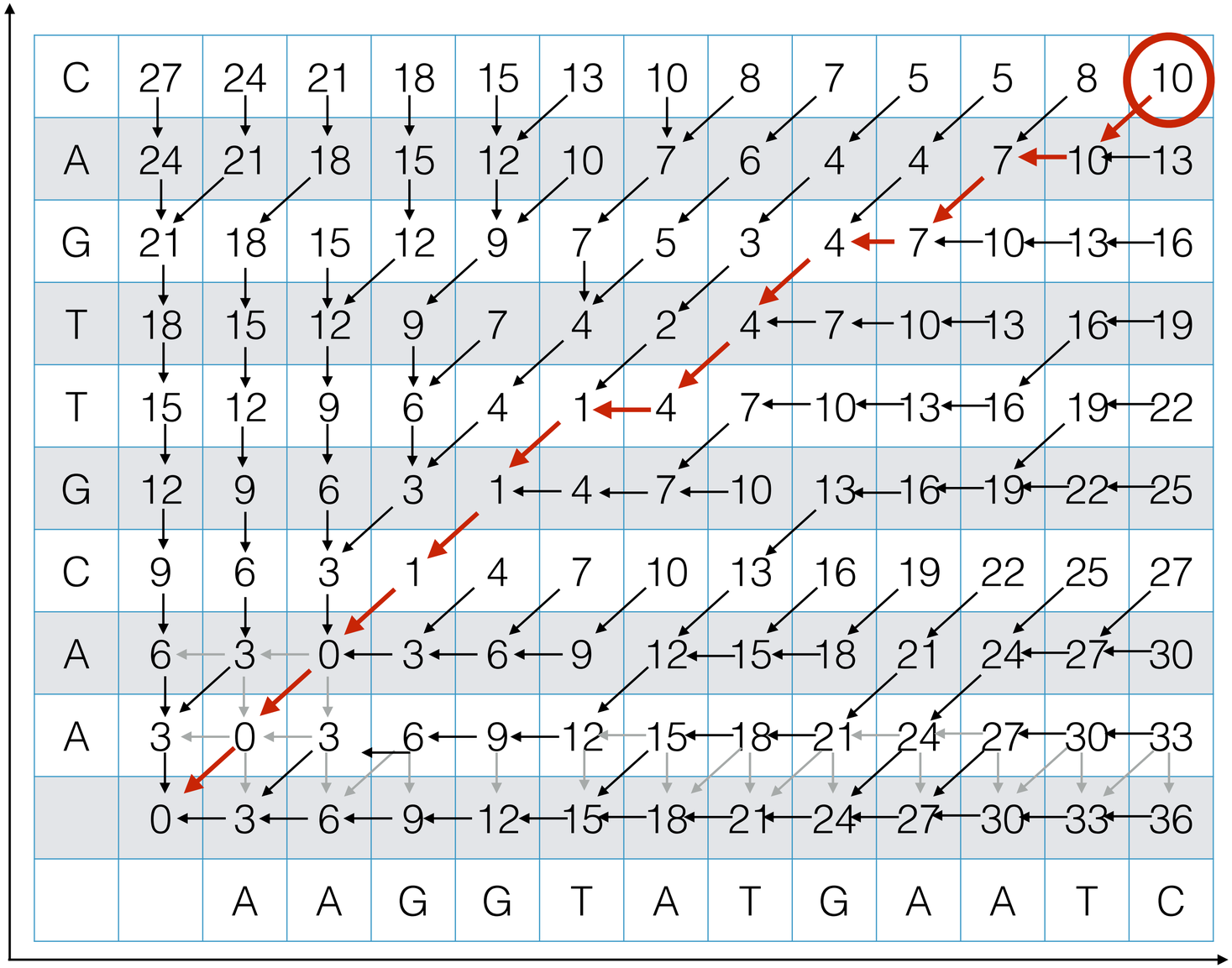
Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

Example


C 27 24 21 18 15 13 10 8 7 5 5 8 10
A 24 21 18 15 12 10 7 6 4 4 7 10 13
G 21 18 15 12 9 7 5 3 4 7 10 13 16
T 18 15 12 9 7 4 2 4 7 10 13 16 19
T 15 12 9 6 4 1 4 7 10 13 16 19 22
G 12 9 6 3 1 4 7 10 13 16 19 22 25
C 9 6 3 1 4 7 10 13 16 19 22 25 27
A 6 3 0 3 6 9 12 15 18 21 24 27 30
A 3 0 3 6 9 12 15 18 21 24 27 30 33
0 3 6 9 12 15 18 21 24 27 30 33 36
A A G G T A T G A A T C

Outputting the Alignment

Build the alignment from right to left.

```
ACGT
A-GA
```

Follow the backtrack pointers starting from entry (n,m).

- If you follow a diagonal pointer, add both characters to the alignment,
- If you follow a left pointer, add a gap to the y-axis string and add the x-axis character
- If you follow a down pointer, add the y-axis character and add a gap to the x-axis string.

*

Recap: Dynamic Programming

The previous sequence alignment / edit distance algorithm is an example of dynamic programming.

> **Main idea of dynamic programming:** solve the subproblems in an order so that when you need an answer, it's ready.

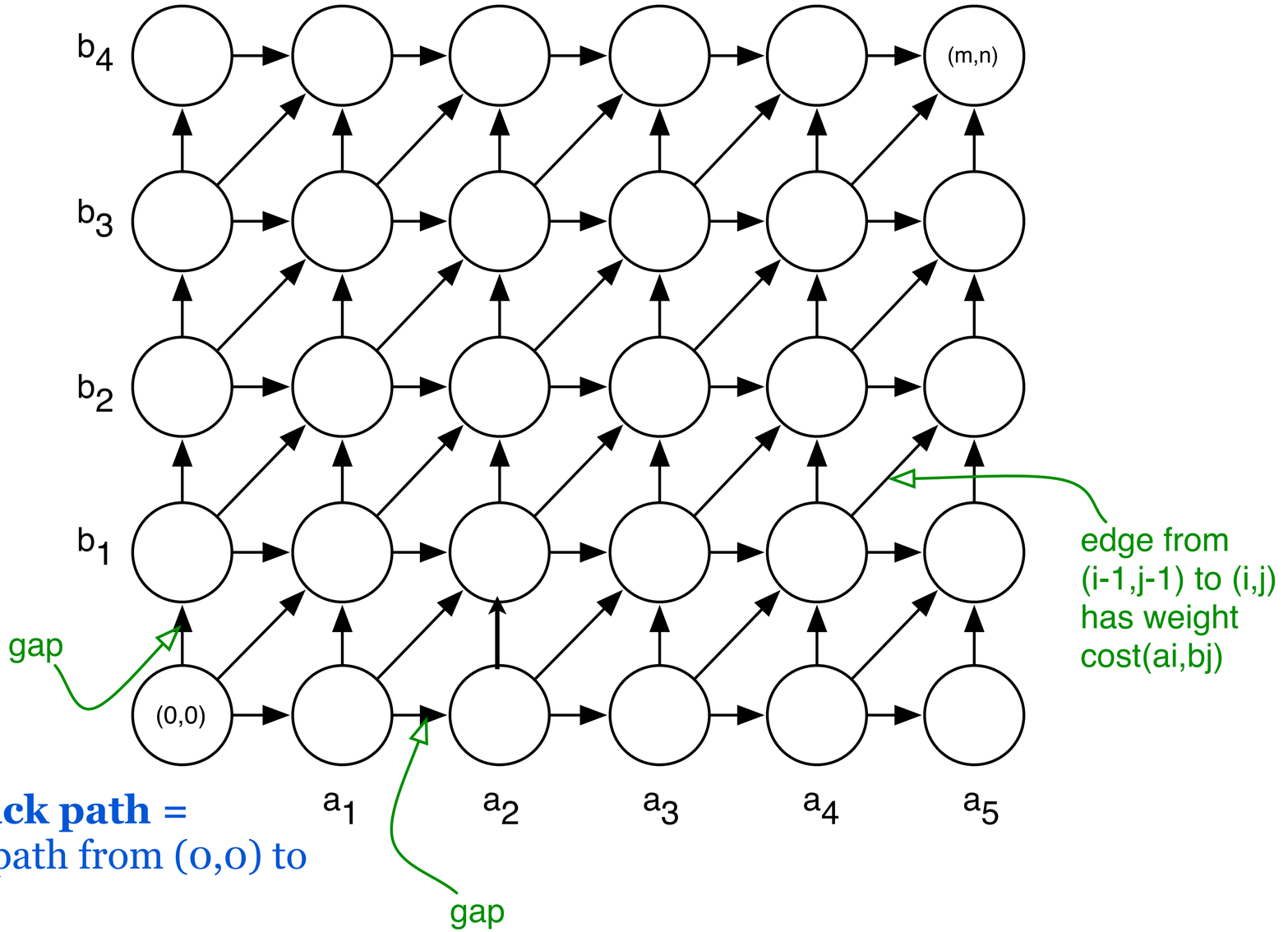
Requirements for DP to apply:

1. Optimal value of the original problem can be computed from some similar subproblems.

2. There are only a polynomial # of subproblems

3. There is a "natural" ordering of subproblems, so that you can solve a subproblem by only looking at **smaller** subproblems.

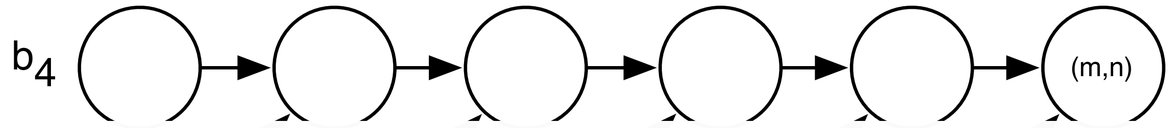
Another View: Recasting as a Graph



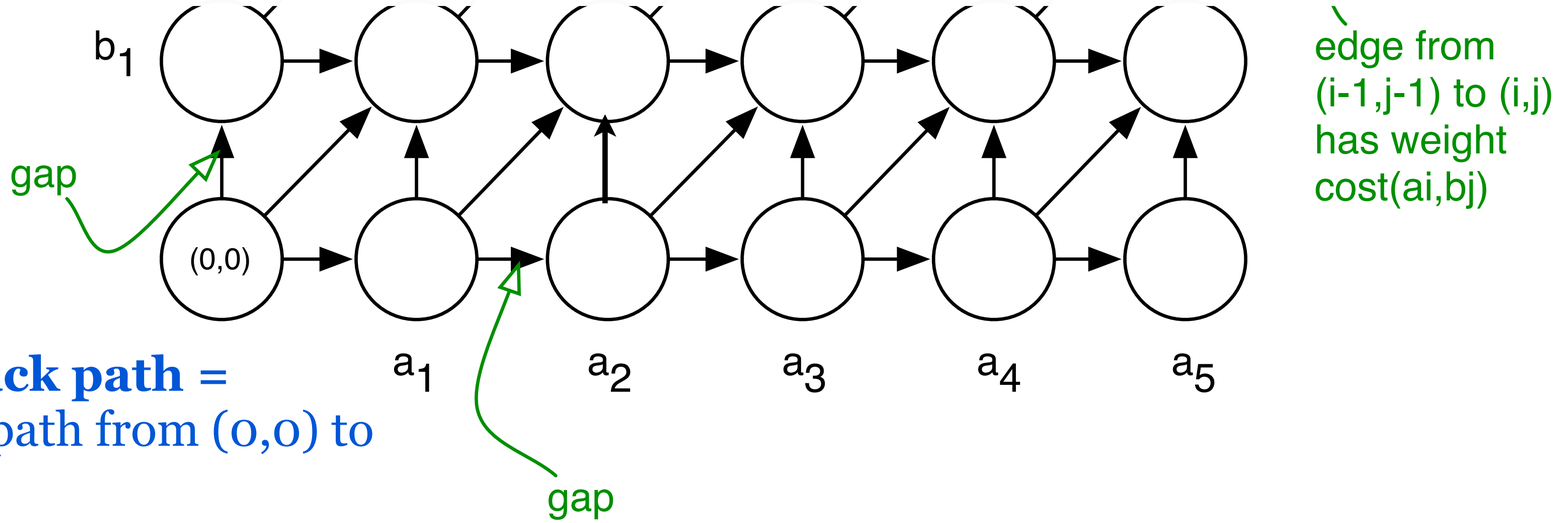
Traceback path =
shortest path from (0,0) to (m,n)

*

Another View: Recasting as a Graph



How would you find a shortest path in this graph *efficiently?*



Traceback path = shortest path from (0,0) to (m,n)

*

Semi-global and local alignment and gap penalties

Maximization vs. Minimization

Edit distance:

$$\mathrm{OPT}(i,j) = \min \begin{cases} \mathrm{cost}(x_i, y_j) + \mathrm{OPT}(i-1, j-1) & \text{match } x_i, y_j \\ c_{\mathrm{gap}} + \mathrm{OPT}(i-1, j) & x_i \text{ is unmatched} \\ c_{\mathrm{gap}} + \mathrm{OPT}(i, j-1) & y_j \text{ is unmatched} \end{cases}$$

Sequence Similarity: replace the *min* with a *max* — find the highest-scoring alignment. Gap costs and bad matches usually get a negative "score".

$$\mathrm{OPT}(i,j) = \max \begin{cases} \mathrm{score}(x_i, y_j) + \mathrm{OPT}(i-1, j-1) \\ s_{\mathrm{gap}} + \mathrm{OPT}(i-1, j) \\ s_{\mathrm{gap}} + \mathrm{OPT}(i, j-1) \end{cases}$$

gap penalty → gap score (probably negative)
match cost → match score

*

Alignment Categories

Global: Require an end-to-end alignment of **x**,**y**

Semi-global (glocal): Gaps at the beginning or end of **x** or **y** are free — useful when one string is significantly shorter than the other or for finding overlaps between strings

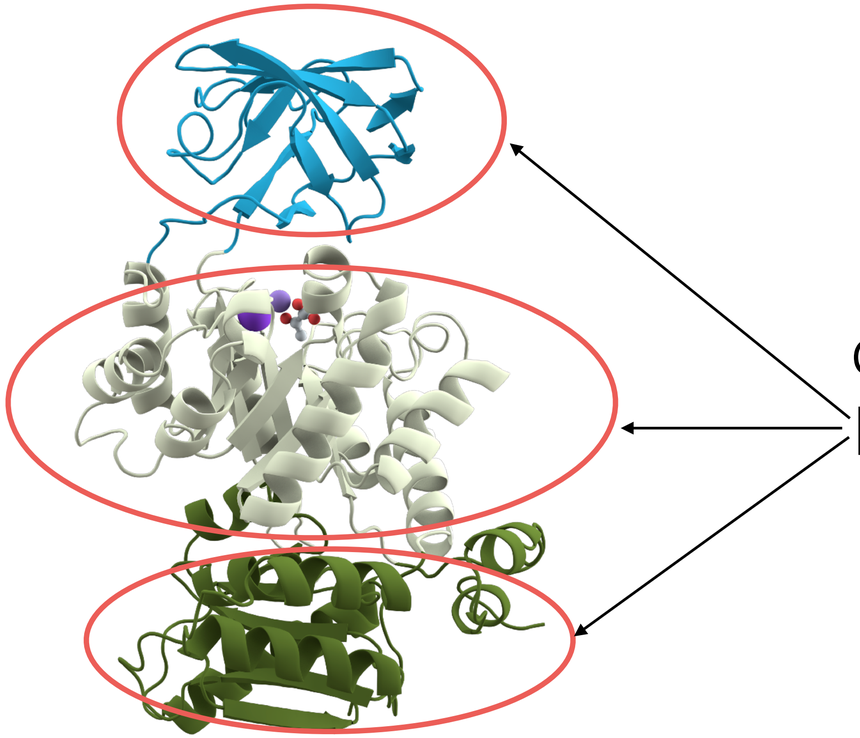
Local: Find the highest scoring alignment between **x'** a substring of **x** and **y'** a substring of **y** — useful for finding similar regions in strings that may not be globally similar

Alignment Categories Motivation

Global: **x** and **y** are similar proteins from closely-related species

Semi-global (glocal): **x** and **y** are sequencing reads we are trying to assemble. We want to find reads where the right end (suffix) of one matches the left end (prefix) of another.

Alignment Categories Motivation

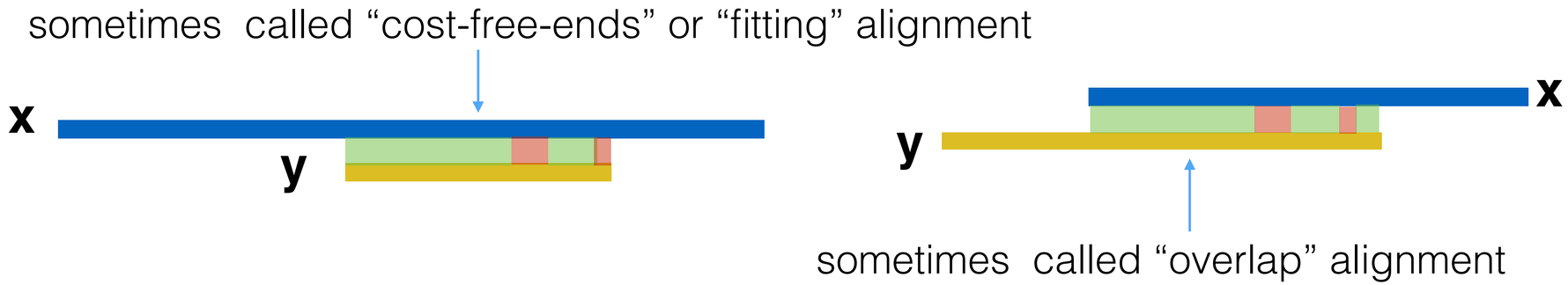


It's possible and somewhat common for specific domains to be conserved, but not the entire protein sequence / structure.

Local: **x** and **y** are similar proteins from potentially distantly related species. We don't expect the entire protein to be conserved, but certain "domains" should be.

Semi-global Alignment Example

Semi-global (glocal): Gaps at the beginning or end of **x** or **y** are free. Useful when one one string is significantly shorter than the other or we want to find an overlap between the suffix of one string and a prefix of the other



Motivation:

Useful for finding similarities that global alignments wouldn’t. Also, can view “read mapping” as a variant of the semi-global alignment problem. Want to align entire read but it’s a tiny fraction of the genome. *Note*: won’t use semi-global alignment with the full genome for read mapping in practice.

Semi-global Alignment Example

Semi-global (glocal): Gaps at the beginning or end of **x** or **y** are free — one useful case is when one string is significantly shorter than the other

sometimes called “cost-free-ends” or “fitting” alignment

We’ll discuss the “fitting” variant for in the next few slides for simplicity, but the same basic idea applies for the “overlap” variant as well.

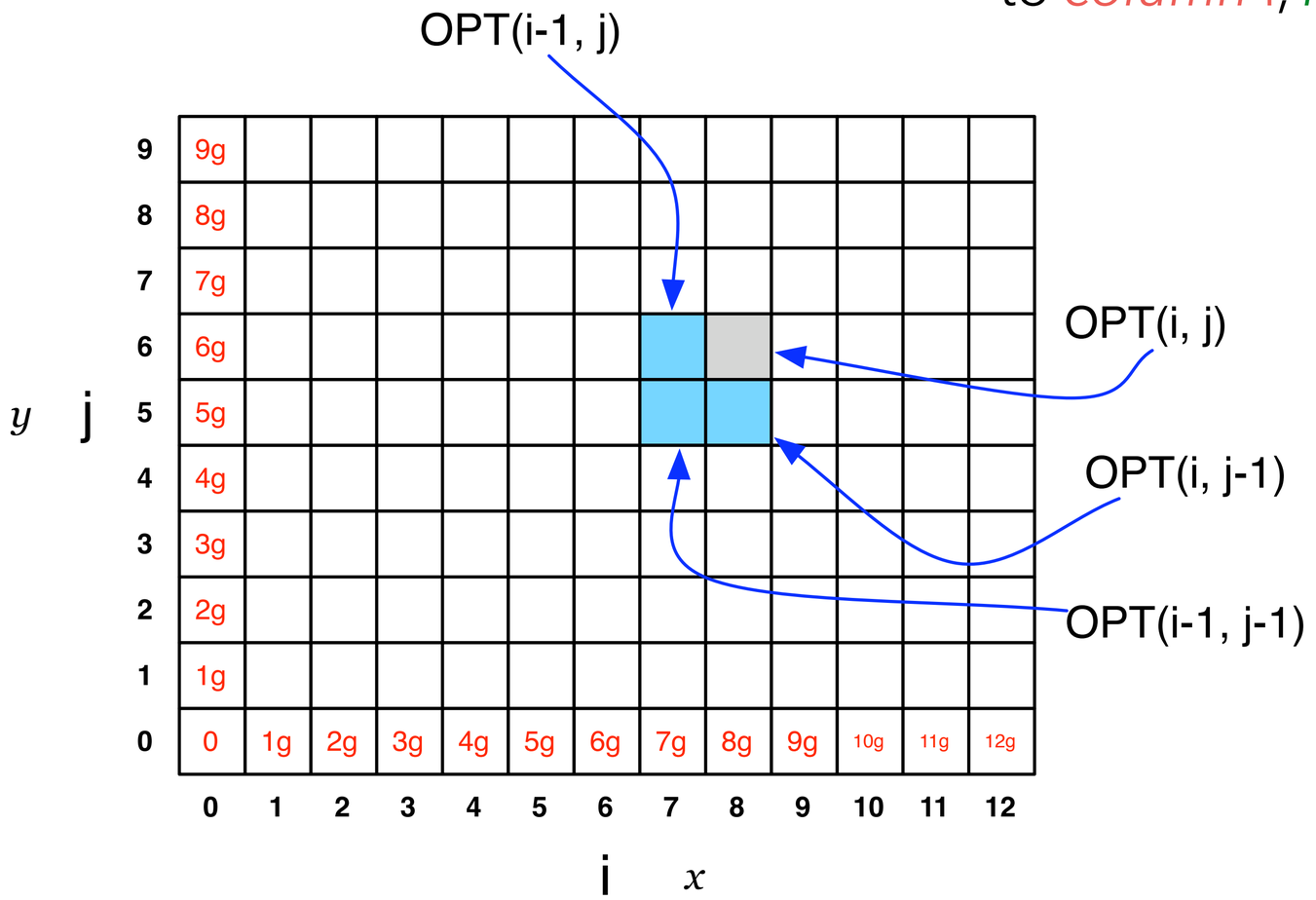
Recall: Global Alignment Matrix

OPT(*i*,*j*) contains the score for the best alignment between:

the first *i* characters of string *x* [**prefix** *i* of *x*]

the first *j* character of string *y* [**prefix** *j* of *y*]

NOTE: observe the non-standard notation here; OPT(**i**,**j**) is referring to *column* i, *row* j of the matrix.



*

How to do semi-global alignment?

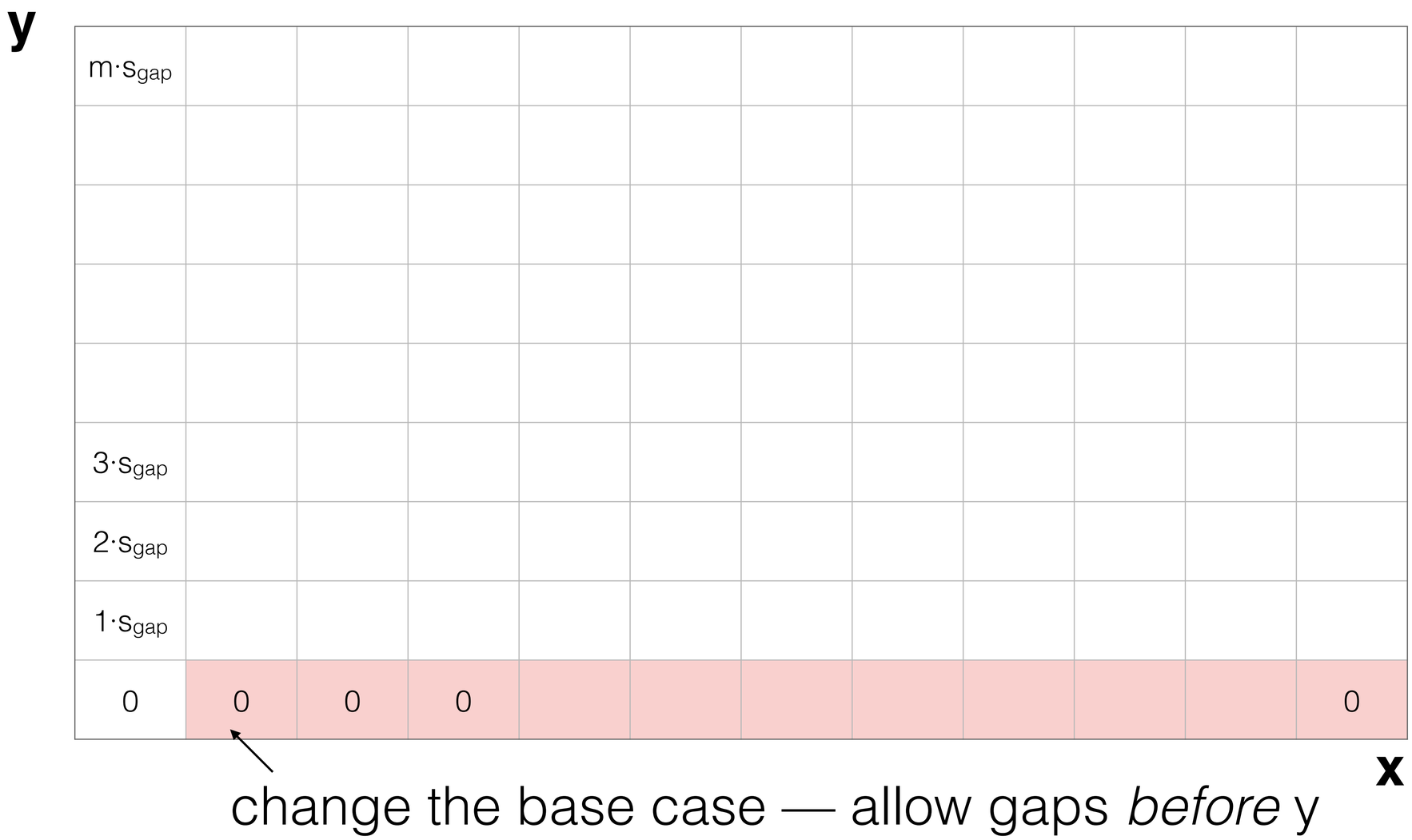
y

$m \cdot s_{gap}$											
$3 \cdot s_{gap}$											
$2 \cdot s_{gap}$											
$1 \cdot s_{gap}$											
0	$1 \cdot s_{gap}$	$2 \cdot s_{gap}$	$3 \cdot s_{gap}$								$n \cdot s_{gap}$

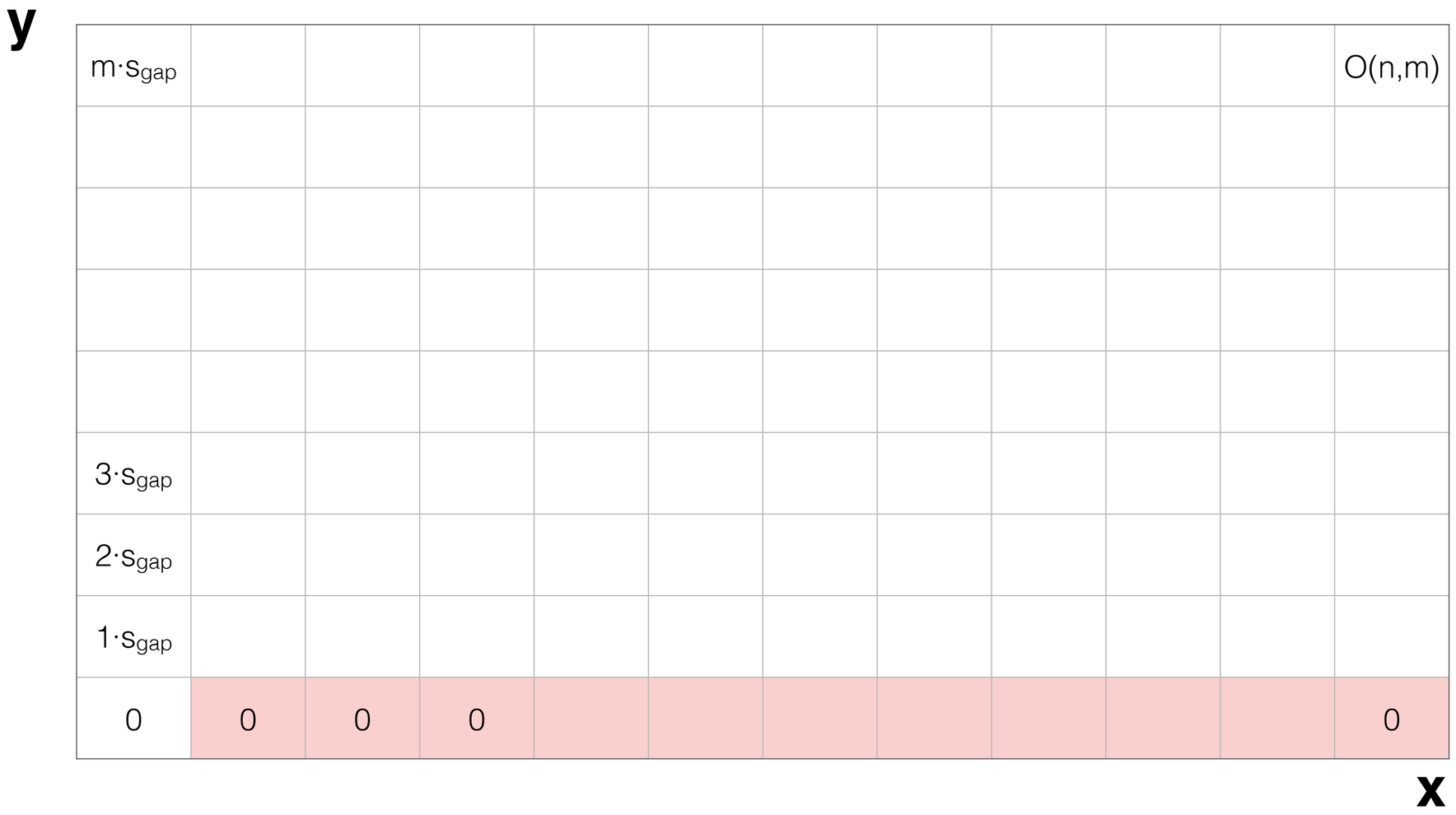
x

Start with the original global alignment matrix

How to do semi-global alignment?

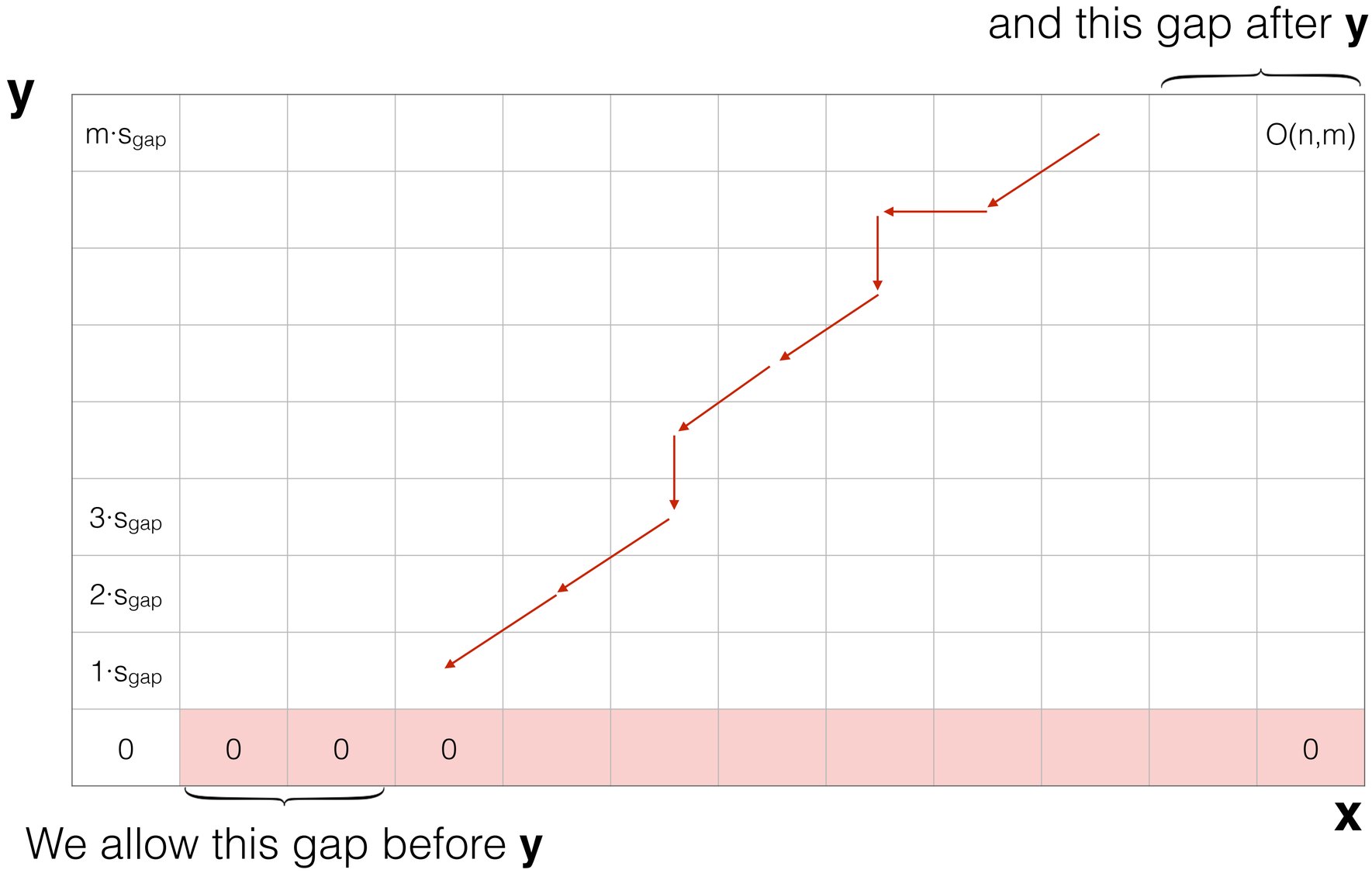


How to do semi-global alignment?



start traceback at $\max_{0<i\leq n} OPT(i,m)$ — this allows gaps after **y**; why?

Semi-global alignment example



Semi-global Alignment

What is the same and different between the “global” and semi-global (“fitting”) alignment problems?

*assuming |y| < |x| and we are “fitting” y into x

Global	Semi-global (“fitting”)
$\mathrm{OPT}(i,j) = \max \begin{cases} \mathrm{score}(x_i, y_j) + \mathrm{OPT}(i-1, j-1) \\ \mathrm{s_{gap}} + \mathrm{OPT}(i-1, j) \\ \mathrm{s_{gap}} + \mathrm{OPT}(i, j-1) \end{cases}$	$\mathrm{OPT}(i,j) = \max \begin{cases} \mathrm{score}(x_i, y_j) + \mathrm{OPT}(i-1, j-1) \\ \mathrm{s_{gap}} + \mathrm{OPT}(i-1, j) \\ \mathrm{s_{gap}} + \mathrm{OPT}(i, j-1) \end{cases}$
Base case: OPT(i,0) = i x s_{gap}	Base case: OPT(i,0) = 0
Traceback starts at OPT(n,m)	Traceback starts at **max** OPT(j,m) $0<j\leq n$

Semi-global Alignment

The recurrence remains the *same*, we only change the base case of the recurrence and the origin of the backtrack

1) Ignore gaps before x → change base case; OPT(0,j) = 0

2) Ignore gaps after x → change traceback; start from $\max_{0<j\leq m} OPT(n,j)$

3) Ignore gaps before y → change base case; OPT(i,0) = 0

4) Ignore gaps after y → change traceback; start from $\max_{0<i\leq n} OPT(i,m)$

Semi-global Alignment

1) Ignore gaps before x
2) Ignore gaps after x
3) Ignore gaps before y
4) Ignore gaps after y

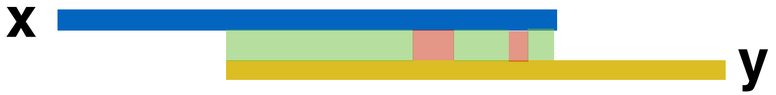
Types of semi-global alignments

use mods 3&4

use mods 1&4

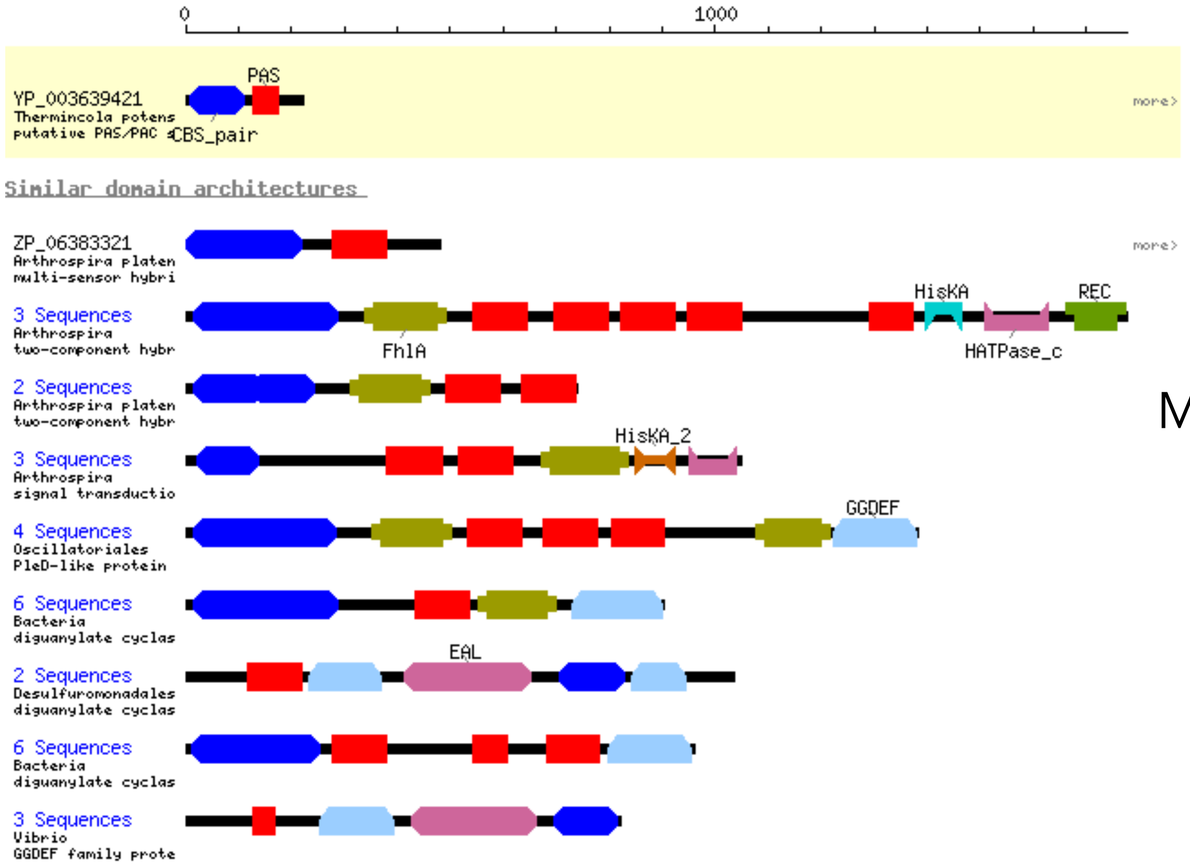
use mods 1&2

use mods 2&3



Local Alignment

Local alignment between a and b: Best alignment between a subsequence of a and **a** subsequence of **b**.



Motivation:

Many genes are composed of *domains*, which are subsequences that perform a particular function.

*

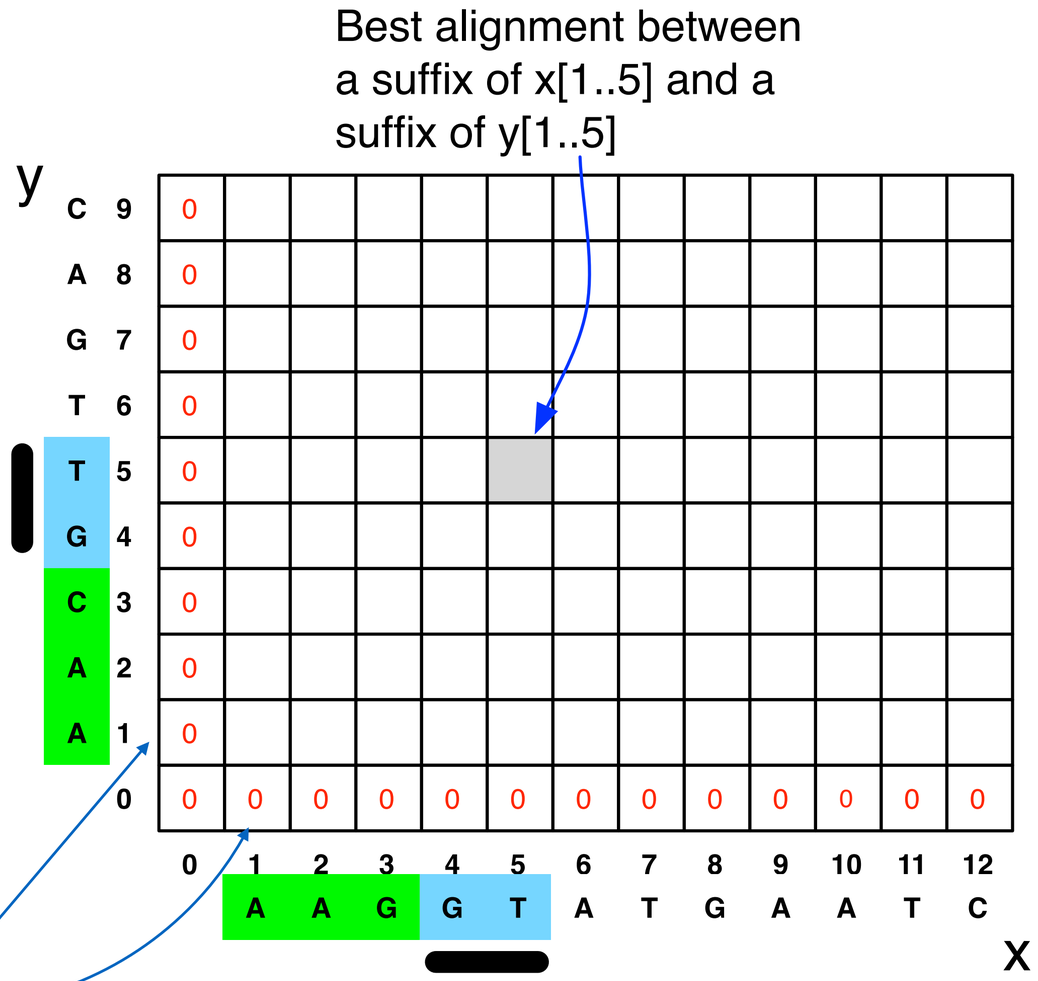
Local Alignment

New meaning of entry of matrix entry:

OPT(i, j) = best score between:

some suffix of $x[1...i]$

some suffix of $y[1...j]$



Same base-case
trick we used in semi-global alignment

Local Alignment

New meaning of entry of matrix entry:

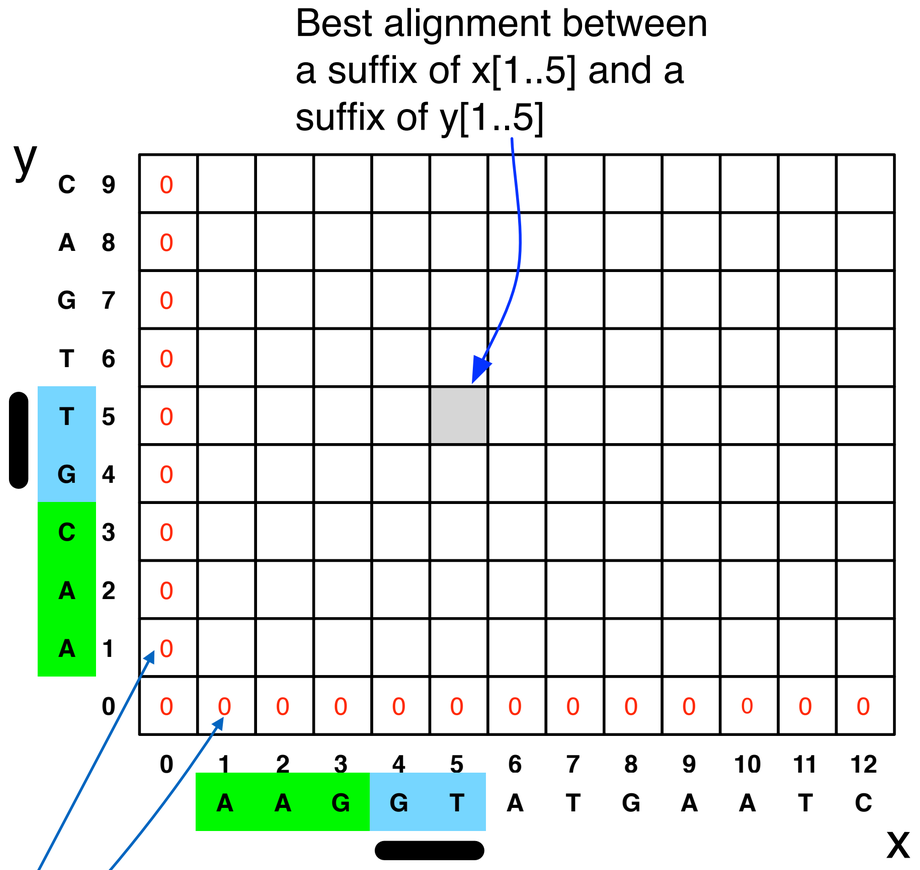
OPT(i, j) = best score between:

some suffix of $x[1...i]$

some suffix of $y[1...j]$

What else do we need to change to allow local alignments?

Hint: The empty alignment is always a valid local alignment!



Same base-case trick we used in semi-global alignment

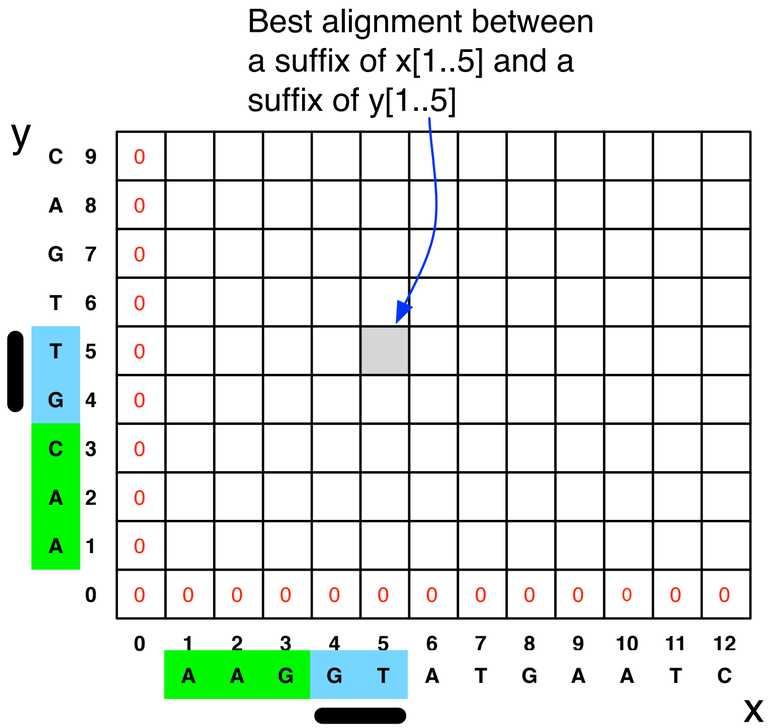
How do we fill in the local alignment matrix?

$$\mathrm{OPT}(i,j) = \max \begin{cases} \mathrm{score}(x_i, y_j) + \mathrm{OPT}(i-1, j-1) & (1) \\ s_{\mathrm{gap}} + \mathrm{OPT}(i-1, j) & (2) \\ s_{\mathrm{gap}} + \mathrm{OPT}(i, j-1) & (3) \\ 0 \end{cases}$$

(1), (2), and (3): same cases as before: match x and y, gap in y, gap in x

New case: 0 allows you to say the best alignment between a suffix of *x* and a suffix of *y* is the empty alignment.

Lets us "start over"



*

Local Alignment

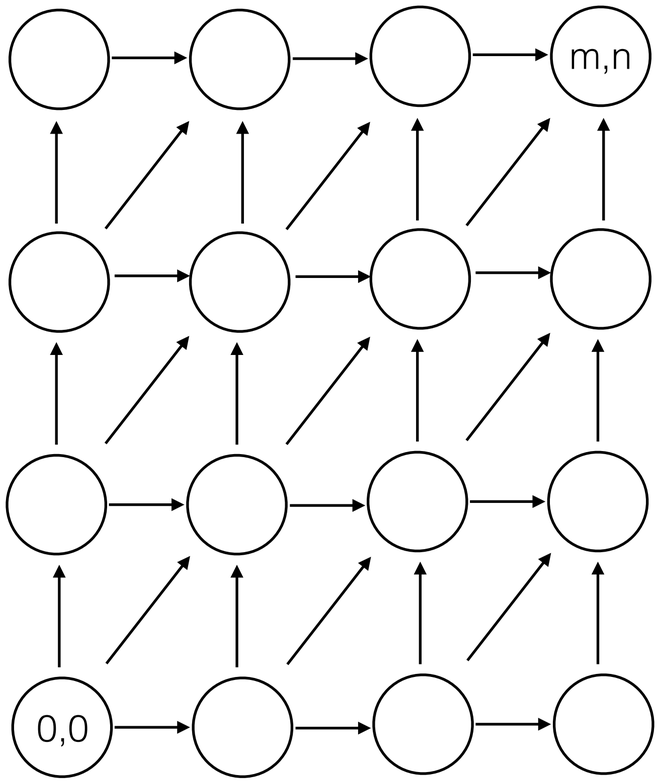
- Initialize first row and first column to be 0.

- The score of the best local alignment is the largest value in the entire array.

- To find the actual local alignment:
  - start at an entry with the maximum score
  - traceback as usual
  - stop when we reach an entry with a score of 0

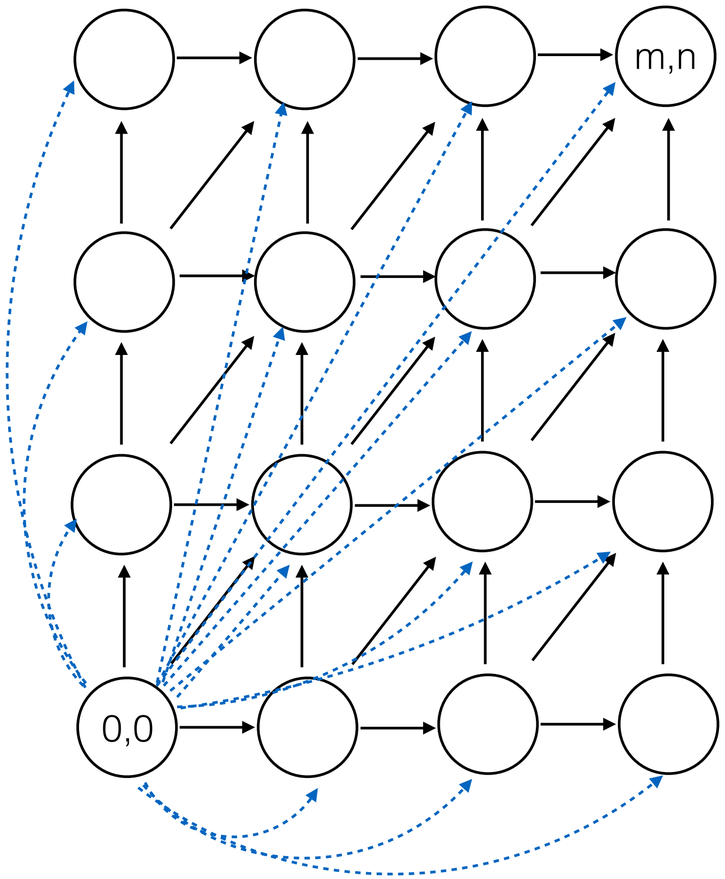
*

Local Alignment in the DAG framework



Local Alignment in the DAG framework

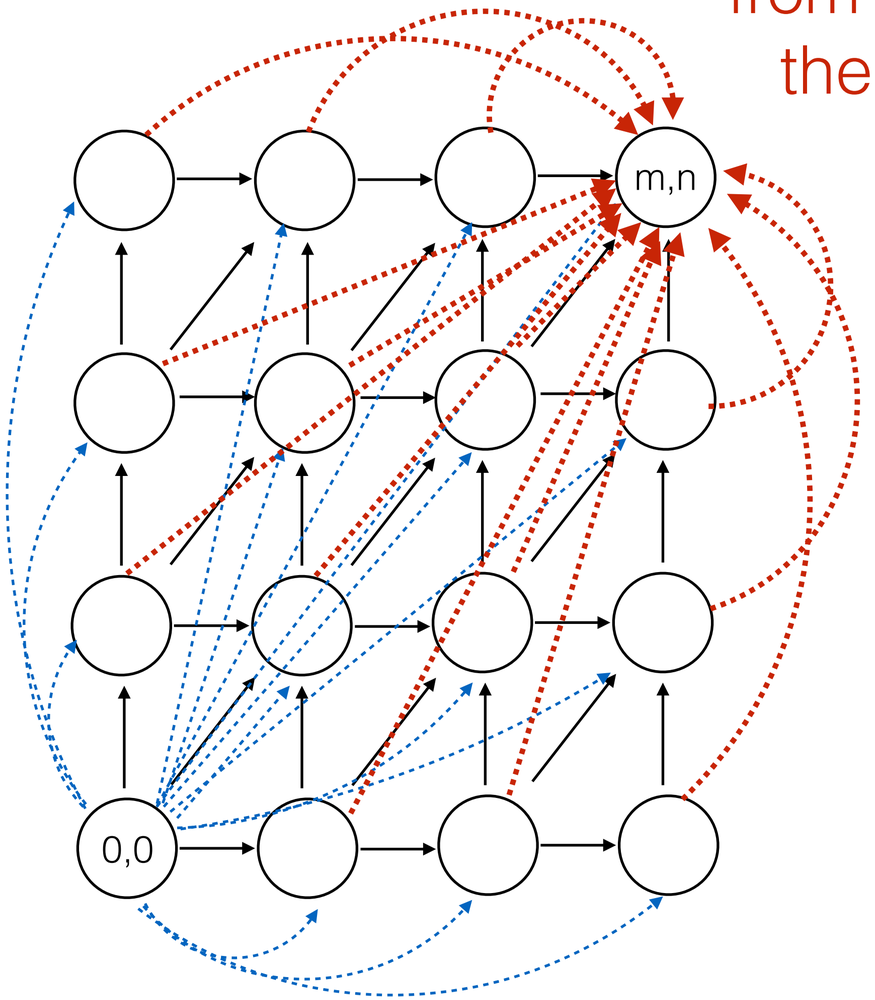
Add 0 score edge from the source to every node



Local Alignment in the DAG framework

Add 0 score edge from the source to every vertex

Add 0 score edge from every vertex to the target vertex



Local Alignment Example #1

`local align("AGCGTAG", "CTCGTC")`

	*	A	G	C	G	T	A	G
*	0	0	0	0	0	0	0	0
C	0	0	0	10	3	0	0	0
T	0	0	0	3	5	13	6	0
C	0	0	0	↖10	3	6	8	1
G	0	0	10	3	↖20	13	6	18
T	0	0	3	5	13	↖**30**	23	16
C	0	0	0	13	6	23	25	18

Score(match) = 10
Score(mismatch) = -5
Score(gap) = -7

Note: this table written top-to-bottom instead of bottom-to-top

*

Local Alignment Example #2

local align("bestoftimes", "soften")

	*	b	e	s	t	o	f	t	i	m	e	s
*	0	0	0	0	0	0	0	0	0	0	0	0
s	0	0	0	10	3	0	0	0	0	0	0	10
o	0	0	0	3	5	13	6	0	0	0	0	3
f	0	0	0	0	0	6	23	16	9	2	0	0
t	0	0	0	0	10	3	16	33	26	19	12	5
e	0	0	10	3	3	5	9	26	28	21	29	22
n	0	0	3	5	0	0	2	19	21	23	22	24

Score(match) = 10
Score(mismatch) = -5
Score(gap) = -7

Note: this table written top-to-bottom instead of bottom-to-top

*

More Local Alignment Examples

Score(match) = 10
Score(mismatch) = -5
Score(gap) = -7

local align("catdogfish", "dog")

	*	c	a	t	d	o	g	f	i	s	h
*	0	0	0	0	0	0	0	0	0	0	0
d	0	0	0	0	10	3	0	0	0	0	0
o	0	0	0	0	3	20	13	6	0	0	0
g	0	0	0	0	0	13	**30**	23	16	9	2

local align("mississippi", "issp")

	*	m	i	s	s	i	s	s	i	p	p	i
*	0	0	0	0	0	0	0	0	0	0	0	0
i	0	0	10	3	0	10	3	0	10	3	0	10
s	0	0	3	20	13	6	20	13	6	5	0	3
s	0	0	0	13	30	23	16	30	23	16	9	2
p	0	0	0	6	23	25	18	23	25	**33**	26	19

local align("aaaa", "aa")

	*	a	a	a	a
*	0	0	0	0	0
a	0	10	10	10	10
a	0	10	**20**	**20**	**20**

*

Local / Global Recap

- Alignment score sometimes called the “edit distance” between two strings.
- Edit distance is sometimes called Levenshtein distance.

- Algorithm for local alignment is sometimes called “Smith-Waterman”
- Algorithm for global alignment is sometimes called “Needleman-Wunsch”

- Same basic algorithm, however.
- Underlies BLAST

*

Side Note: Lower Bounds

- Suppose the lengths of *x* and *y* are *n*.
- Clearly, need at least Ω(n) time to find their global alignment (have to read the strings!)
- The DP algorithms show global alignment can be done in $O(n^2)$ time.

#: Backurs, Arturs, and Piotr Indyk. "Edit distance cannot be computed in strongly subquadratic time (unless SETH is false)." *Proceedings of the forty-seventh annual ACM symposium on Theory of computing*. ACM, 2015.

*

Side Note: Lower Bounds

- Suppose the lengths of *x* and *y* are *n*.
- Clearly, need at least Ω(n) time to find their global alignment (have to read the strings!)
- The DP algorithms show global alignment can be done in $O(n^2)$ time.
- A trick called the "Four Russians Speedup" can make a similar dynamic programming algorithm run in $O(n^2 / \log n)$ time.
  - We probably won't talk about the Four Russians Speedup.
  - The important thing to remember is that only one of the four authors is Russian...

    (Alrazarov, Dinic, Kronrod, Faradzev, 1970)
- Open questions: Can we do better? Can we prove that we can't do better? No#

#: Backurs, Arturs, and Piotr Indyk. "Edit distance cannot be computed in strongly subquadratic time (unless SETH is false)." *Proceedings of the forty-seventh annual ACM symposium on Theory of computing*. ACM, 2015.

*

Space-efficient alignment

Space is often the limiting factor

O(nm) time is a problem, but as I've said, we **strongly believe** we can't to much better.

Can we do better in terms of *space?*

It turns out we can — at the same asymptotic time complexity!

Combining dynamic programming with the divide-and-conquer algorithm design technique.

Hirshberg's algorithm

Warmup — optimal *score* in linear space

Consider our DP matrix:

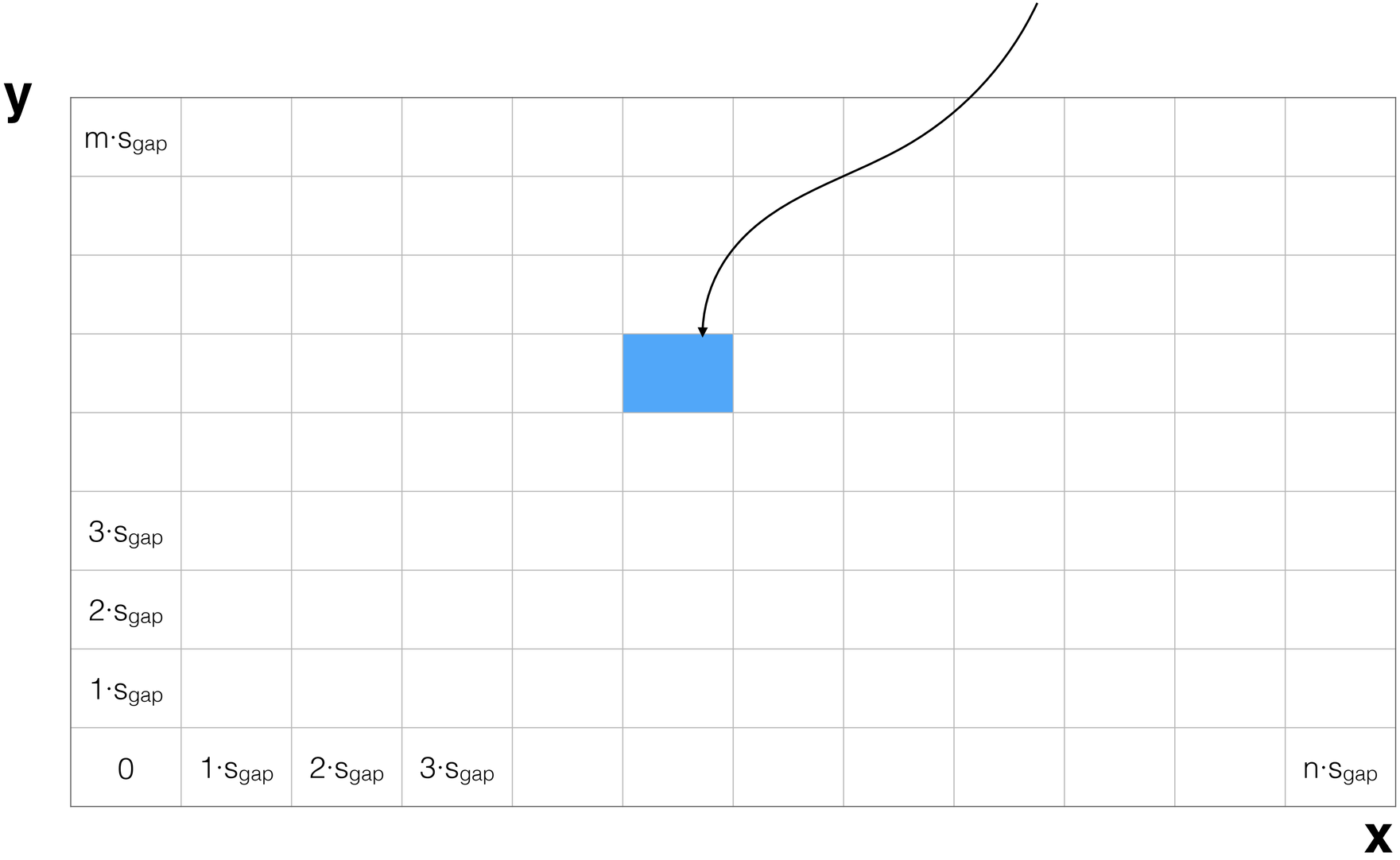
y

$m \cdot s_{gap}$											
$3 \cdot s_{gap}$											
$2 \cdot s_{gap}$											
$1 \cdot s_{gap}$											
0	$1 \cdot s_{gap}$	$2 \cdot s_{gap}$	$3 \cdot s_{gap}$								$n \cdot s_{gap}$

x

Warmup — optimal *score* in linear space

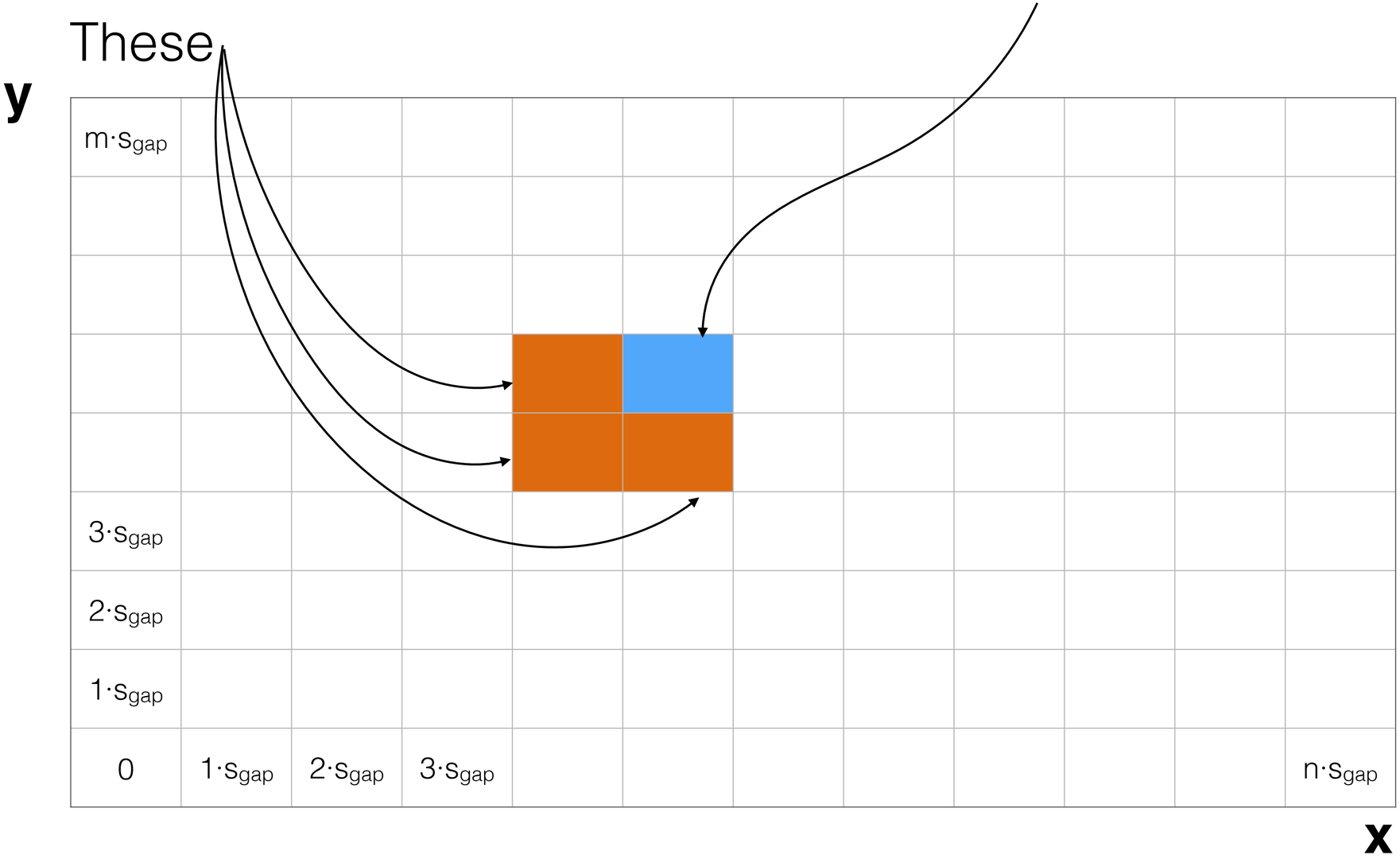
What scores to I need to know to fill in the answer here?



Warmup — optimal *score* in linear space

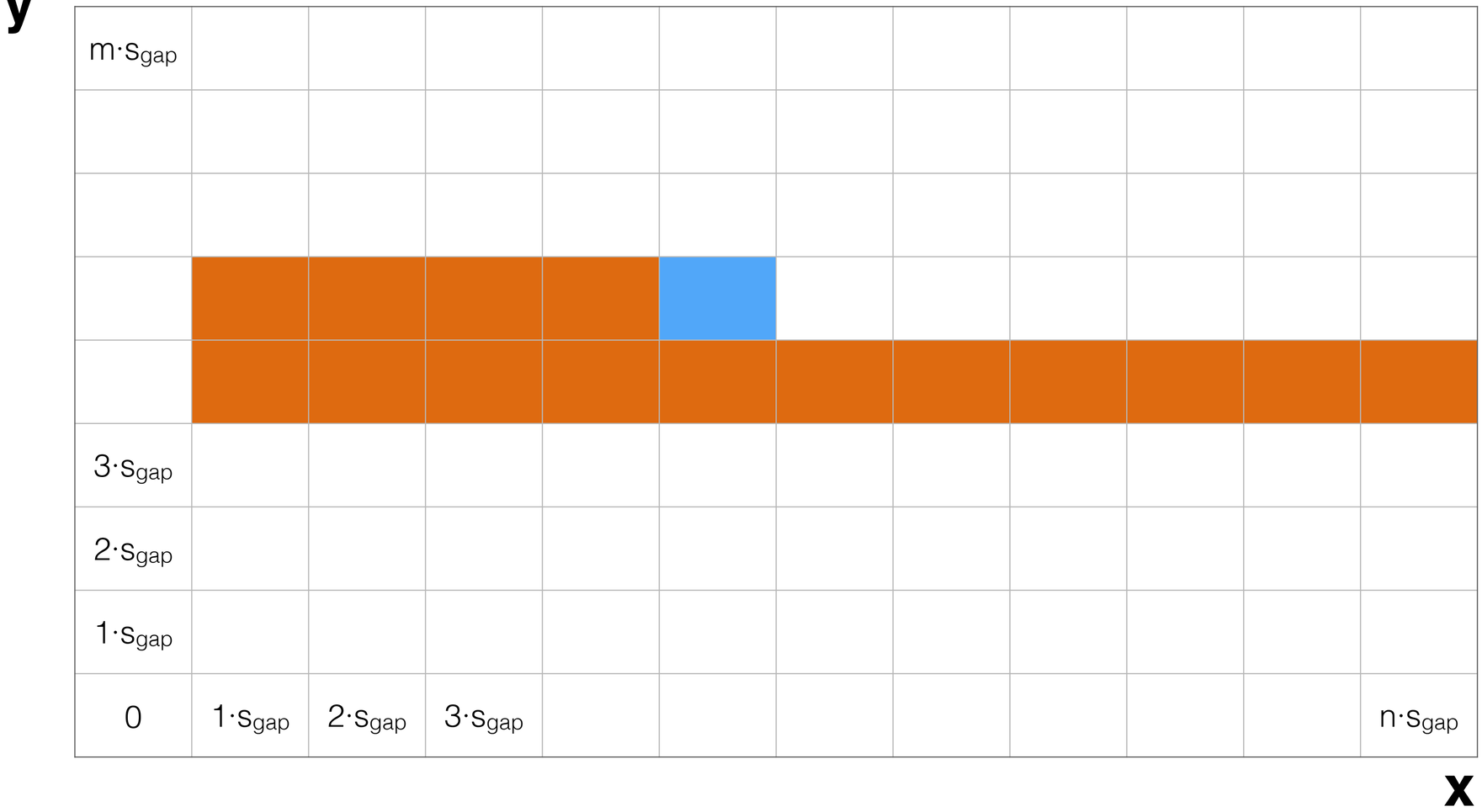
What scores to I need to know to fill in the answer here?

These



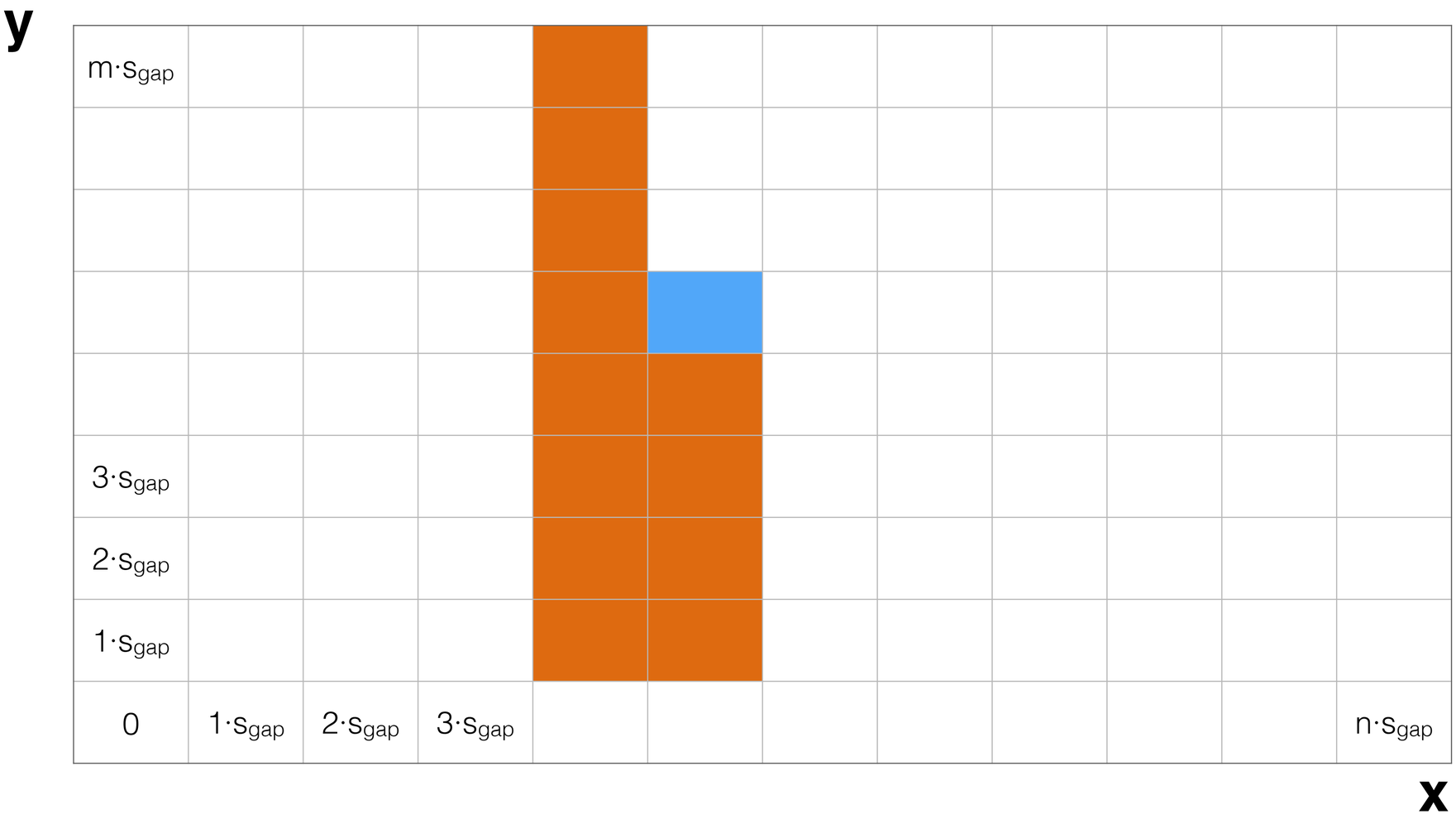
Warmup — optimal *score* in linear space

If we fill rows left - right, and bottom to top, to fill in row i, we *only* need scores from row i-1.



Warmup — optimal *score* in linear space

Columns also work; if we go left - right, and bottom to top, to fill in column i, we *only* need scores from col i-1.



Warmup — optimal *score* in linear space

If we fill rows left - right, and bottom to top, to fill in row i, we *only* need scores from row i-1.

Thus, we can compute the optimal *score,* keeping at most 2 rows / columns in memory at once.

Each row / column is *linear* in the length of one of the strings, and so we can compute the optimal *score*, in *linear space.*

How can we compute the optimal *alignment*?

This method won't work for computing the optimal alignment; we need *all* rows to be able to follow the backtracking arrows.

How can we find the optimal *alignment* in linear space?

Hirschberg's algorithm provides a solution.

Re-using subproblems

Consider, again, the meaning of the DP matrix

What is contained in the highlighted row?

y

$m \cdot s_{gap}$											
$3 \cdot s_{gap}$											
$2 \cdot s_{gap}$											
$1 \cdot s_{gap}$											
0	$1 \cdot s_{gap}$	$2 \cdot s_{gap}$	$3 \cdot s_{gap}$								$n \cdot s_{gap}$

x

Re-using subproblems

Consider, again, the meaning of the DP matrix

score of *every* prefix of **x** against *all* of **y** in this row

y

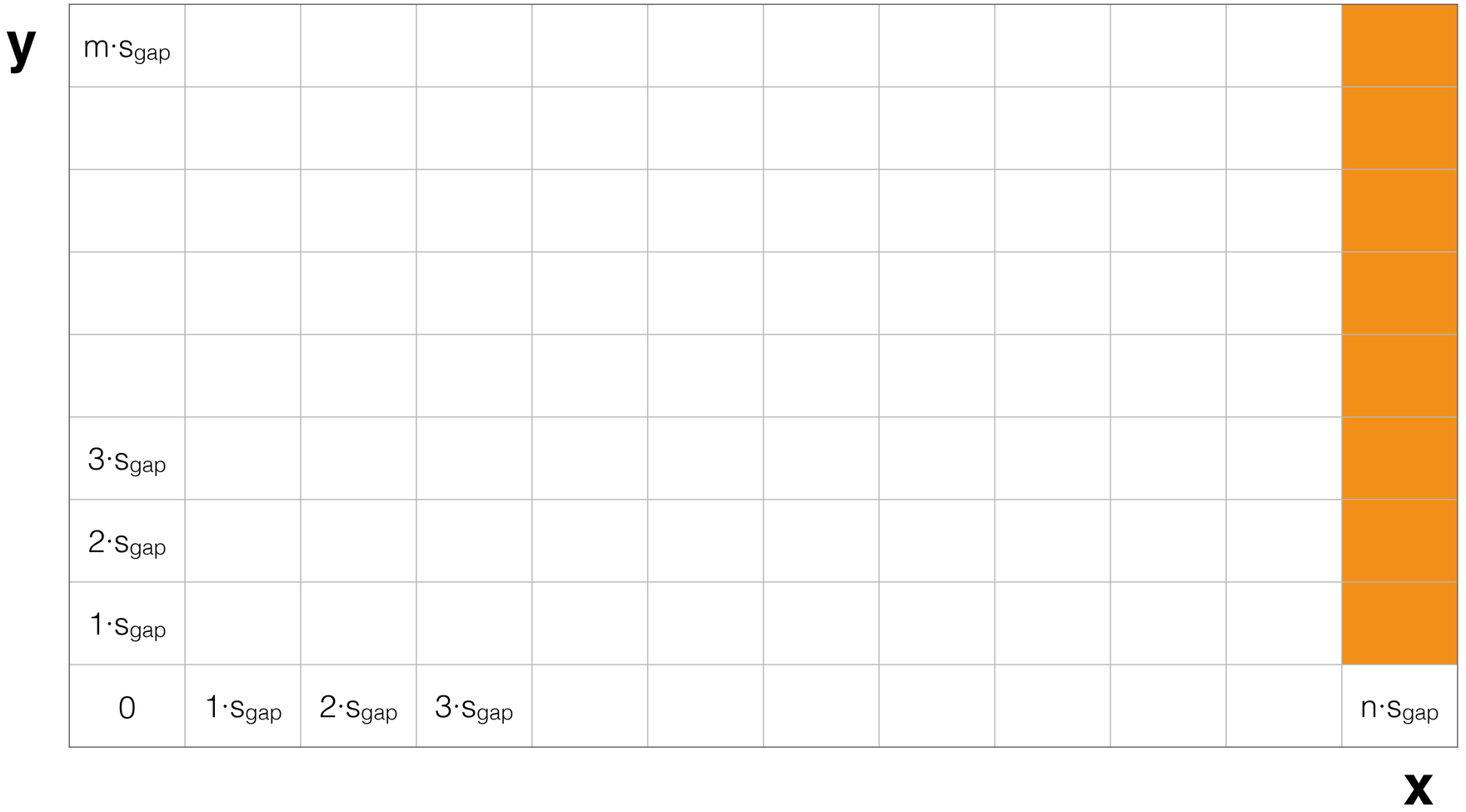
$m \cdot s_{gap}$											
$3 \cdot s_{gap}$											
$2 \cdot s_{gap}$											
$1 \cdot s_{gap}$											
0	$1 \cdot s_{gap}$	$2 \cdot s_{gap}$	$3 \cdot s_{gap}$								$n \cdot s_{gap}$

x

Re-using subproblems

Consider, again, the meaning of the DP matrix

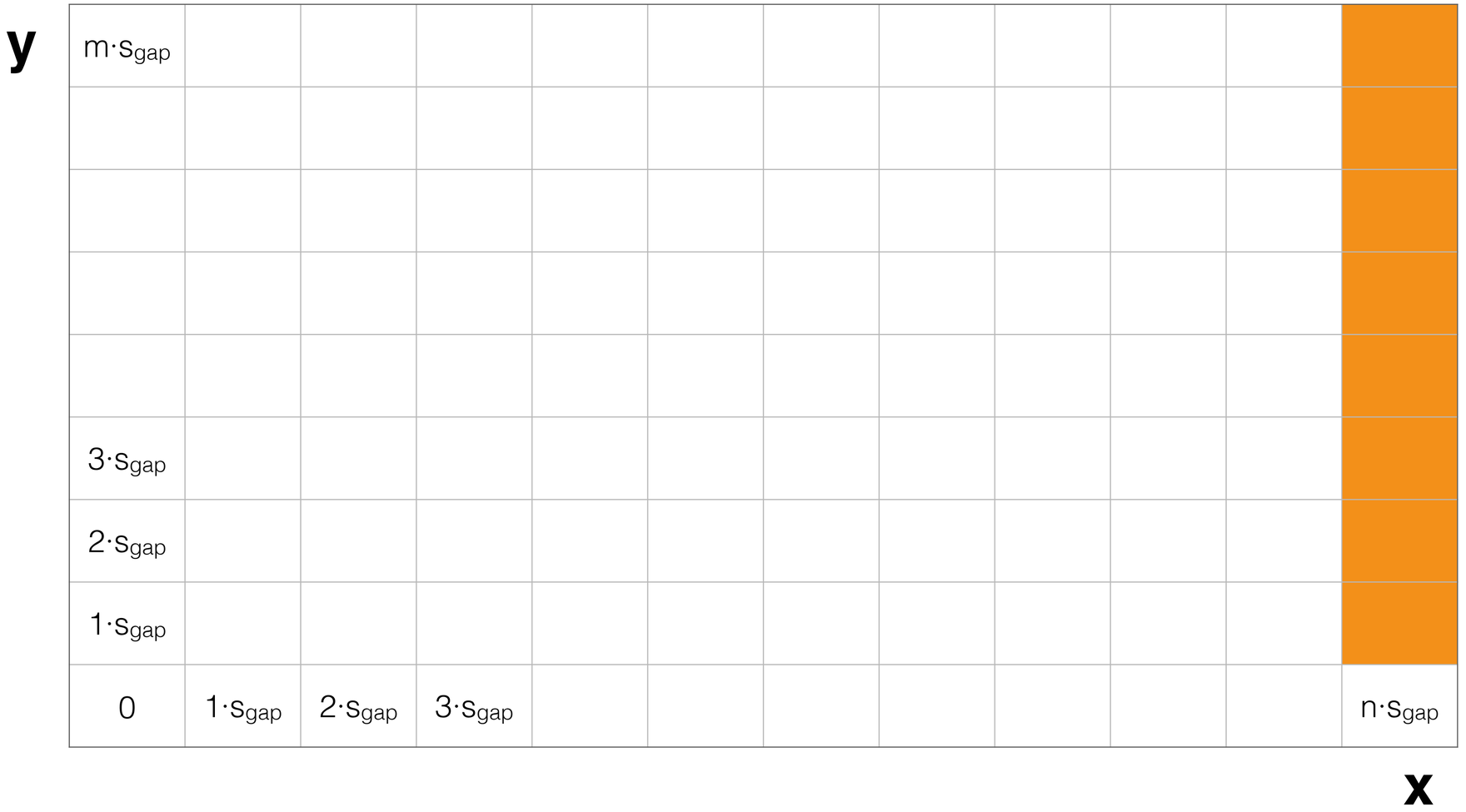
What is contained in the highlighted column?



Re-using subproblems

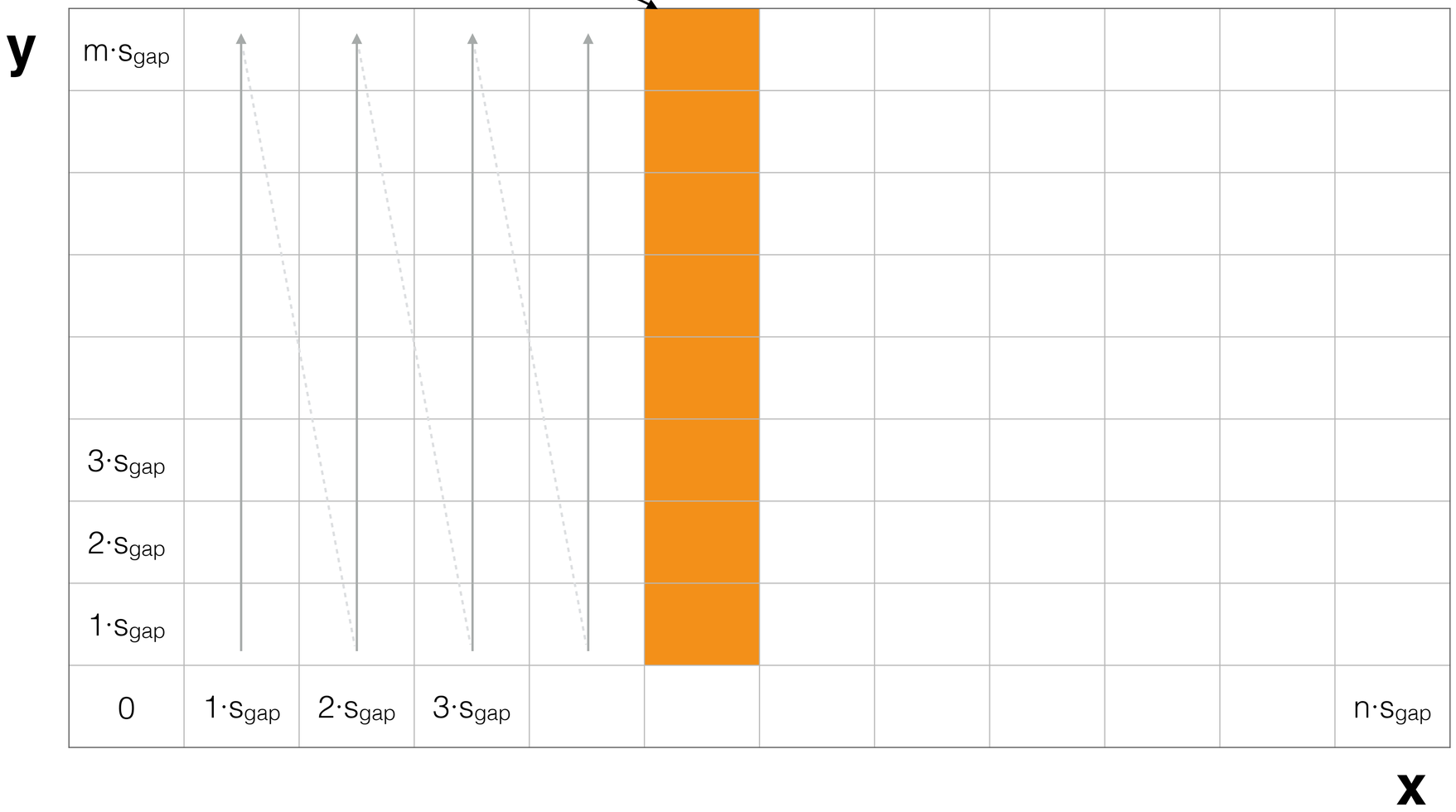
Consider, again, the meaning of the DP matrix

score of *every* prefix of **y** against *all* of **x** in this column



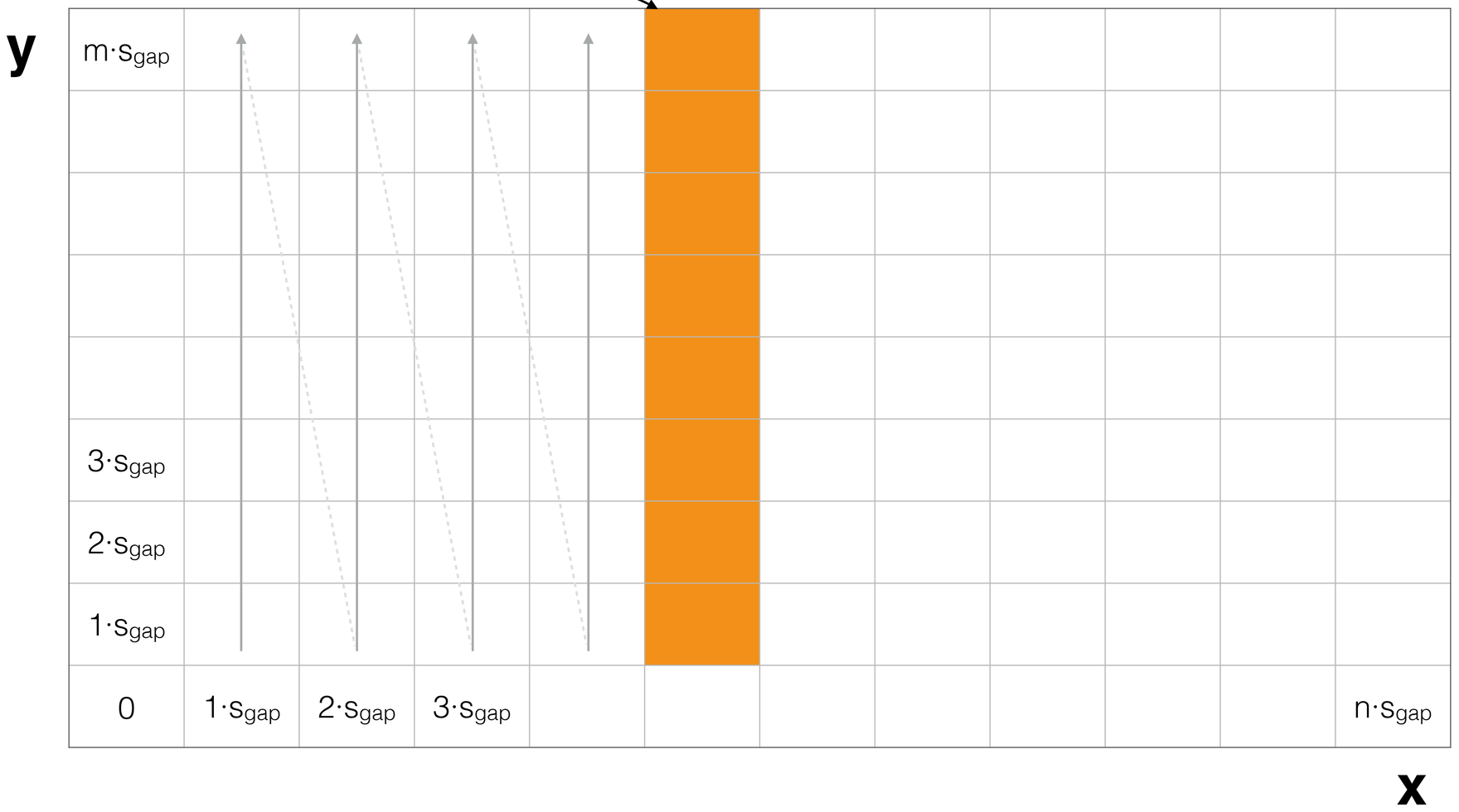
Re-using subproblems

score of *every* prefix of **y** against ith prefix of **x** in the ith column. How do we get these values efficiently?



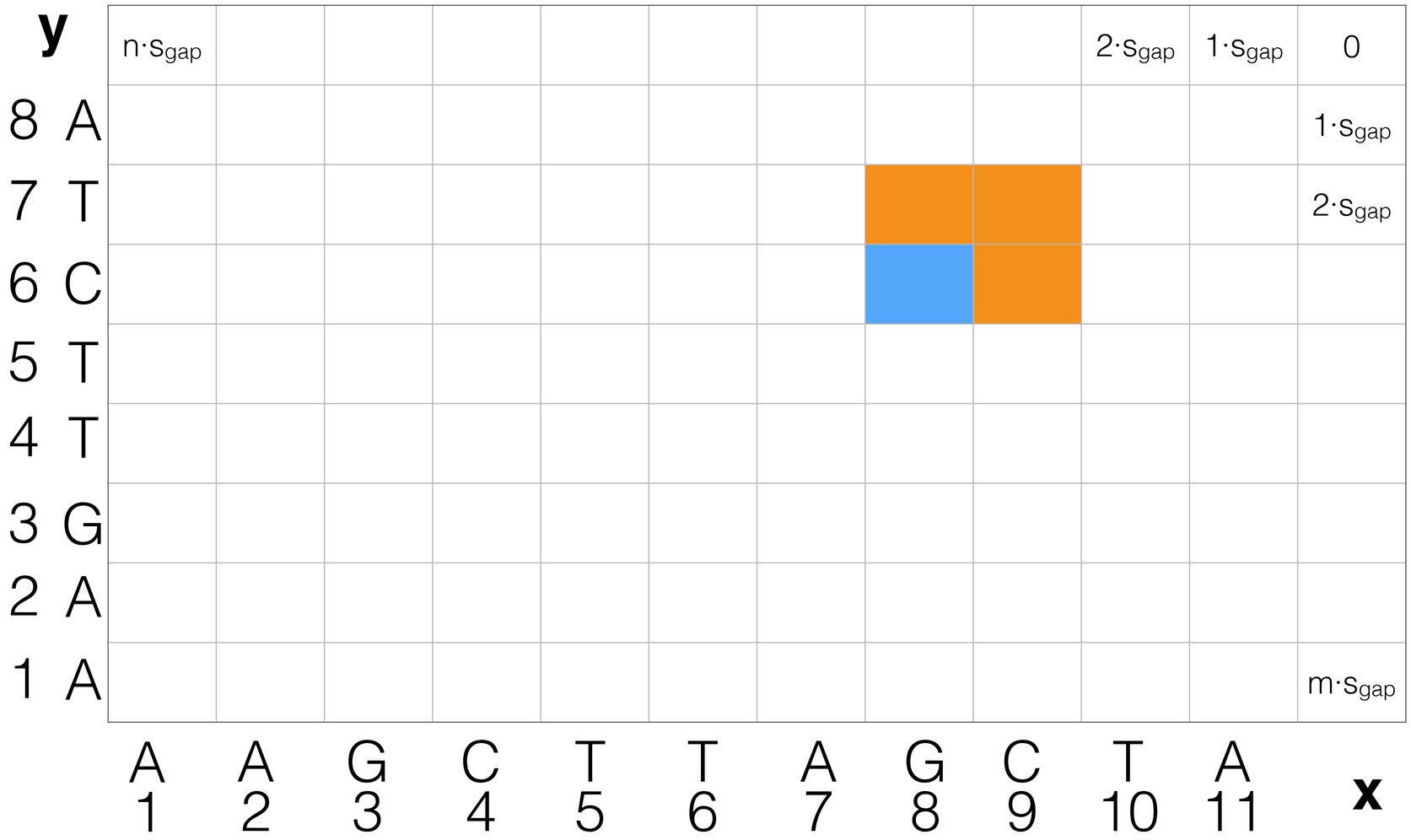
Re-using subproblems

score of *every* prefix of **y** against ith prefix of **x** in the ith column. Easy if we fill in by columns instead of rows.

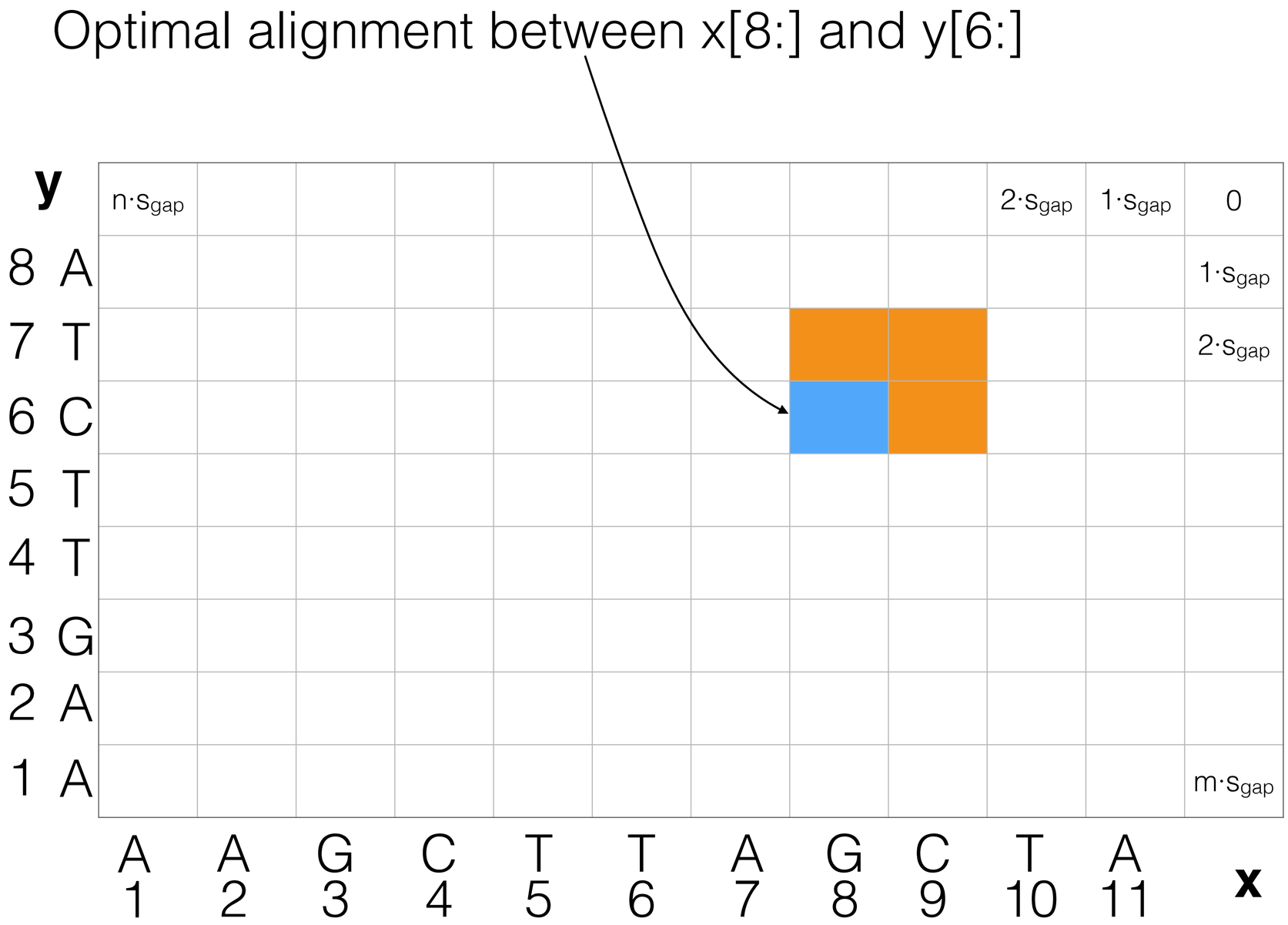


What about suffixes?

Consider filling in the DP matrix from the *opposite* direction (top right to bottom left)

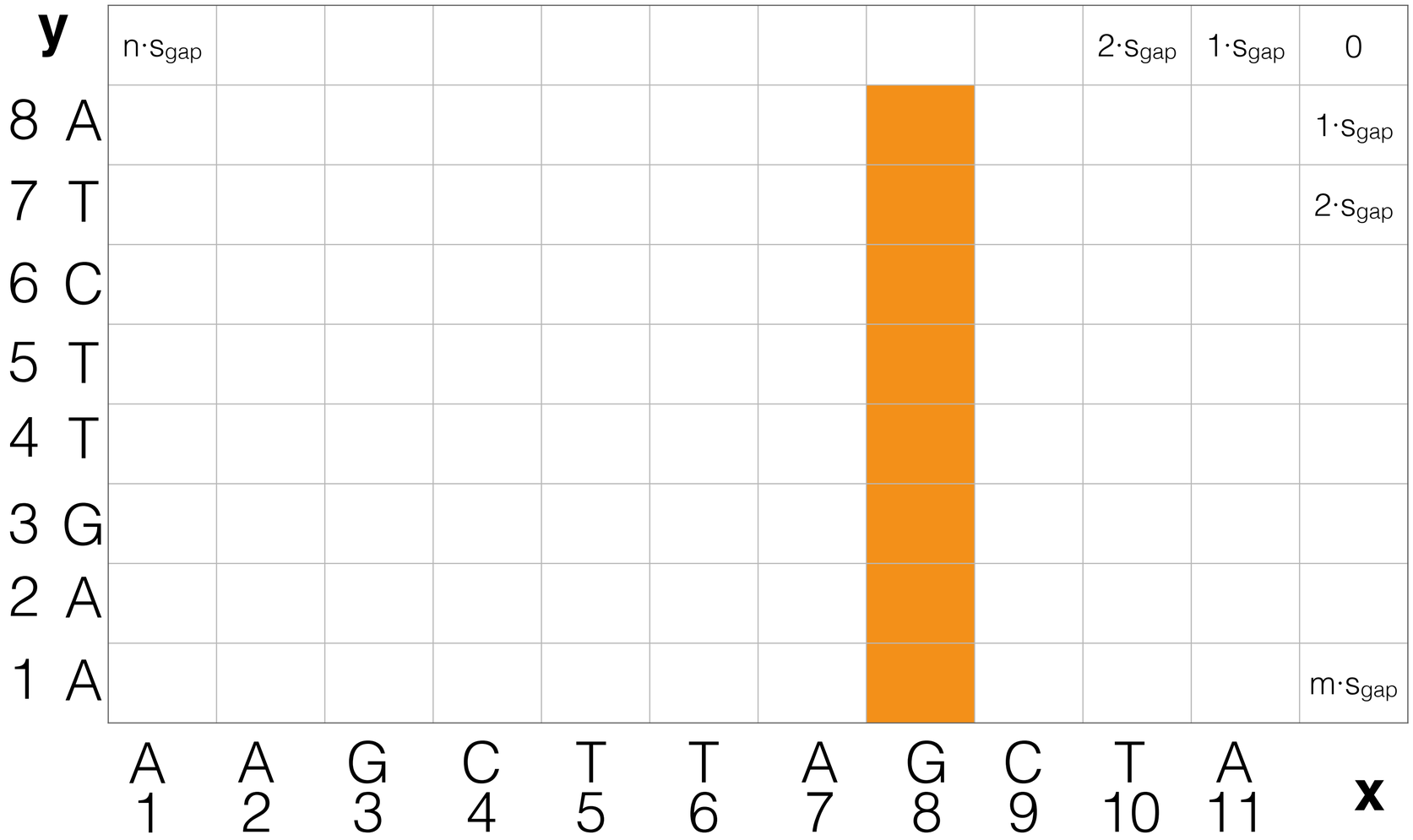


What about suffixes?



What about suffixes?

This lets us compute optimal score between a *suffix* of **x** with *all suffixes* of **y**



What about suffixes?

Prefixes (forward):

$$\mathrm{OPT}\,[i,j] = \max \begin{cases} \mathrm{score}\,(x_i, y_j) + \mathrm{OPT'}\,[i-1, j-1] \\ \mathrm{gap} + \mathrm{OPT}\,[i, j-1] \\ \mathrm{gap} + \mathrm{OPT}\,[i-1, j] \end{cases}$$

Suffixes (backward):

$$\mathrm{OPT'}\,[i,j] = \max \begin{cases} \mathrm{score}\,(x_{i+1}, y_{j+1}) + \mathrm{OPT'}\,[i+1, j+1] \\ \mathrm{gap} + \mathrm{OPT'}\,[i, j+1] \\ \mathrm{gap} + \mathrm{OPT'}\,[i+1, j] \end{cases}$$

This lets us build up optimal alignments for increasing length suffixes of **x** and **y**

What about suffixes?

Prefixes (forward):

$$\mathrm{OPT}\,[i,j] = \max \begin{cases} \mathrm{score}\,(x_i, y_j) + \mathrm{OPT'}\,[i-1, j-1] \\ \mathrm{gap} + \mathrm{OPT}\,[i, j-1] \\ \mathrm{gap} + \mathrm{OPT}\,[i-1, j] \end{cases}$$

Suffixes (backward):

$$\mathrm{OPT'}\,[i,j] = \max \begin{cases} \mathrm{score}\,(x_{i+1}, y_{j+1}) + \mathrm{OPT'}\,[i+1, j+1] \\ \mathrm{gap} + \mathrm{OPT'}\,[i, j+1] \\ \mathrm{gap} + \mathrm{OPT'}\,[i+1, j] \end{cases}$$

This lets us build up optimal alignments for increasing length suffixes of **x** and **y**

What about suffixes?

Prefixes (forward):

$$\mathrm{OPT}\,[i,j] = \max \begin{cases} \mathrm{score}\,(x_i, y_j) + \mathrm{OPT'}\,[i-1, j-1] \\ \mathrm{gap} + \mathrm{OPT}\,[i, j-1] \\ \mathrm{gap} + \mathrm{OPT}\,[i-1, j] \end{cases}$$

Suffixes (backward):

$$\mathrm{OPT'}\,[i,j] = \max \begin{cases} \mathrm{score}\,(x_{i+1}, y_{j+1}) + \mathrm{OPT'}\,[i+1, j+1] \\ \mathrm{gap} + \mathrm{OPT'}\,[i, j+1] \\ \mathrm{gap} + \mathrm{OPT'}\,[i+1, j] \end{cases}$$

note: the slight change in indexing here. It will make writing our solution easier.

Finding the optimal alignment

How does this help us compute the optimal alignment in linear space?

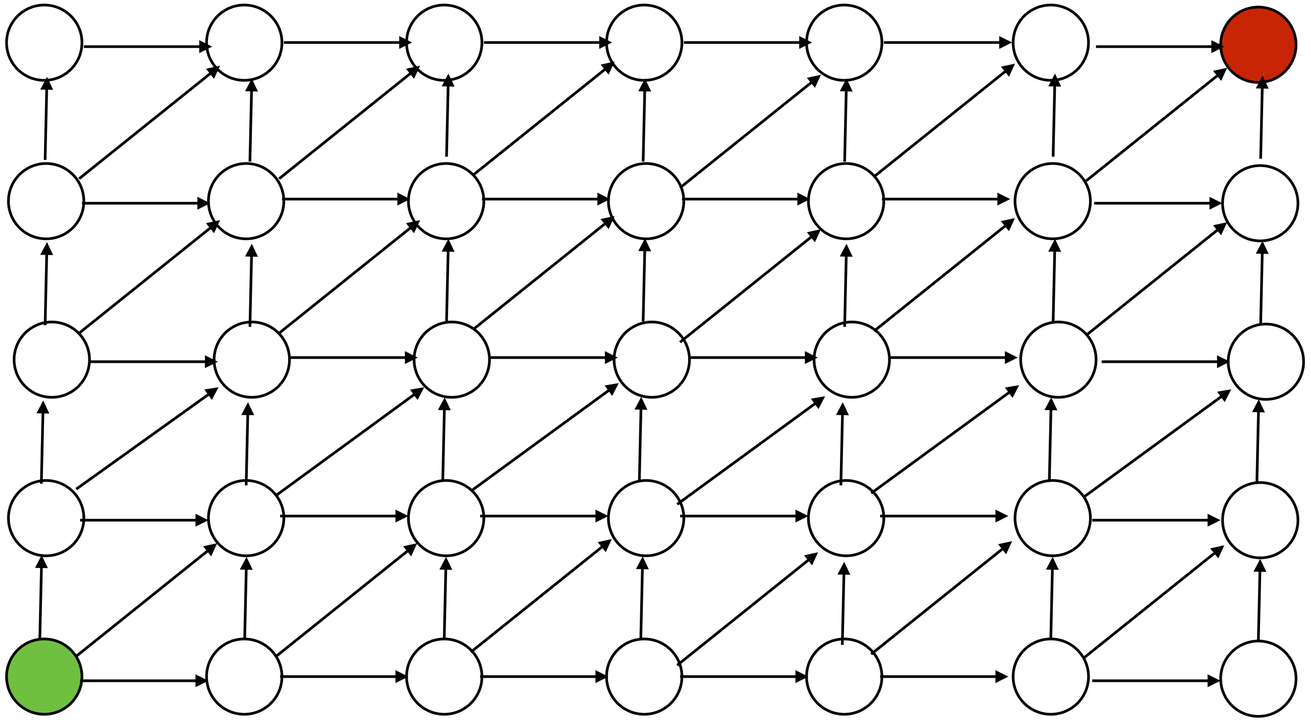
Algorithmic idea: Combine both dynamic programs using *divide-and-conquer*

Divide-and-conquer splits a problem into smaller sub-problems and combines the results (much like DP).

Examples: MergeSort & Karatsuba multiplication

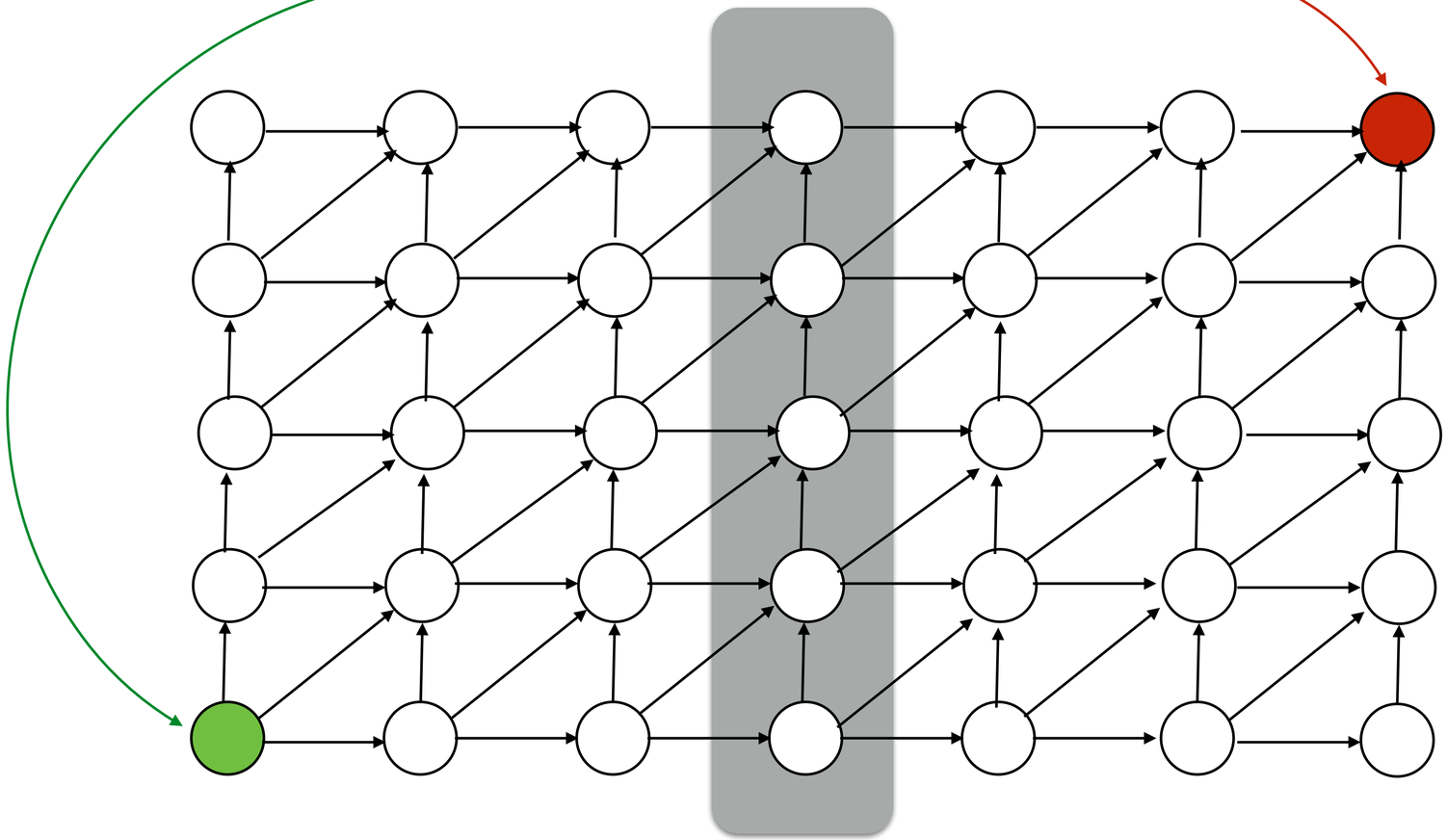
Think about this in “graph” land

What do we know about the structure of the optimal path in our “edit-DAG”?



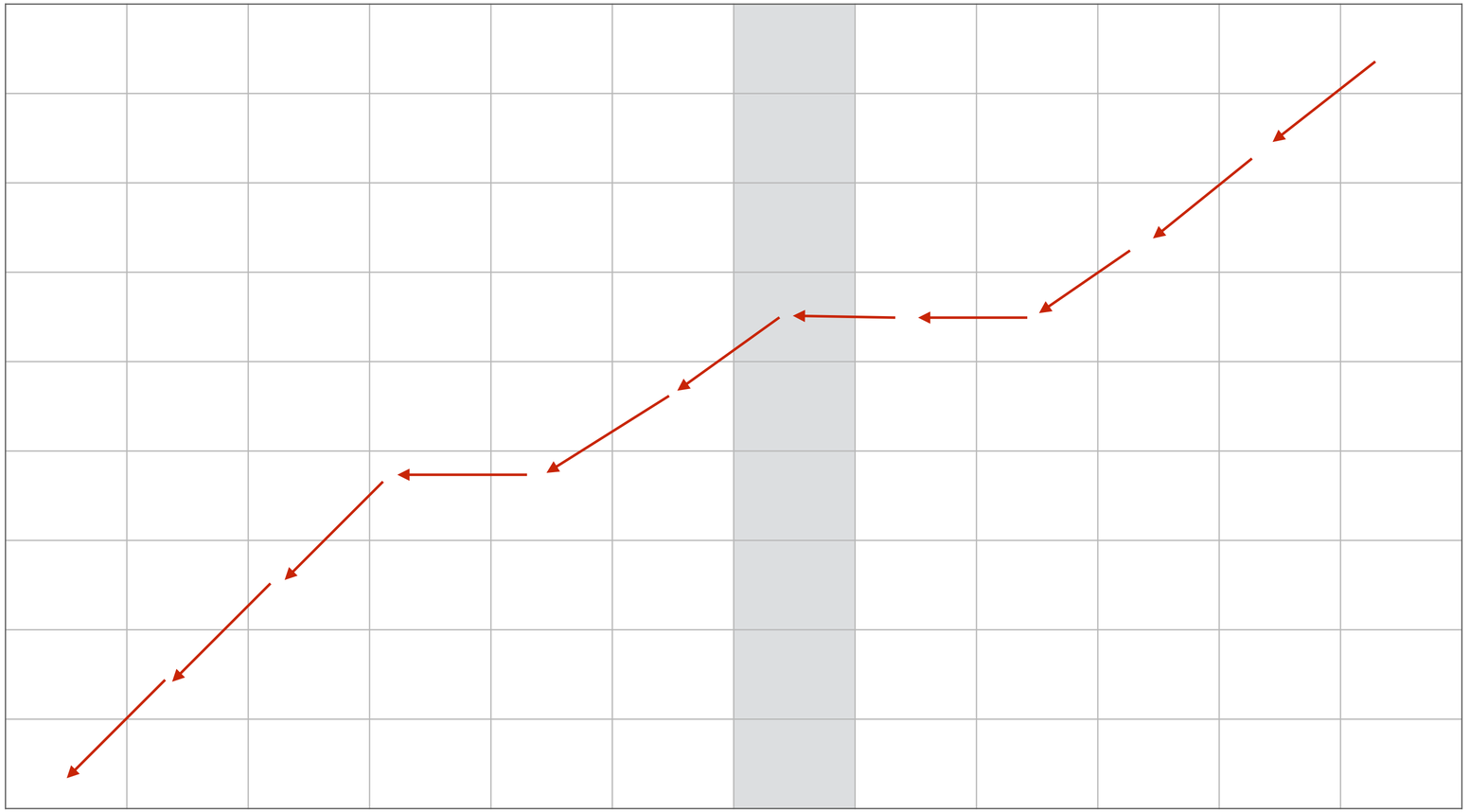
Think about this in “graph” land

Can’t get from here to there without passing through the middle.



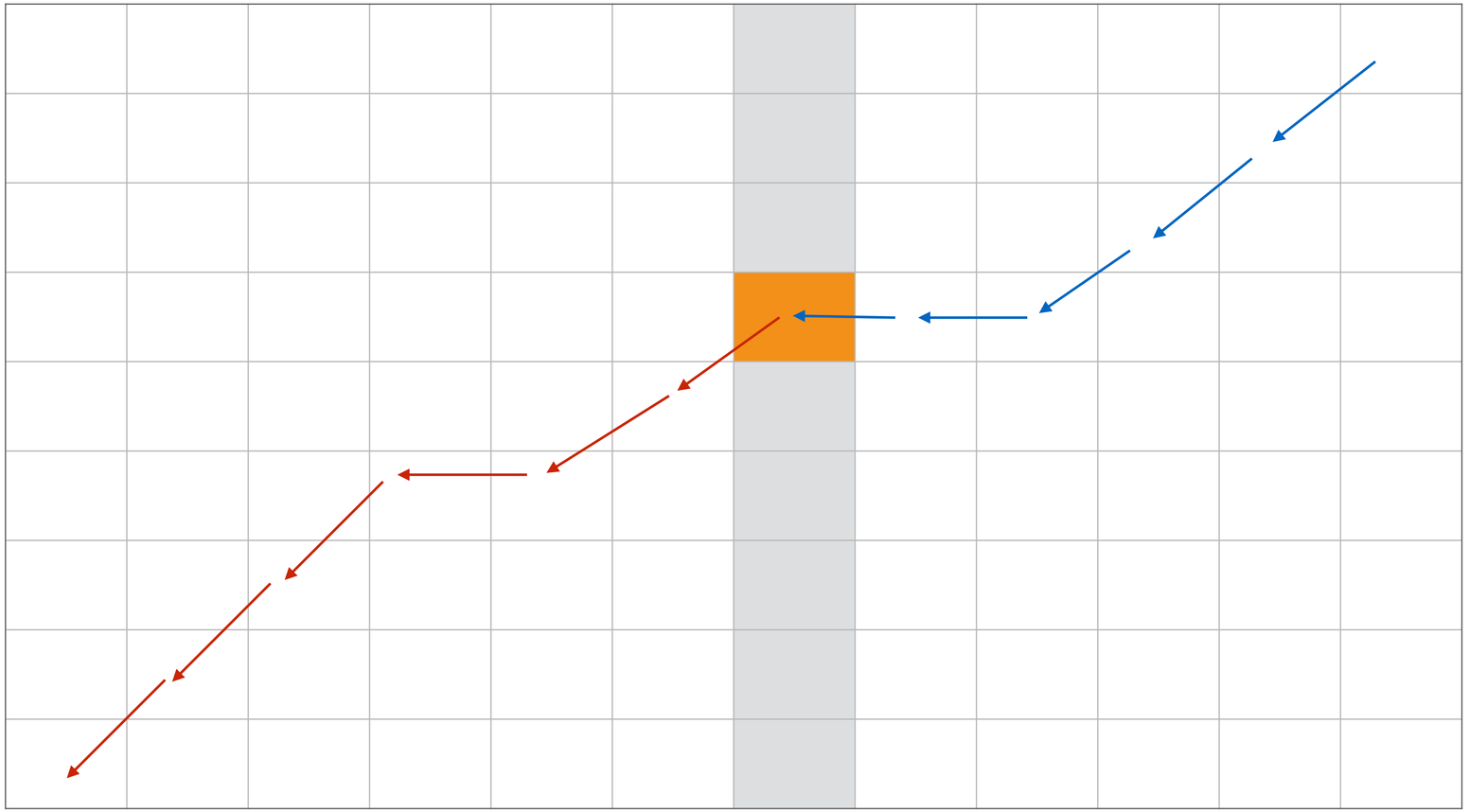
Finding the optimal alignment

Consider the middle column — we *know* that the optimal aln. must use some cell in this column; which one?



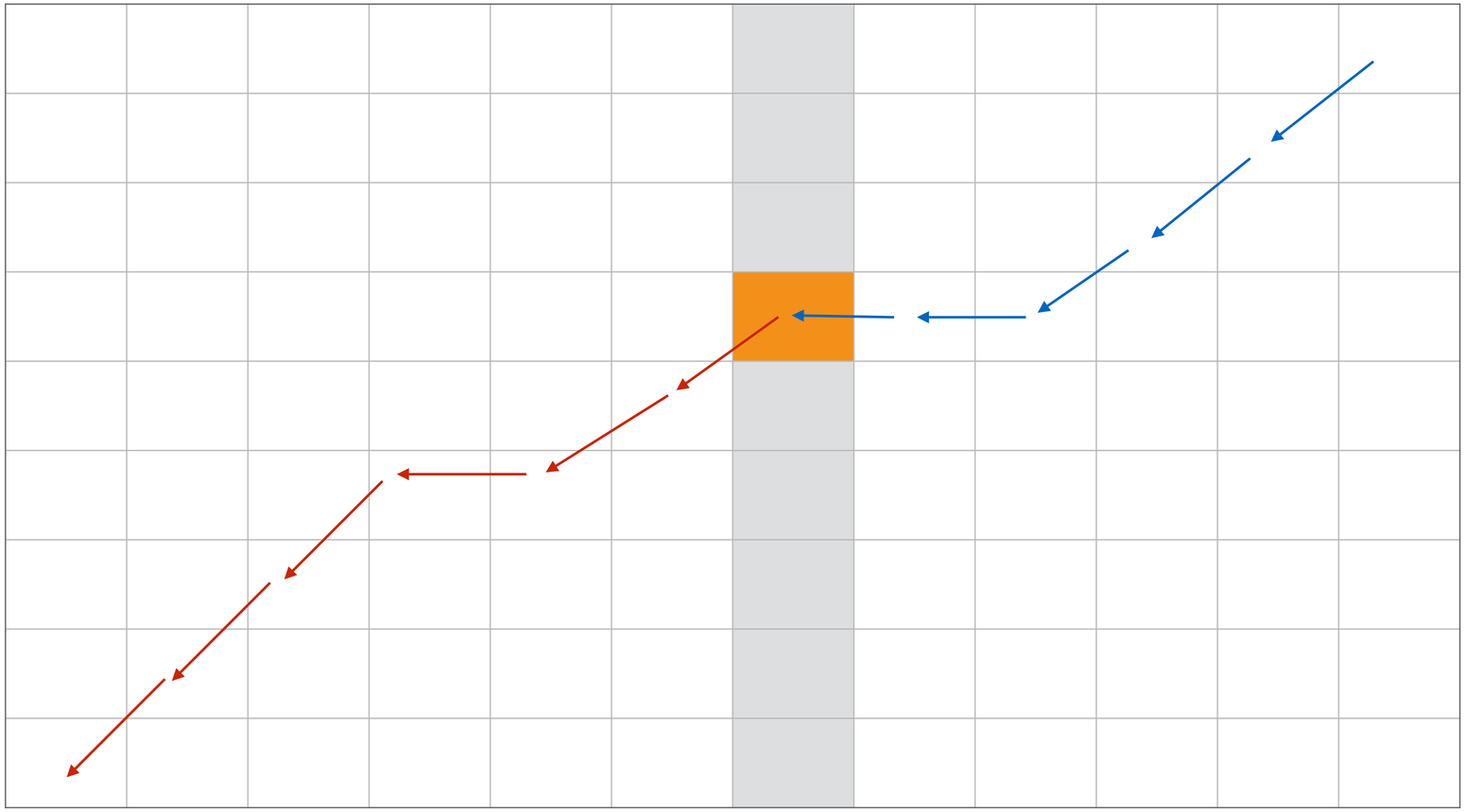
Finding the optimal alignment

It uses the cell (i,j) such that OPT[i,j] + OPT'[i,j] has the **highest score**. Equivalently, the *best path* uses some vertex *v* in the middle col. and glues together the best paths from the source *to* v and *from* v to the sink.



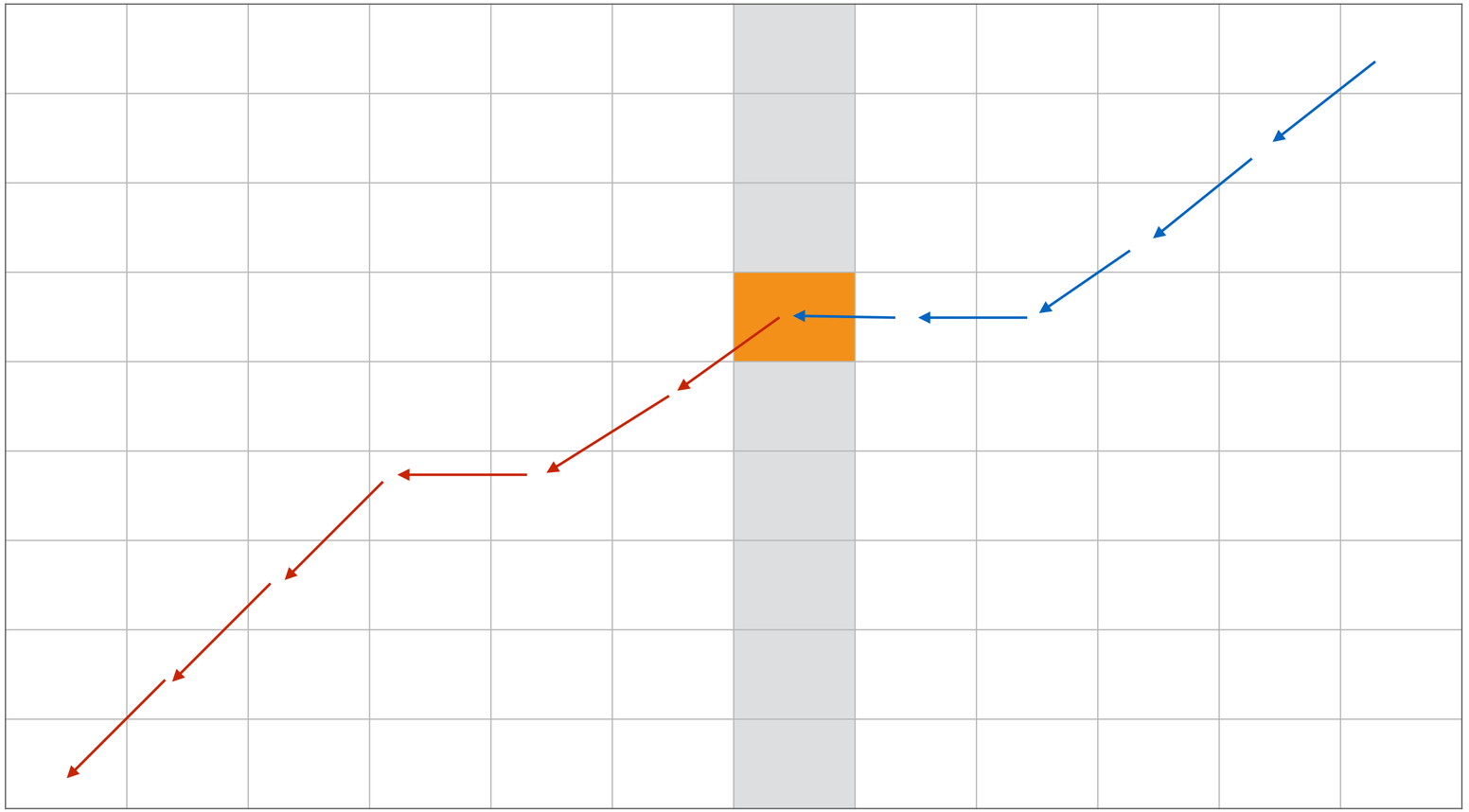
Finding the optimal alignment

Claim: OPT[i,j] and OPT'[i,j] can be computed in linear space using the trick from above for finding an optimal **score** in linear space



Algorithmic Idea

Devise a D&C algorithm that finds the optimal alignment path recursively, using the space-efficient scoring algorithm for each subproblem.



D&C Alignment

```
DCAlignment(x, y):
    n = |x|
    m = |y|
    if m <= 2 or n <= 2:
        use “normal” DP to compute OPT(x, y)
    compute space-efficient OPT(x[1:n/2], y)
    compute space-efficient OPT’(x[n/2+1:n], y)
    let q be the index maximizing OPT[n/2,q] + OPT’[n/2,q]
    add back pointer of (n/2,q) to the optimal alignment P
    DCAlignment(x[1:n/2], y[1:q])
    DCAlignment(x[n/2+1:n], y[q+1:m])
    return P
```

Adopted from “Algorithm Design” Kleinberg & Tardos (Ch. 6.7 pg 288)

D&C Alignment

How can we show that this entire process still takes quadratic time?

Let T(n,m) be the running time on strings **x** and **y** of length n and m, respectively. We have:

$$T(n,m) \leq cnm + T(n/2, q) + T(n/2, m-q)$$

`DCAlignment(x[1:n/2], y[1:q])`

`DCAlignment(x[n/2+1:n], y[q+1:m])`

with base cases:

$T(n,2) \leq cn$

$T(2,m) \leq cm$

Adopted from “Algorithm Design” Kleinberg & Tardos (Ch. 6.7 pg 289 — 290)

D&C Alignment

Base:

$T(n,2) \leq cn$

$T(2,m) \leq cm$

Inductive:

$T(n,m) \leq cnm + T(n/2, q) + T(n/2, m-q)$

Problem: we don't know what q is. First, assume both **x** and **y** have length n and q=n/2
(will remove this restriction later)

$T(n) \leq 2T(n/2) + cn^2$

This recursion solves as $T(n) = O(n^2)$

Leads us to guess T(n,m) grows like O(nm)

Adopted from "Algorithm Design" Kleinberg & Tardos (Ch. 6.7 pg 289 — 290)

Smarter Induction

Base:

$T(n,2) \leq cn$

$T(2,m) \leq cm$

Inductive:

$T(n,m) \leq knm$

Proof:

$$T(n,m) \leq cnm + T(n/2, q) + T(n/2, m-q)$$
$$\leq cnm + kqn/2 + k(m-q)n/2$$
$$\leq cnm + kqn/2 + kmn/2 - kqn/2$$
$$= [c+(k/2)]\, mn$$

Thus, our proof holds if $k=2c$, and $T(n,m) = O(nm)$ QED

Adopted from “Algorithm Design” Kleinberg & Tardos (Ch. 6.7 pg 289 — 290)

Conclusion

Trivially, we can compute the *cost* of an optimal alignment in linear space

By arranging subproblems intelligently we can define a "reverse" DP that works on suffixes instead of prefixes

Combining the "forward" and "reverse" DP using a divide and conquer technique, we can compute the optimal *solution* (not just the score) in linear space.

This still only takes O(nm) time; constant factor more work than the "forward"-only algorithm.

Doing better (in time) *in practice*

Can we do better *in practice*?

What about when we know that the edit distance is small (say e << mn)?
What about when we only care about edit distances < some threshold?

Yes!

JOURNAL ARTICLE

Optimal gap-affine alignment in $O(s)$ space

Santiago Marco-Sola ✉, Jordan M Eizenga, Andrea Guarracino, Benedict Paten, Erik Garrison, Miquel Moreto

Bioinformatics, Volume 39, Issue 2, February 2023, btad074,
https://doi.org/10.1093/bioinformatics/btad074
Published: 07 February 2023 **Article history** ▾

O(ns) time and O(s) space where s is the score of the *optimal alignment*!

https://doi.org/10.1093/bioinformatics/btad074

Exact global alignment using A* with seed heuristic and match pruning

Ragnar Groot Koerkamp *,† **and Pesho Ivanov** *,†

Department of Computer Science, ETH Zurich, Switzerland

*To whom correspondence should be addressed.
†These authors contributed equally to this work.

←

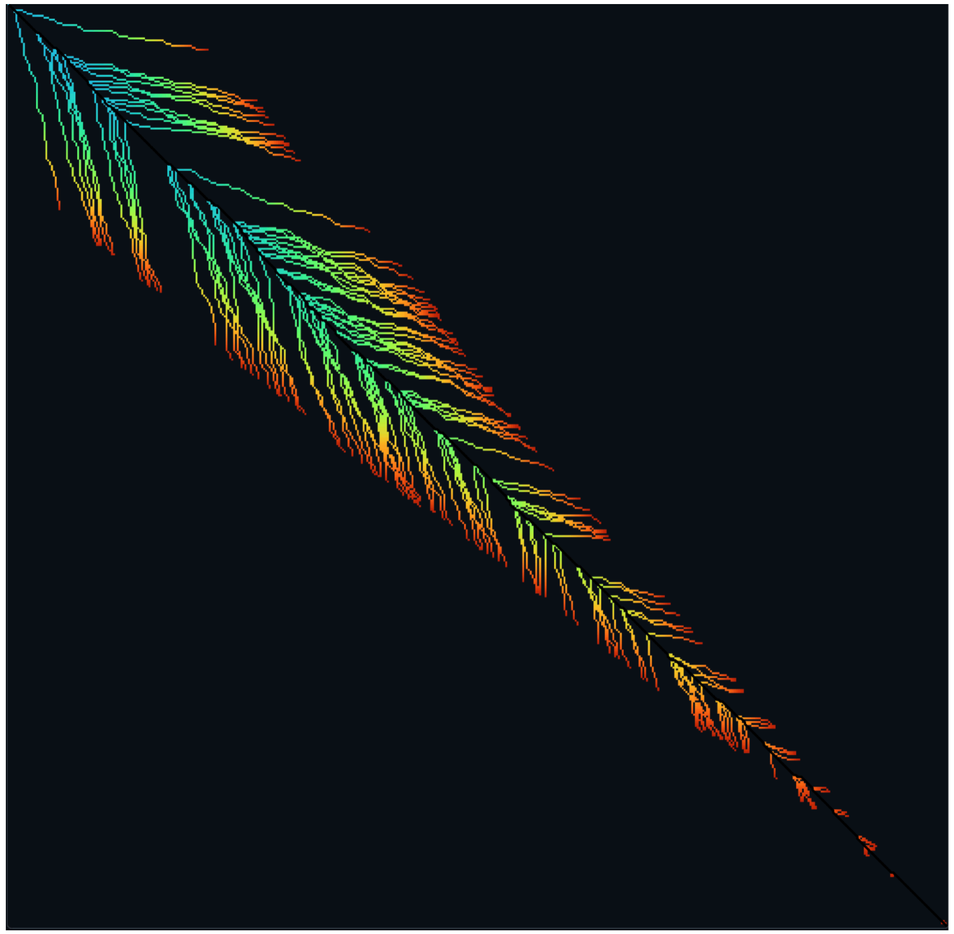
Practically even fewer comparisons than BiWFA

https://www.biorxiv.org/content/10.1101/2022.09.19.508631v2

Doing better (in time) *in practice*

Beautiful motivation:

https://github.com/RagnarGrootKoerkamp/astar-pairwise-aligner



Diagonal transition (WFA)

Bonus : General & Affine Gap Penalties

General Gap Penalties

```
AAAGAATTCA
A-A-A-T-CA
```

vs.

```
AAAGAATTCA
AAA----TCA
```

These have the same score, but the second one is often more plausible.

A single insertion of "GAAT" into the first string could change it into the second — Biologically, this is much more likely as **x** could be transformed into **y** in "one fell swoop".

- Currently, the score of a run of k gaps is $s_{gap} \times k$
- It might be more realistic to support general gap penalty, so that the score of a run of k gaps is $|\textbf{gscore}(k)| < |(s_{gap} \times k)|$.
- Then, the optimization will prefer to group gaps together.

*

General Gap Penalties — The Problem

```
AAAGAATTCA
A-A-A-T-CA
```

vs.

```
AAAGAATTCA
AAA----TCA
```

Previous DP no longer works with general gap penalties.

Why?

*

General Gap Penalties — The Problem

```
AAAGAATTCA
A-A-A-T-CA
```

vs.

```
AAAGAATTCA
AAA----TCA
```

The score of the *last character* depends on *details* of the previous alignment:

```
AAAGAAC
AAA---
```

vs.

```
AAAGAATC
AAA---
```

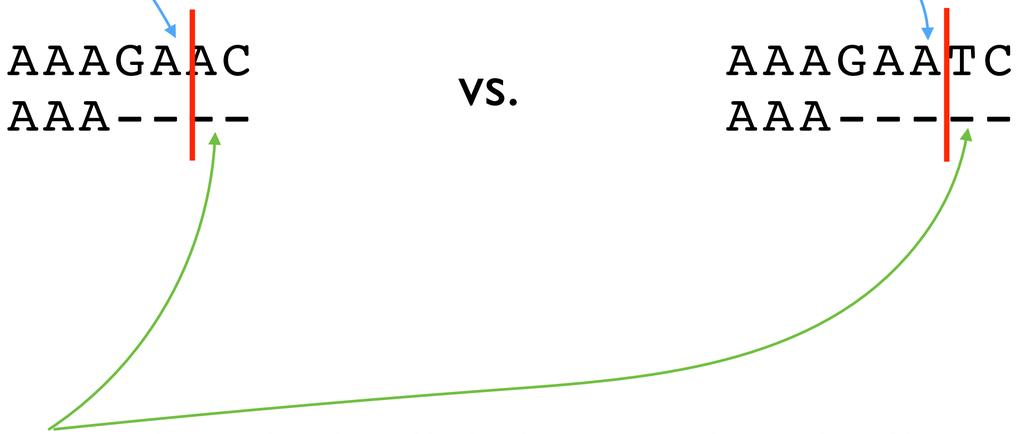
We need to "know" how long a final run of gaps is in order to give a score to the last subproblem.

*

General Gap Penalties — The Problem

The score of the *last character* depends on *details* of the previous alignment:

Knowing the optimal alignment at the substring ending here.



Doesn't let us simply build the optimal alignment ending here.

*

Three Matrices

We now keep 3 different matrices:

M(i,j) = score of best alignment of x[1..i] and y[1..j] ending with a character-character **match or mismatch**.

X(i,j) = score of best alignment of x[1..i] and y[1..j] ending with a **gap in X.**

Y(i,j) = score of best alignment of x[1..i] and y[1..j] ending with a **gap in Y.**

$$\mathrm{M}(i,j) = \mathrm{score}(x_i, y_j) + \max \begin{cases} \mathrm{M}(i-1, j-1) \\ \mathrm{X}(i-1, j-1) \\ \mathrm{Y}(i-1, j-1) \end{cases}$$

$$\mathrm{X}(i,j) = \max \begin{cases} \mathrm{M}(i, j-k) + \mathrm{gscore}(k) & \text{for } 1 \le k \le j \\ \mathrm{Y}(i, j-k) + \mathrm{gscore}(k) & \text{for } 1 \le k \le j \end{cases}$$

$$\mathrm{Y}(i,j) = \max \begin{cases} \mathrm{M}(i-k, j) + \mathrm{gscore}(k) & \text{for } 1 \le k \le i \\ \mathrm{X}(i-k, j) + \mathrm{gscore}(k) & \text{for } 1 \le k \le i \end{cases}$$

*

The M Matrix

We now keep 3 different matrices:

M(i,j) = score of best alignment of x[1..i] and y[1..j] ending with a character-character **match or mismatch**.

X(i,j) = score of best alignment of x[1..i] and y[1..j] ending with a **gap in X**.

Y(i,j) = score of best alignment of x[1..i] and y[1..j] ending with a **gap in Y**.

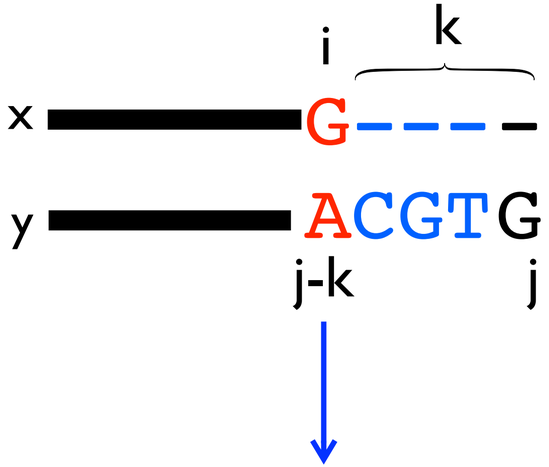
By definition, alignment ends in a match/mismatch.

$$\mathrm{M}(i,j) = \mathrm{score}(x_i, y_j) + \max \begin{cases} \mathrm{M}(i-1, j-1) \\ \mathrm{X}(i-1, j-1) \\ \mathrm{Y}(i-1, j-1) \end{cases}$$

Any kind of alignment is allowed before the match/mismatch.

*

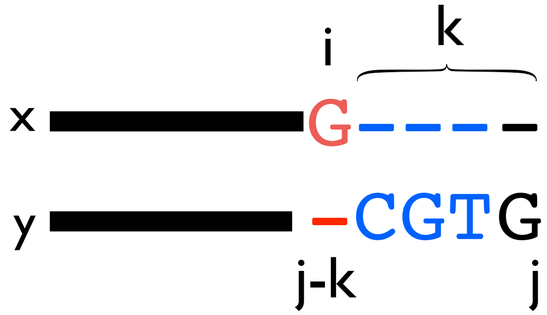
The X (and Y) matrices



k decides how long to make the gap.

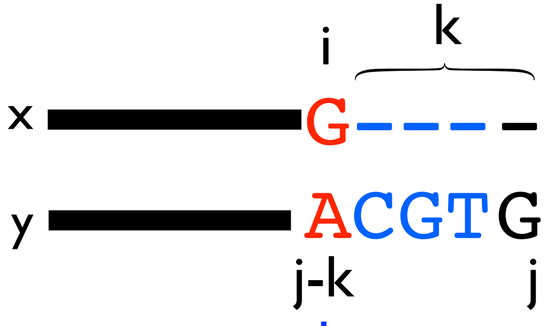
We have to make the whole gap at once in order to know how to score it.

$$\mathrm{X}(i,j) = \max \begin{cases} \mathrm{M}(i, j-k) + \mathrm{gscore}(k) & \text{for } 1 \le k \le j \\ \mathrm{Y}(i, j-k) + \mathrm{gscore}(k) & \text{for } 1 \le k \le j \end{cases}$$



*

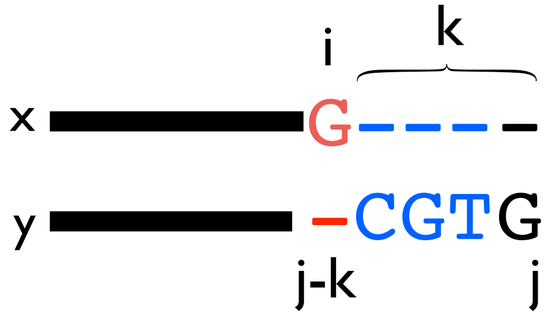
The X (and Y) matrices



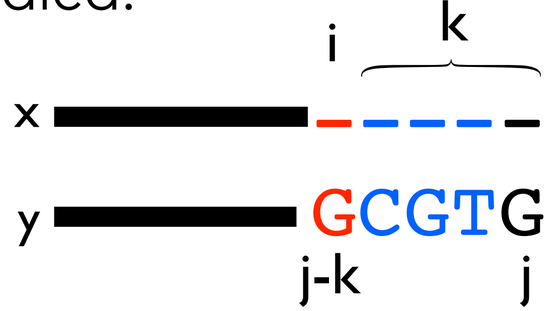
k decides how long to make the gap.

We have to make the whole gap at once in order to know how to score it.

$$\mathrm{X}(i,j) = \max \begin{cases} \mathrm{M}(i, j-k) + \mathrm{gscore}(k) & \text{for } 1 \le k \le j \\ \mathrm{Y}(i, j-k) + \mathrm{gscore}(k) & \text{for } 1 \le k \le j \end{cases}$$



This case is automatically handled.



*

Running Time for Gap Penalties

$$\mathrm{M}(i,j) = \mathrm{score}(x_i, y_j) + \max \begin{cases} \mathrm{M}(i-1, j-1) \\ \mathrm{X}(i-1, j-1) \\ \mathrm{Y}(i-1, j-1) \end{cases}$$

$$\mathrm{X}(i,j) = \max \begin{cases} \mathrm{M}(i, j-k) + \mathrm{gscore}(k) & \text{for } 1 \le k \le j \\ \mathrm{Y}(i, j-k) + \mathrm{gscore}(k) & \text{for } 1 \le k \le j \end{cases}$$

$$\mathrm{Y}(i,j) = \max \begin{cases} \mathrm{M}(i-k, j) + \mathrm{gscore}(k) & \text{for } 1 \le k \le i \\ \mathrm{X}(i-k, j) + \mathrm{gscore}(k) & \text{for } 1 \le k \le i \end{cases}$$

Final score is max {M(n,m), X(n,m), Y(n,m)}.

How do you do the traceback?

Runtime:

- Assume |X| = |Y| = n for simplicity: $3n^2$ subproblems
- $2n^2$ subproblems take O(n) time to solve (**because we have to try all k**)

⇒ $O(n^3)$ total time

*

Affine Gap Penalties

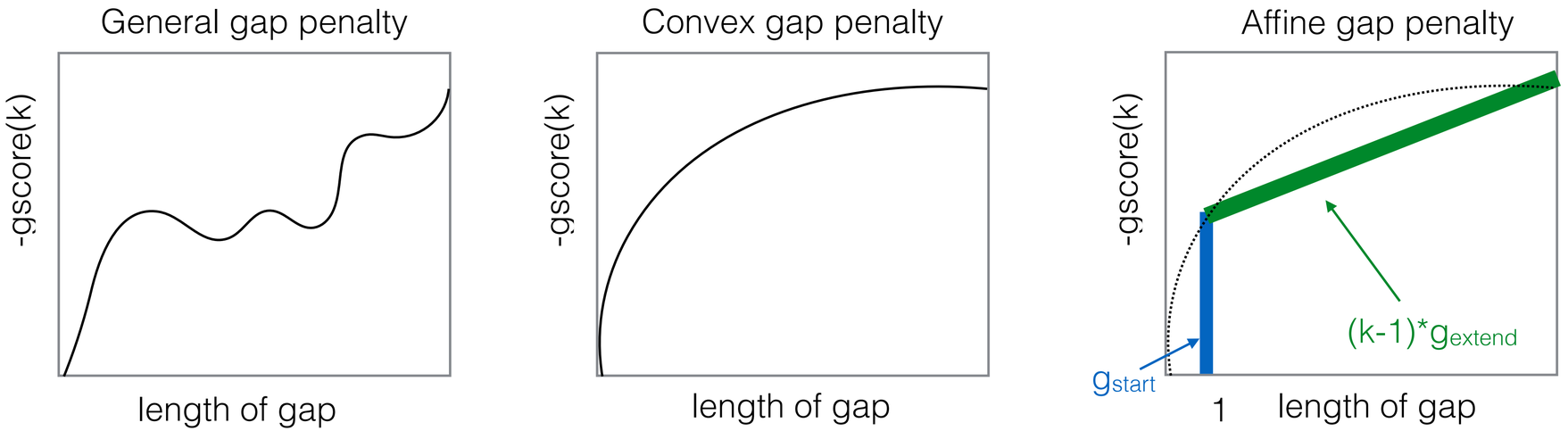
- $O(n^3)$ for general gap penalties is usually too slow...
- We can still encourage spaces to group together using a special case of general penalties called *affine gap penalties*:

  g_{start} = the cost of starting a gap

  g_{extend} = the cost of extending a gap by one more space

  $gscore(k) = g_{start} + (k-1) \times g_{extend}$

less restrictive ⇒ more restrictive

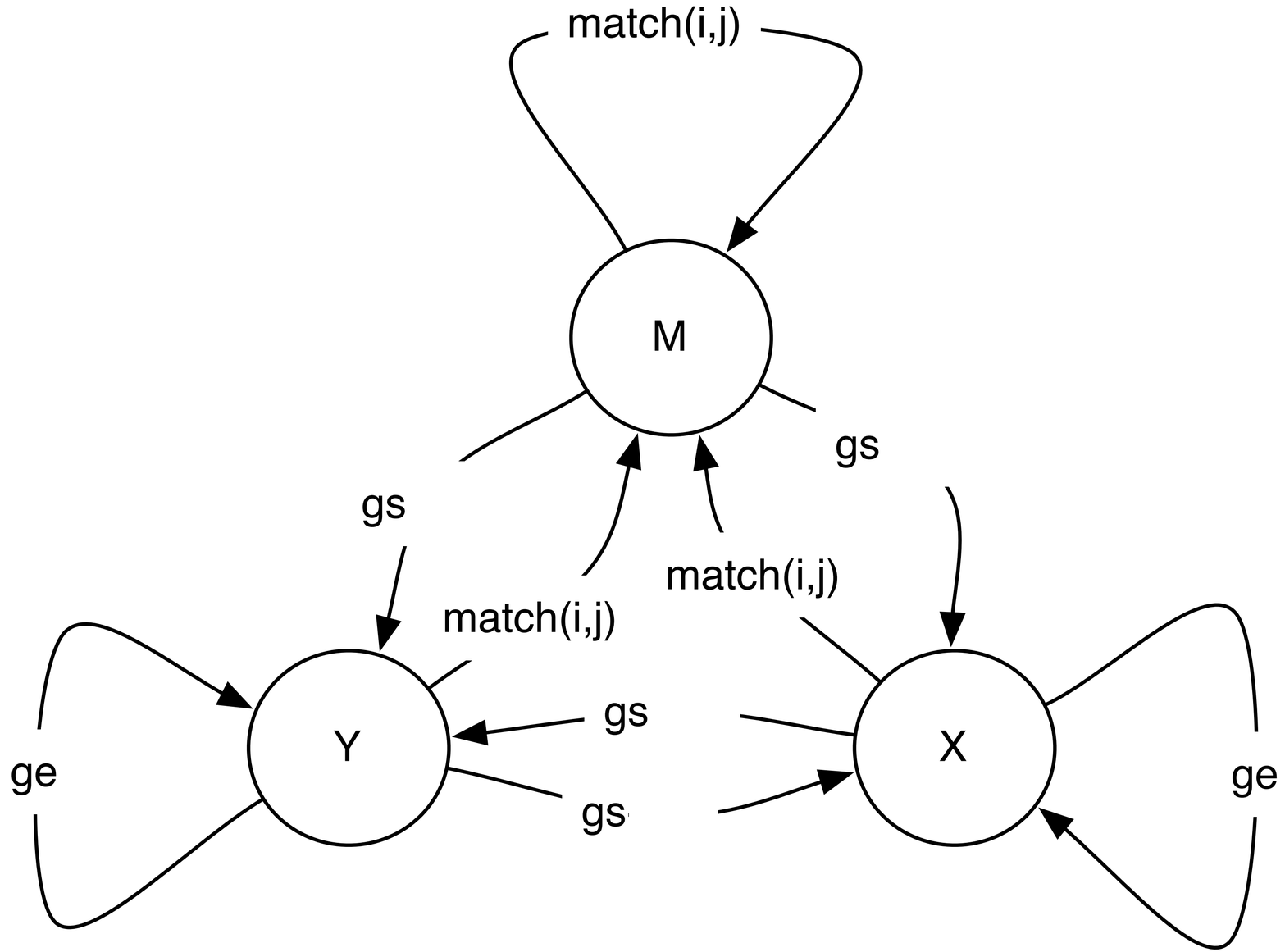


Benefit of Affine Gap Penalties

- Same idea of using 3 matrices, but now we *don't need to search over all gap lengths*, we just have to know whether we are **starting a new gap** or **not**.

*

Affine Gap as Finite State Machine



*

Affine Gap Penalties

$$\mathrm{M}(i,j) = \mathrm{score}(x_i, y_i) + \max \begin{cases} \mathrm{M}(i-1, j-1) \\ \mathrm{X}(i-1, j-1) \\ \mathrm{Y}(i-1, j-1) \end{cases}$$

(mis)match between x and y

If previous alignment ends in (mis)match, this is a new gap

$$\mathrm{X}(i,j) = \max \begin{cases} g_{\mathrm{start}} + \mathrm{M}(i, j-1) \\ g_{\mathrm{extend}} + \mathrm{X}(i, j-1) \\ g_{\mathrm{start}} + \mathrm{Y}(i, j-1) \end{cases}$$

gap in x

If we're using the X matrix, then we're extending a gap.

$$\mathrm{Y}(i,j) = \max \begin{cases} g_{\mathrm{start}} + \mathrm{M}(i-1, j) \\ g_{\mathrm{start}} + \mathrm{X}(i-1, j) \\ g_{\mathrm{extend}} + \mathrm{Y}(i-1, j) \end{cases}$$

gap in y

If we're using the Y matrix, then we're starting a new gap in this string.

*

Affine Base Cases (Global)

- M(0, *i*) = "score of best alignment between 0 characters of *x* and *i* characters of *y* that ends in a match" = -∞ because no such alignment can exist.

- X(0, *i*) = "score of best alignment between 0 characters of *x* and *i* characters of *y* that ends in a gap in *x*" = gap_start + (i-1) × gap_extend because this alignment looks like:

```
---------
yyyyyyyyy
```

- X(i, 0) = "score of best alignment between i characters of x and 0 characters of y that ends in a gap in X" = -∞

```
xxxxxxxxx-
----------
```

← not allowed

- M(i, 0) = M(0, i) and Y(0, i) and Y(i, 0) are computed using the same logic as X(i, 0) and X(0, i)

*

Affine Gap Runtime

- 3*mn* subproblems
- Each one takes **constant** time
- Total runtime O(*mn*):
  - back to the run time of the basic running time.

Traceback

- Arrows now can point **between** matrices.
- The possible arrows are given, as usual, by the recurrence.
  - E.g. What arrows are possible leaving a cell in the M matrix?

*

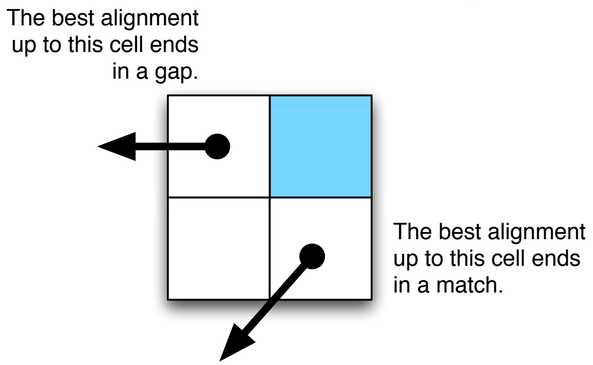
Why do you "need" 3 functions?

- Alternative WRONG algorithm:

```
M(i,j) = max(
    M(i-1, j-1) + cost(x_i, y_j),
    M(i-1, j) +(g_start if Arrow(i-1, j) != ←, else g_extend),
    M(j, i-1) + (g_start if Arrow(i, j-1) != ↓ , else g_extend)
)
```

WRONG Intuition: we only need to know whether we are starting a gap or extending a gap.

The arrows coming out of each subproblem tell us how the best alignment ends, so we can use them to decide if we are starting a new gap.



PROBLEM: The best alignment for strings x[1..i] and y[1..j] doesn't have to be used in the best alignment between x[1..i+1] and y[1..j+1]

*

Why 3 Matrices: Example

match = 5, mismatch = -2, gap = -1, gap_start = -10

x=CARTS, y=CAT

```
CART
CA-T
```

OPT(4, 3) = optimal score = 15 - 10 = 5

```
CARTS
CA-T-
```

WRONG(5, 3) = 15 - 10 - 10 = -5

```
CARTS
CAT--
```

OPT(5, 3) = 10 - 2 - 10 - 1 = -3

this is why we need to keep the X and Y matrices around.
they tell us the score of ending with a gap in one of the sequences.

*

Recap

- Local alignment: extra "0" case.
- General gap penalties require 3 matrices and $O(n^3)$ time.
- Affine gap penalties require 3 matrices, but only $O(n^2)$ time.

*